

SanPyCAD Brep

Python Function Reference Guide

213 functions in **backend/brep.py** · 189 exposed to scripts · 24 not yet exposed

Generated September 21, 2026

About This Guide

This guide documents every public Python function defined in `backend/brep.py`, the real B-rep (build123d/OCCT) geometry kernel behind SanPyCAD Brep. It was generated directly from the current source: every signature, docstring, and source line below reflects the code as it exists today, not a hand-maintained summary.

Exposed vs. not-yet-exposed

A function is **exposed to scripts** when it is reachable by name from the app's own Python script editor — i.e. it is listed in `python_eval.py`'s namespace-building code. 189 of the 213 functions catalogued here are exposed this way. The remaining 24 are fully implemented in `brep.py` but are not yet wired into that namespace, so a script cannot call them by name today even though the code exists. Each entry below is marked **Exposed to scripts** or **Not yet exposed to scripts** accordingly.

Source line references

Each entry lists a `backend/brep.py:LINE` reference pointing at the function's `def` line in the protected source, for anyone maintaining the codebase directly.

Used in examples

Where a function is called by name in one of the bundled example scripts (`drill.py`, `hollow_cylinder_with_star.py`, `hook.py`, `tap.py`), those files are listed as a quick pointer to a real, working usage. Most functions are not yet called from an example — that is a gap in the examples, not necessarily a sign the function is unused or unreliable.

Organization

Functions are grouped into 19 functional categories (below), each sorted alphabetically within the category. An alphabetical index of all function names with page numbers closes out the guide.

Table of Contents

Math & Angle Utilities (7)	11
ang()	11
ang3points()	11
ang_2lineccw()	11
ang_2linecw()	11
cw()	11
d2r()	11
r2d()	12
Points, Vectors & Lines (22)	12
c23()	12
c32()	12
closest_points_between_two_lines()	12
distanceOfPointFromLine()	12
fit_pline2line()	13
l_len()	13
l_lenv()	13
line2length()	13
line3d()	13
line_as_unit_vector()	14
line_as_vector()	14
mid_line()	14
mid_point()	14
min_d_points()	14
perpendicularProjectionOfPointOnLine()	14
point_in_polygon()	15
points()	15
remove_extra_points()	15
solid_line_vector()	15
sort_points()	16
surface_line_vector()	16
to_points()	17

Arcs & Circles (17) 17

arc()	17
arc_2p()	17
arc_2p_3d()	17
arc_3p()	17
arc_3p_3d()	18
arc_edge()	18
cir_2p()	18
cir_3p()	18
circle()	18
cp_3p()	19
cp_arc()	19
cp_cir_3d()	19
extend_arc2d()	19
extend_arc3d()	19
tangent_arc()	19
tcct()	20
two_cir_tarc()	20

2D Profiles & Curve Generators (18) 20

bezier()	20
bspline_closed()	20
bspline_open()	20
cosinewave()	21
cr2dt()	21
cr3dt()	22
honeycomb()	22
interp_spline()	22
pies1()	23
sample_curve()	23
sec2vector()	23
section_view()	23
sinewave()	24
turtle2d()	24
turtle3d()	24
xyz()	25

xzc()	25
yzc()	25

Polygons, Wires & Faces (13) 25

c_polygon()	25
cap_wire()	25
concave_hull()	26
contiguous_chains()	26
convex_hull()	26
face_from_wire()	27
hull()	27
i_p2d()	27
mixed_polygon()	27
mixed_wire()	28
polygon()	28
square()	29
wire_from_edges()	29

Planes & Coordinate Frames (3) 30

equation_of_plane()	30
plane()	30
plane_from_equation()	31

Measurement & Analysis (11) 31

area()	31
bb()	31
bb2d()	31
cog()	31
draft_check()	31
draft_check_markers()	33
draft_check_points()	33
max_fillet_radius()	33
projected_area()	34
projected_area_mesh()	35
volume()	36

Primitive Solids (6)	36
box()	36
c_solid()	36
cube()	37
cylinder()	37
helix()	37
sphere()	38
 Extrude, Sweep, Loft & Revolve (9)	 38
extrude_wave2path()	38
linear_extrude()	38
loft()	39
loft_fillet()	39
p_line3d()	40
prism()	41
rotate_extrude()	41
sweep_sec2path()	42
swp_c()	44
 Surfaces (17)	 44
face_islands()	44
faces_facing()	45
faces_in_box()	45
faces_within_boundary()	46
fill_face_holes()	46
mirror_surface()	47
offset_face()	47
project_curve_on_face()	48
project_face_on_shape()	48
surface_cap_solid()	48
surface_from_2_waves()	49
surface_from_closed_rings()	50
surface_from_points_grid()	51
surface_from_points_grid_closed()	51
surface_normals()	52
surface_solid_closed_intersection()	52

surface_solid_open_intersection()	52
-----------------------------------	----

Fillets & Chamfers (8)
53

chamfer()	53
convert_3lines2fillet()	53
corner_n_radius_list()	54
fillet()	54
fillet3points()	55
organic_fillet()	55
solid_from_fillet()	56
solid_from_fillet_closed()	57

Offsetting & Shelling (5)
57

erode()	57
erosion_series()	58
offset()	58
offset_3d()	59
offset_shell()	60

Transformations (20)
60

axis_rot_1()	60
axis_rot_1_pts()	60
flip()	61
mirror()	61
mirror_line()	61
mirror_point()	61
resize_solid()	61
resize_solid_centered()	62
rot()	62
rot2d()	62
rot_p()	62
rotate()	63
scale()	63
translate()	63
xrot()	63
xrot4d()	63

yrot()	64
yrot4d()	64
zrot()	64
zrot4d()	64

Booleans & Assembly Operations (8) 64

difference()	64
group()	64
intersection()	65
split()	65
two_solids_intersection()	65
union()	66
union_fuzzy()	66
union_multi()	67

Mesh, Triangulation & Point-Cloud Solids (16) 67

mesh_boundary_loop()	67
mesh_cap_ribbon()	68
mesh_cap_solid()	69
mesh_split_surfaces()	70
mesh_split_triangles()	71
o_3dc_tri()	72
planarize_ring()	73
shape_outline_raster()	73
slice_mesh_to_rings()	74
smooth_mesh()	75
solid_from_closed_rings()	76
solid_from_faces()	77
solid_from_mesh_slices()	78
solid_from_wedge_grid()	79
to_mesh()	81
to_triangles()	81

Projection & Intersection Queries (10) 81

nlos()	81
normal_vector()	82

plos()	. 82
plos_v()	. 82
plos_v_1()	. 82
psos()	. 82
psos_n()	. 83
psos_v()	. 84
psos_v_1()	. 84
s_int1()	. 84

Edge / Face / Vertex Utilities (11) 84

close_faces_to_plane()	. 84
edges_in_box()	. 85
label_edges()	. 86
label_faces()	. 86
seam_edges()	. 87
seg()	. 88
shell_from_faces()	. 88
show_edges()	. 89
slider_from_faces()	. 89
vertex()	. 91
wrap_around()	. 91

Import, Export & Drawings (8) 91

export_brep()	. 91
export_drawing_svg()	. 91
export_step()	. 92
export_stl()	. 92
import_brep()	. 92
import_step()	. 92
technical_drawing_views()	. 93
text()	. 94

Application-Specific Helpers (4) 95

pick_key_stations()	. 95
slider_housing()	. 95
thicken()	. 97

track_points()	97
----------------	----

Math & Angle Utilities (7)

ang()

```
ang(x, y)
```

Exposed to scripts | backend/brep.py:2635 | Used in: —

The polar angle (degrees, 0-360) of the 2D vector (x, y) measured from the origin -- ocad.py's own ang(), ported. Same computation as math.degrees(math.atan2(y, x)), just wrapped into 0-360 instead of -180..180 (and the exact same math already used internally throughout this file, e.g. by arc_3p()/ang_2lineccw()/ etc, via the internal _polar_angle_deg() helper -- ang() just exposes that under its original name for direct use in your own scripts):

```
p1, p2 = [3, 4], [-3, 2]
v = [p2[0] - p1[0], p2[1] - p1[1]]
a = ang(v[0], v[1])
```

ang3points()

```
ang3points(p0, p1, p2)
```

Exposed to scripts | backend/brep.py:2677 | Used in: —

The interior angle (degrees, 0-360 -- not clamped to 0-180, so it tells you which way the path bends) at the middle point of the 3-point path p0-p1-p2. 2D.

ang_2lineccw()

```
ang_2lineccw(p0, p1, p2)
```

Exposed to scripts | backend/brep.py:2653 | Used in: —

The counter-clockwise angle (degrees, 0-360) of the ray p0->p2 measured from the ray p0->p1 -- "how far ccw do you turn from pointing at p1 to pointing at p2, both from p0". 2D.

ang_2linecw()

```
ang_2linecw(p0, p1, p2)
```

Exposed to scripts | backend/brep.py:2668 | Used in: —

Clockwise counterpart of ang_2lineccw().

cw()

```
cw(sec)
```

Not yet exposed to scripts | backend/brep.py:3168 | Used in: —

Whether the closed 2D point list `sec` winds clockwise (returns 1) or counter-clockwise (returns -1). ocad.py's own cw(), but re-implemented here via the signed-area (shoelace) formula instead of ocad's own ray-casting approach -- mathematically exact for any simple (non-self-intersecting) polygon, convex or concave, and considerably simpler. Same return convention as the original:

```
cw([[0,0],[10,0],[5,5]]) -> -1 (counter-clockwise)
cw([[0,0],[10,0],[5,-5]]) -> 1 (clockwise)
```

d2r()

```
d2r(d)
```

Not yet exposed to scripts | backend/brep.py:3137 | Used in: —

Converts degrees to radians -- ocad.py's own d2r(), ported. Works on a single number or a plain list of numbers.

r2d()

```
r2d(r)
```

Not yet exposed to scripts | backend/brep.py:3129 | Used in: —

Converts radians to degrees -- ocad.py's own r2d(), ported. Works on a single number or a plain list of numbers.

Points, Vectors & Lines (22)

c23()

```
c23(pts)
```

Exposed to scripts | backend/brep.py:2841 | Used in: —

Converts a 2D point list to 3D by padding each point with z=0 -- ocad.py's own c23()/c2t3(), ported. Points that are already 3D pass through unchanged. Handy right before feeding a plain 2D point list (turtle2d()/cr2dt()'s output, etc) to a function that expects real 3D points:

```
c23([[0, 0], [10, 0]]) # -> [[0, 0, 0], [10, 0, 0]]
```

c32()

```
c32(pts)
```

Exposed to scripts | backend/brep.py:2854 | Used in: —

Converts a 3D point list to 2D by dropping z -- ocad.py's own c32()/c3t2(), ported (the inverse of c23()). Points that are already 2D pass through unchanged.

```
c32([[1, 2, 3], [4, 5, 6]]) # -> [[1, 2], [4, 5]]
```

closest_points_between_two_lines()

```
closest_points_between_two_lines(l1, l2)
```

Exposed to scripts | backend/brep.py:968 | Used in: —

The pair of closest points [p_on_l1, p_on_l2] between two (possibly non-intersecting, non-parallel, finite) 3D line SEGMENTS l1=[a,b] and l2=[c,d]. ocad.py's own closest_points_between_two_lines(), ported onto plain numpy -- pure vector-geometry math, no build123d needed, so this works the same whether or not the rest of this session's environment has build123d installed.

Handles the general skew-lines case algebraically (solving where the shared perpendicular between the two INFINITE lines meets each of them), then falls back to checking segment-endpoint and clamped- projection candidates whenever that unconstrained closest approach falls outside one or both segments' own extents, or the segments are (near-)parallel (the algebraic solve is degenerate there -- a zero denominator).

Note the name: unlike "line" might suggest, l1/l2 are treated as finite 2-point segments, not infinite lines, so the closest pair can legitimately land on a segment's own endpoint rather than a true perpendicular foot.

Known inherited limitation (present in ocad.py's own version too, not introduced by this port): for TRULY parallel segments the algebraic solve is fully degenerate (a zero denominator, not just a small one), so the parallel-case fallback only compares the 4 endpoint-to-endpoint pairings -- it does NOT compute a true perpendicular foot. If one segment's own extent sits entirely "inside" the other's when projected onto the shared direction (e.g. l1 spans z=0..10 and l2 spans z=3..7, both along z), the real closest points are a perpendicular pair, but this returns an endpoint pairing instead -- correct distance-order-of-magnitude, but not the true minimum. Near-parallel (not exactly parallel) segments are unaffected -- the algebraic solve handles those fine.

distanceOfPointFromLine()

```
distanceOfPointFromLine(p0, l1)
```

Exposed to scripts | backend/brep.py:2726 | Used in: —

Perpendicular distance of `p0` from polyline `l1` -- [] if perpendicularProjectionOfPointOnLine() finds no valid foot.

fit_pline2line()

```
fit_pline2line(polyline, line)
```

Exposed to scripts | backend/brep.py:1212 | Used in: —

Fits `polyline` (an open 2D or 3D point list, e.g. sinewave()/ bezier()/turtle3d() output) onto `line` (a straight 2-point line/vector) -- ocad.py's own fit_pline2line(), ported as plain point-list math (no build123d needed, like turtle3d()/ sinewave()/l_len() above): scales every segment of `polyline` by the same factor so its start-to-end chord length matches `line`'s length, slides it so its start point lands on `line`'s start point, then rotates it about that point so its own chord points the same direction as `line` -- the polyline's own shape (bends, wiggles, etc) is preserved throughout; only its overall position, length, and orientation change. Handy for laying a wavy/curved profile down along an arbitrary 3D line before sweeping/lofting it (see surface_from_4_lines-style workflows: fit each edge curve between the right pair of corner points first).

Always returns 3D points ([x,y,z]), even if `polyline` was given as 2D -- the whole point is usually to lay a flat curve down along an arbitrary 3D `line`, which generally means it has to leave the XY plane to line up, so flattening the result back to 2D would just throw away the fit. `line` can be given as just its 2 endpoints, or a longer point list (e.g. turtle3d() output) -- only its first and last points are used.

```
l1 = turtle3d([[0, 10, 0], [0, 0, 20]])
l2 = sinewave(50, 3, 2, 50)
l3 = fit_pline2line(l2, l1)
show(color(p_line3d(l1, 0.1), "blue"))
show(color(p_line3d(l2, 0.3), "magenta"))
show(color(p_line3d(l3, 0.3), "cyan"))
```

l_len()

```
l_len(line)
```

Exposed to scripts | backend/brep.py:1097 | Used in: —

The length of a 2-point line [p0,p1] -- 2D or 3D. l_len([[0,0], [10,0]]) -> 10.0

l_lenv()

```
l_lenv(sec)
```

Exposed to scripts | backend/brep.py:1104 | Used in: —

Perimeter of the closed point list `sec` -- the sum of every segment length from seg(sec).

line2length()

```
line2length(l1, length=1)
```

Not yet exposed to scripts | backend/brep.py:3145 | Used in: —

Changes the length of the 2-point line `l1`=[p0,p1] to `length`, keeping p0 fixed -- ocad.py's own line2length(), ported.

line3d()

```
line3d(points, d=0.5, closed=False)
```

Exposed to scripts | backend/brep.py:7981 | Used in: —

A polyline made of small real cylinder segments (diameter `d`) connecting each consecutive point in `points` -- a lightweight, genuinely different construction from p_line3d()/sweep_sec2path() (no sweep() call at all, and no rounded end caps): one plain bd.Cylinder primitive per segment, oriented with a real build123d Plane (the same origin/z_dir trick sweep_sec2path()/ surface_line_vector() use internally to orient a cross-section along a direction), grouped with

bd.Compound() -- the same "don't union disjoint debug geometry" trick points()/show_edges() already use (a real boolean fuse between segments would be pure wasted CSG work for something that's just meant to be looked at).

The point-list sibling of show_edges() (which does the same job for a real Shape's edge selection, not a raw point list) -- handy for a quick, cheap look at ANY point list (sinewave()/cosinewave()/ rot_p()'s output, cr3dt()'s output, a bezier() control path, l1/l2 before combining them with surface_from_2_waves(), etc) without the tessellate()-only-works-on-Faces limitation a bare Edge/Wire (e.g. bd.Polyline(*points) on its own) would hit trying to show() it directly:

```
l1 = rot_p('x90', sinewave(60, 3, 8, 60))
l2 = rot_p('x90z90', cosinewave(60, 3, 8, 60))
show(line3d(l1, 0.3), 'blue')
show(line3d(l2, 0.3), 'magenta')
```

`closed=True` also connects the last point back to the first (same closed_loop convention as turtle3d()/p_line3d()/sweep_sec2path()). 2D points are placed at z=0.

line_as_unit_vector()

```
line_as_unit_vector(line)
```

Exposed to scripts | backend/brep.py:1177 | Used in: —

Unit-length version of line_as_vector().

line_as_vector()

```
line_as_vector(line)
```

Exposed to scripts | backend/brep.py:1170 | Used in: —

The vector from a 2-point line's start to its end -- e.g. line_as_vector([[0,0],[10,0]]) -> [10.0, 0.0].

mid_line()

```
mid_line(l1, l2)
```

Not yet exposed to scripts | backend/brep.py:3159 | Used in: —

A line drawn through the midpoints of each corresponding pair of points in `l1` and `l2` (same length, point-for-point) -- ocad.py's own mid_line(), ported.

mid_point()

```
mid_point(line)
```

Exposed to scripts | backend/brep.py:1110 | Used in: —

The point exactly halfway along `line` by arc length -- for a plain 2-point line that's just the average of the endpoints; for a longer polyline, the point half its total length along the path (not the average of all the points).

min_d_points()

```
min_d_points(sec, min_d=0.1)
```

Exposed to scripts | backend/brep.py:3545 | Used in: —

Thins `sec` so consecutive KEPT points are always at least `min\_d` apart (measured from the last point actually kept, not just each point's immediate predecessor in `sec`).

perpendicularProjectionOfPointOnLine()

```
perpendicularProjectionOfPointOnLine(p0, l1)
```

Exposed to scripts | backend/brep.py:2708 | Used in: —

The perpendicular projection of ``p0`` onto polyline ``l1`` -- the closest point that actually lies ON one of ``l1``'s segments (foot strictly between that segment's endpoints); [] if no segment has a valid perpendicular foot.

point_in_polygon()

```
point_in_polygon(point, poly)
```

Exposed to scripts | backend/brep.py:8279 | Used in: —

True if ``point`` (`[[x,y]]`) lies inside the closed 2D polygon ``poly`` (`[[x,y], ...]`) -- standard ray-casting test. Handy for filtering a point set (or, per-vertex, a shape like `honeycomb()`'s hexagons) down to whatever's actually inside an arbitrary boundary:

```
cells = honeycomb(3, 8, 8)
boundary = cr2dt([[3,2,1],[8,3,3],[5,7,1],[-8,0,2],[-5,20,1]])
inside = [h for h in cells if all(point_in_polygon(v, boundary) for v in h)]
```

points()

```
points(pts, d=0.5, shape='cube')
```

Exposed to scripts | backend/brep.py:7938 | Used in: —

Marker geometry at each point in ``pts`` (a flat list of 2D or 3D points) -- `ocad.py`'s own `points()` module, ported: a small cube (or sphere) of size ``d`` at every point, unioned into one shape, so you can `show()` a raw point list (`turtle2d()/turtle3d()`'s output, a `bezier()/bspline_closed()` control polygon, anything) to see exactly where its points land -- the same debugging role `show_edges()` plays for a real B-rep edge selection, but for plain point lists that never became a shape at all.

```
ctrl = [[0, 0], [5, 15], [15, 15], [20, 0]]
show(points(ctrl, 1), c="red") # where are the control points?
show(linear_extrude(polygon(bezier(ctrl, 30)), h=2))
```

``shape`` is `"cube"` (default, matches `ocad.py`'s own `points()` exactly) or `"sphere"`. 2D points are placed at `z=0`.

REAL PERFORMANCE NOTE: markers are grouped with a lightweight `Compound()` rather than `union()`'d together -- for a big point list (a densely-sampled `cr2dt()/bezier()/etc` curve, say), `union()`ing every marker one at a time is the same real `O(N)` chained-boolean cost documented in `p_line3d()`'s own docstring, and completely unnecessary here: these markers are disjoint, disposable debug dots that never need to be a single watertight solid, so a real boolean fuse between them is pure wasted CSG work. `Compound()` just groups them for display/export with no solving at all.

remove_extra_points()

```
remove_extra_points(points_list)
```

Exposed to scripts | backend/brep.py:3531 | Used in: —

Removes duplicate entries from ``points_list``, keeping each one's FIRST occurrence and original order -- works on plain values or point lists alike.

solid_line_vector()

```
solid_line_vector(sol, vec, both=0)
```

Exposed to scripts | backend/brep.py:9377 | Used in: `hollow_cylinder_with_star.py`, `hook.py`

A straight prism/beam solid along ``vec``, built by lofting between two translated copies of the 2D cross-section ``sol`` -- native `build123d translate()/loft()`, no point-list conversion or `c_polygon()` smoothing involved.

``sol`` can be an already-built Shape (`Sketch/Wire/Face`, e.g. `circle()/polygon()`'s output), OR a plain closed point list -- e.g. `Sketch on Face`'s own `"<name>_3d"` companion variable (built by `to_plane()`), a profile already placed on an arbitrary plane in 3D but not yet a real `build123d` object. Same "either a Shape or a raw point list" support `surface_line_vector()` already offers.

For a raw point list, the two translated copies are first run through `step_export.py`'s own exact B-rep reconstruction (the same bicubic-B-spline fit `swp()/swp_c()/swp_surf()` use in `python_eval.py`) -- this is what actually SMOOTHS a faceted

profile (e.g. a circle sketched in the 2D Sketch panel, which is really an N-sided polygon) back into round faces instead of literal flat facets: `step_export.py`'s corner detection (default `corner_deg=40`) treats a run of near-collinear points -- like consecutive vertices of a fine circle approximation, each only a few degrees apart -- as one smooth stretch and fits a single curved B-spline through it, only breaking at points that are ACTUAL sharp corners. Only if that exact fit isn't possible for this profile (`step_export.py` unavailable, or its own section-spread/cross-section checks reject it) does this fall back to `mixed_polygon()` (which, unlike `polygon()`, accepts a profile on any plane, not just flat XY) lofted literally facet-for-facet -- the always-correct but always-faceted path `translate()` alone (which only operates on already-built Shapes, so it can't be used on a raw point list directly) would have produced.

``both=0`` (the default): the solid runs from ``sol`` itself (at the origin end, i.e. ``vec * 0``) to ``sol`` translated by the FULL ``vec`` -- a one-directional prism of length `|vec|`.

``both=1``: the solid runs from ``sol`` translated by ``+vec`` to ``sol`` translated by ``-vec`` instead -- centered ON ``sol``'s own position rather than starting there, twice as long (`2*|vec|`) as the ``both=0`` case for the same ``vec``. Handy for cutting a groove/slot THROUGH a shape along ``vec`` and out both sides, without having to work out the two translated copies by hand -- see `14_wall_hook_dome_groove.py`'s own groove cut for the hand-rolled version of exactly this pattern (``loft([translate([0, i, 0], l2) for i in [-30, 30]])``), which this function generalizes.

```
sol = circle(5)
beam = solid_line_vector(sol, [0, 0, 40], both=1)
show(beam) # a prism running 40 up AND 40 down from sol
```

sort_points()

```
sort_points(sec, list1)
```

Exposed to scripts | backend/brep.py:3560 | Used in: —

For every point in ``sec``, finds its nearest point in ``list1`` -- returns a list the same length as ``sec``. Brute-force nearest-neighbor search (fine for hand-built profile/path point counts).

surface_line_vector()

```
surface_line_vector(line, vector, both_sides=False)
```

Exposed to scripts | backend/brep.py:4307 | Used in: —

Builds a real ruled Face by sweeping ``line`` (a point list, e.g. `sinewave()/turtle3d()/fit_pline2line()` output) straight along ``vector`` -- `ocad.py`'s own `surface_line_vector()`, ported as a real B-rep ruled surface (build123d's own native `Face.make_surface_from_curves()`, an exact ruled surface between two curves computed by OCCT) instead of a 2-row point-list "surface" meant for `swp_surf()`'s later mesh triangulation -- like the rest of the mesh-native surface family, that intermediate representation is dropped here in favor of returning a real, already-usable Face directly (union/loft/extrude/export-ready, no separate meshing step needed).

``both_sides=False`` (default) sweeps from ``line`` itself out to ``line`` translated by ``vector`` (a one-sided sweep -- ``line`` runs along one edge of the resulting Face). ``both_sides=True`` sweeps from ``line`` shifted by ``-vector`` to ``line`` shifted by ``+vector`` instead (a symmetric sweep -- ``line`` runs through the middle of the resulting Face, not along either edge).

``line`` can also be a real Shape instead of a raw point list -- a Wire/Edge (e.g. `arc_edge()`, `interp_spline()`'s output), or a filled 2D shape like `polygon()/circle()`'s output (its boundary is used automatically, same `_as_path_wire()` step `p_line3d()/sweep_sec2path()` use for a Shape argument). Translating a real Shape by ``vector`` uses `translate()`'s own real B-rep move (exact, no faceting), rather than shifting individual points.

```
# rotate()/translate() only work on real Shapes, not raw point
# lists (see fit_pline2line()'s own note) -- for a point list
# like sinewave()'s output, build it straight into the plane
# you want instead (here XZ: local (x,y) -> world (x,0,y)).
l1 = [[x, 0, y] for x, y in sinewave(50, 3, 2, 50)]
s1 = surface_line_vector(l1, [0, 20, 0], both_sides=True)
show(color(p_line3d(l1, 0.3), "blue"))
show(color(s1, "cyan", alpha=0.5))
```

to_points()

```
to_points(shape)
```

Exposed to scripts | backend/brep.py:12707 | Used in: —

Extracts the ordered boundary point list `[[x,y,z], ...]` from a real build123d Shape (e.g. `circle()/polygon()/square()/mirror()/rotate()`'s own output) -- the reverse direction of `polygon()`: walks the outer wire's edges in winding order and collects each edge's own start point. For a `polygon()`-built Face with only straight edges (e.g. `circle(r, s=n)`'s faceted n-gon form), this reconstructs exactly the original point list that built it, same count and order, even after real Shape-level operations (`rotate()/translate()/mirror()/etc`) have moved it around.

Handy for round-tripping a real Shape back into plain point-list territory to use numpy directly (array stacking/pairing/transpose tricks, same style as `ocad.py` scripts), or any of `brep.py`'s own point-list functions (`fit_pline2line()`, `corner_n_radius_list()`, etc) that don't accept a Shape:

```
a = circle(15, s=5)
b = rot('z36', circle(7, s=5))
pa, pb = to_points(a), to_points(b)
pairs = array([pa, pb]).transpose(1, 0, 2) # (5, 2, 3) -- pa[i]/pb[i] paired
```

For a Face/Wire with any CURVED edges (a `s=None` exact `circle()`, or a `mixed_polygon()` with a spline segment), each curved edge still only contributes its own start point here -- the curve itself isn't resampled into more points along the way. Use `to_mesh()` instead if you need a faceted approximation of a genuinely curved boundary.

Arcs & Circles (17)

arc()

```
arc(radius=0, start_angle=0, end_angle=0, cp=(0, 0), s=20)
```

Exposed to scripts | backend/brep.py:2953 | Used in: —

The most basic arc primitive: a point list tracing a circle of `radius` around center `cp`, sweeping from `start_angle` to `end_angle` degrees (0 = +X axis, counter-clockwise-positive), as `s` segments (s+1 points) -- `ocad.py`'s own `arc()`, ported directly (angles/center/radius given explicitly, unlike `arc_2p()/arc_3p()` below, which instead *fit* an arc to given points). Handy for a simple pie-slice/fillet-by-hand/dial-gauge curve where you already know the center and sweep you want:

```
a1 = arc(radius=10, start_angle=180, end_angle=270, cp=[10, 10], s=20)
show(p_line3d(a1, .3))
```

arc_2p()

```
arc_2p(p1, p2, r, cw=1, s=20)
```

Exposed to scripts | backend/brep.py:3000 | Used in: —

An arc between 2 points `p1`, `p2` with radius `r`, as an `s`-segment point list. There are always two circles of radius `r` through 2 points (centers on either side of the chord); `cw=1` (default) traces the clockwise arc between them (the one on that circle), `cw=-1` the counter-clockwise one (the other circle).

arc_2p_3d()

```
arc_2p_3d(n1, p0, p1, r, cw=1, s=20)
```

Exposed to scripts | backend/brep.py:3065 | Used in: —

3D counterpart of `arc_2p()` -- an arc between two 3D points `p0`, `p1`, lying in the plane with normal `n1` through `p0`.

arc_3p()

```
arc_3p(p1, p2, p3, s=30)
```

Exposed to scripts | backend/brep.py:3045 | Used in: —

An arc through 3 2D points `p1`, `p2`, `p3` (their circumcircle's arc from p1 through p2 to p3), as an `s`-segment point list.

arc_3p_3d()

```
arc_3p_3d(points, s=20)
```

Not yet exposed to scripts | backend/brep.py:3211 | Used in: —

An arc through 3 non-collinear 3D points, as an `s`-segment point list. ocad.py's own arc_3p_3d(), ported: finds the points' plane normal via normal_vector(), rotates them flat into that plane's own local 2D frame (about the axis perpendicular to both the normal and world -Z), runs the existing 2D arc_3p() there, then rotates the result back out to the original 3D orientation.

arc_edge()

```
arc_edge(radius, start_angle=0, end_angle=360, cp=(0, 0))
```

Exposed to scripts | backend/brep.py:2970 | Used in: —

The exact, real-B-rep counterpart of arc(): a genuine analytic circular-arc Edge (build123d's own Edge.make_circle(), a true curved curve, not a faceted point list), same radius/start_angle/end_angle/cp meaning as arc(). No `s` -- there's no facet count to choose, same reasoning as circle(s=None).

Use this (not arc()) anywhere a *real curve* is wanted, especially as sweep_sec2path()'s `path3d`: feeding arc()'s point list straight in (even via c23()) turns it into a 20-straight-segment Polyline, and with the default transition="round" that means OCCT has to build a rounded joint at every one of those ~19 corners -- slow, and pointless when the path is really just a circular arc. arc_edge() sweeps as one smooth analytic curve instead, same as circle() already does for a *full* circle:

```
path = arc_edge(20, -90, 90)
a = sweep_sec2path(circle(5), path) # fast, exact
show(a)
```

arc() itself is still the right choice for turtle-path point-list math (feeding into p_line3d(), s_int1(), offset() trimming, etc) -- this is only for when you specifically want a real curved Edge.

cir_2p()

```
cir_2p(p1, p2, r, cw=1, s=20)
```

Exposed to scripts | backend/brep.py:3017 | Used in: —

A full circle of radius `r` through both `p1` and `p2`, as an `s`-segment point list -- `cw` picks which of the two possible circles (see arc_2p()'s docstring).

cir_3p()

```
cir_3p(p1, p2, p3, s=50)
```

Exposed to scripts | backend/brep.py:3057 | Used in: —

A full circle through 3 2D points, as an `s`-segment point list -- same circumcenter/radius as arc_3p()/cp_3p().

circle()

```
circle(r=1, cp=[0, 0], s=None)
```

Exposed to scripts | backend/brep.py:218 | Used in: —

A B-rep 2D disc, centered at the origin by default -- same as real OpenSCAD's circle(r) (which is likewise always centered; real OpenSCAD's circle() has no `center` parameter -- use `cp` to shift it instead, or translate() the result).

`s` (segment count -- same name/meaning as ocad.py's own circle(r, cp, s)) controls whether this is exact or faceted:

s=None (the default) -- an exact round B-rep circle. No facet

count needed, since the underlying curve is exact -- this is different from `ocad.py`'s `circle()` (which always faceted, defaulting to `s=50`) and from real OpenSCAD's `circle()` (which always faceted too, at whatever `$fn/$fa/$fs` resolve to); deviates from both on purpose, since "exact" is strictly better than "a 50-sided polygon" when nothing forces a facet count in the first place.

`s=<some integer>` -- instead builds an `s`-sided regular polygon

approximating a circle (via `polygon()`), the same way `ocad.py`'s `circle()` does (this is also how real OpenSCAD's own `circle($fn=...)` behaves at a low `$fn`/segment count). Useful for genuinely wanting flat sides, not just an approximation:

`circle(10, s=6)` is a hexagon, `circle(10, s=3)` a triangle, etc.

`cp_3p()`

```
cp_3p(p1, p2, p3)
```

Exposed to scripts | backend/brep.py:3028 | Used in: —

The center (circumcenter) of the circle through 3 points ``p1``, ``p2``, ``p3`` -- 2D or 3D (3 points always share a plane).

`cp_arc()`

```
cp_arc(arc1)
```

Not yet exposed to scripts | backend/brep.py:3190 | Used in: —

The center point of a circle/arc ``arc1`` (a point list) -- `ocad.py`'s own `cp_arc()`, but sampling 3 points evenly spaced by INDEX (first, one-third, two-thirds along the list) rather than `ocad`'s own degree-count heuristic, so it works regardless of ``arc1``'s own point-per-degree density.

`cp_cir_3d()`

```
cp_cir_3d(cir)
```

Not yet exposed to scripts | backend/brep.py:3239 | Used in: —

The center point of a circle/arc ``cir`` given as 3D points (at least 3, not necessarily coplanar-with-Z) -- `ocad.py`'s own `cp_cir_3d()`, ported, using the same plane-flattening trick as `arc_3p_3d()` above (see its own docstring), finishing with `cp_3p()` in the flattened 2D frame instead of `arc_3p()`.

`extend_arc2d()`

```
extend_arc2d(a, theta=0, s=20, both=0)
```

Not yet exposed to scripts | backend/brep.py:3265 | Used in: —

Extends the 2D arc ``a`` (a point list, e.g. from `arc()`/`arc_2p()`/`arc_3p()`) by ``theta`` degrees at its end (`both=0`, default) or at both ends equally (`both=1`) -- `ocad.py`'s own `extend_arc2d()`, ported.

`extend_arc3d()`

```
extend_arc3d(a, theta=0, s=20, both=0)
```

Not yet exposed to scripts | backend/brep.py:3287 | Used in: —

3D counterpart of `extend_arc2d()` -- extends the 3D arc ``a`` by ``theta`` degrees. `ocad.py`'s own `extend_arc3d()`, ported: flattens ``a`` into its own plane the same way `arc_3p_3d()`/`cp_cir_3d()` do, calls `extend_arc2d()` there, then rotates the result back out to 3D.

`tangent_arc()`

```
tangent_arc(a, b, radius, side=0)
```

Exposed to scripts | backend/brep.py:9922 | Used in: —

A circular arc of `radius`, tangent to both `a` and `b`, where each of `a`/`b` is either a real build123d Edge (e.g. `circle(5).edges()[0]` for a circle, or a straight Edge for a line) or a raw 2-point line `[x1,y1],[x2,y2]`/`[[x1,y1,z1],[x2,y2,z2]]` (auto-built into a line Edge for you). Covers ocad.py's whole tangent-construction family in one call -- `two_cir_tarc()` (circle-circle), `two_cir_tarc_internal()`, `fillet_line_circle()/fillet_line_circle_internal()` (line-circle) -- since build123d's own native ConstrainedArcs solves all of these the same way regardless of what `a`/`b` are, there's no need for ocad.py's four separate hand-rolled trig functions.

```
c1 = circle(10).edges()[0]
c2 = translate([20, 0, 0], circle(6)).edges()[0]
show(show_edges([tangent_arc(c1, c2, radius=4)]))
```

There are usually 2 valid tangent arcs for a given radius/side pair -- `side` (0 or 1) picks between them; if only one exists, `side` is ignored. `radius` is a real geometric constraint, not just a style knob: too small to bridge the gap between `a` and `b` and no tangent arc exists at all -- for two circles of radius r_1/r_2 whose centers are `d` apart, a fillet nestled between them needs roughly $\text{radius} \geq (d - r_1 - r_2) / 2$ at minimum. Raises a clear error (not a cryptic one) if that happens.

tcct()

```
tcct(r1, r2, cp1, cp2, cw=1)
```

Exposed to scripts | backend/brep.py:3098 | Used in: —

The internal (crossing) tangent line between two circles -- radius `r1` centered at `cp1`, radius `r2` centered at `cp2` -- as [touch_point_on_circle1, touch_point_on_circle2]. Only exists when the circles don't overlap (center distance $\geq r_1 + r_2$); raises otherwise. There are always two internal tangent lines (mirrored across the line joining the centers); `cw`=1 (default) picks one, `cw`=-1 the other.

two_cir_tarc()

```
two_cir_tarc(c1, c2, r, side=0, s=50)
```

Not yet exposed to scripts | backend/brep.py:3404 | Used in: —

An arc of radius `r` tangent to both circles/arcs `c1` and `c2` (point lists) -- ocad.py's own `two_cir_tarc()`, ported. There are 2 possible tangent arcs on either side of the circles' shared centerline; `side` (0 or 1) picks which one.

2D Profiles & Curve Generators (18)

bezier()

```
bezier(pl, s=20)
```

Exposed to scripts | backend/brep.py:3695 | Used in: —

A Bezier curve through control points `pl` (passes through the first/last point only, pulled toward the others), as an `s`-point list -- standard Bernstein-polynomial construction. 2D or 3D.

bspline_closed()

```
bspline_closed(pl, deg=3, s=100)
```

Exposed to scripts | backend/brep.py:3755 | Used in: —

Closed-loop counterpart of `bspline_open()` -- a periodic B-spline curve through control points `pl` (wraps back to its own start), degree `deg`, as an `s`-point list. 2D or 3D.

bspline_open()

```
bspline_open(pl, deg=3, s=100)
```

Exposed to scripts | backend/brep.py:3713 | Used in: —

An open (clamped) B-spline curve through control points `pl``, degree `deg`` ($0 \leq \text{deg} < \text{len}(\text{pl})$), as an `s``-point list -- passes through the first and last control points; `deg`=1` gives a plain polyline through every point, higher degrees smooth it out more. Standard Cox-de Boor recursive basis functions. 2D or 3D.

cosinewave()

```
cosinewave(length, cycles, amplitude, n)
```

Exposed to scripts | backend/brep.py:8245 | Used in: —

Same as `sinewave()` but starting at the wave's peak (cosine phase) instead of zero -- `ocad.py`'s own `cosinewave()`, ported.

cr2dt()

```
cr2dt(sec, s=20)
```

Exposed to scripts | backend/brep.py:442 | Used in: —

Builds a closed 2D outline from turtle-style relative `[dx,dy]` (or `[dx,dy,r]`) offsets, rounding each vertex with a fillet arc of radius `r` (0 -- the default when a point omits it -- means "leave this vertex sharp").

`s` (segment count -- same `s=None/s=<int>` meaning as `circle()`'s own `s``) controls whether the rounded corners are exact or faceted:

`s=<some integer>` (the default, 20) -- a plain 2D point list, each

rounded corner approximated by `s`` straight segments, same as before this parameter grew the `s=None` option below. Feed it into

```
polygon() for a filled profile:  
sec = cr2dt([[0,0,2],[10,0,2],[0,10,2],[-10,0,2]], 10)  
show(linear_extrude(polygon(sec), h=3))
```

`s=None` -- instead returns a real B-rep Wire, its straight

stretches exact Line edges and its rounded corners exact circular Arc edges (built from the exact same center/radius/tangent-point math this function always computes -- `s=None` just stops discretizing that math into sample points and builds the real curve from it instead). No facet count needed, same reasoning as

`circle()`'s own `s=None`. Feed the result into `face_from_wire()` for

a filled profile instead of `polygon()` (which takes a point list, not a Wire -- see `polygon()`'s own docstring):

```
sec = cr2dt([[0,0,2],[10,0,2],[0,10,2],[-10,0,2]]) # s=None  
show(linear_extrude(face_from_wire(sec), h=3))
```

The short name of `ocad.py`'s `corner_radius_with_turtle()` (`cr2dt` is that function's own alias there too), reimplemented from scratch here -- plain trig (an unsigned "shortest way around" arc between the two tangent points at each corner), not a port of the original's own ~15-branch case analysis by incoming/outgoing corner orientation, since the shortest-way-around trick handles either polygon winding direction without needing to tell them apart.

`sec: [[dx,dy], ...]` or `[[dx,dy,r], ...]` -- same turtle convention as

```
turtle2d() (each entry moves relative to wherever you just  
were; the first entry is effectively the absolute starting  
point, since the running sum starts at [0,0]), with an optional  
trailing fillet radius per point.
```

`s`: segments per rounded corner when an int (a sharp vertex, `r=0`, is

```
always just 1 point either way); None for an exact Wire instead  
(see above).
```

Like `corner_radius_with_turtle()` itself, this ALWAYS treats `sec`` as a closed loop -- the corner-rounding at point 0 uses the LAST point as its "previous" neighbor, and vice versa -- so if you want the first/last points left exactly as given (e.g. building one edge of an open profile, not a closed polygon), leave their radius at 0 (the default when omitted).

Raises if a requested radius doesn't fit between a vertex and either neighboring point (same "radius too large" check the original makes, just against each corner's own two edges directly here, rather than needing the original's incoming/outgoing-radius bookkeeping across the whole loop) -- true whether s is an int or None, since both share the same tangent-point math.

cr3dt()

```
cr3dt(pnts, s=5)
```

Exposed to scripts | backend/brep.py:619 | Used in: —

3D counterpart of cr2dt() -- builds a closed 3D outline from turtle-style relative [dx,dy,dz] (or [dx,dy,dz,r]) offsets, rounding each vertex with a fillet arc of radius r. Any two edges meeting at a vertex share a plane (two lines through a common point always do), so this is the exact same fillet-arc construction cr2dt() uses, just built inside that per-corner plane instead of always the flat XY one. The short name of ocad.py's corner_radius3d_with_turtle() -- reimplemented from scratch, not a port (see cr2dt()'s docstring for why).

s (same s=None/s=<int> meaning as cr2dt()'s own `s` -- see its docstring for the full explanation): an int (default 5) returns a plain 3D point list, each rounded corner approximated by `s` straight segments -- unchanged from before this parameter grew the s=None option. s=None instead returns a real B-rep Wire, exact Line edges for the straight stretches and exact circular Arc edges (each built in its own corner's actual 3D plane, via Edge.make_three_point_arc() through the corner's already-computed tangent points and arc midpoint) for the rounded corners -- feed THAT straight into sweep_sec2path()'s path3d exactly like the point- list form (sweep_sec2path() accepts an already-built Wire directly, see its own docstring), just with a genuinely round path instead of one approximated by `s` short straight segments.

pnts: [[dx,dy,dz], ...] or [[dx,dy,dz,r], ...] -- same turtle

convention as turtle3d(), with an optional trailing fillet radius per point.

s: segments per rounded corner when an int; None for an exact Wire

instead (see above).

Same closed-loop wraparound behavior as cr2dt() -- leave the radius at 0 (the default) on any point you don't want rounded using a wraparound neighbor (e.g. the first/last point of what's conceptually an open path -- see cr2dt()'s docstring). Feed the result straight into sweep_sec2path()'s path3d for a pre-rounded path (an alternative to that function's own transition="round").

honeycomb()

```
honeycomb(r, n1, n2)
```

Exposed to scripts | backend/brep.py:8255 | Used in: —

A hex grid of `n1` x `n2` regular hexagons of "radius" `r` (center to vertex) -- ocad.py's own honeycomb(), reimplemented as plain hex-grid math: returns a list of hexagons, each a closed 6-point 2D loop, ready for `[polygon(h) for h in honeycomb(...)]` or filtering with point_in_polygon().

interp_spline()

```
interp_spline(points, closed=False)
```

Exposed to scripts | backend/brep.py:8854 | Used in: —

A smooth curve that passes EXACTLY through every point in `points` (2D or 3D) -- unlike bezier()/bspline_open()/bspline_closed() above (which only approximate through their control points), this is a true interpolating spline: ocad.py's own interpolation_bspline_open()/interpolation_bspline_closed(), ported, as a thin wrapper around build123d's native Edge.make_spline() (GeomAPI_Interpolate under the hood -- exact, not fitted). `closed=True` makes a periodic (closed-loop) spline through the points instead of an open curve -- same idea as bspline_closed()'s closed_loop convention. Returns a real B-rep Edge, feed it to polygon()-style usage yourself if you need a filled face (Face(Wire([...]))-equivalent isn't wrapped here since the point-list functions above already cover the common polygon()

case for approximating curves).

pies1()

```
pies1(section, pts)
```

Exposed to scripts | backend/brep.py:8303 | Used in: —

Filters a list of 2D points down to just the ones that lie inside the closed polygon `section` -- ocad.py's own `pies1()`, ported as a thin wrapper around `point_in_polygon()` rather than its original numpy line-intersection formulation (same result, much simpler): returns the subset of `pts` for which `point_in_polygon(p, section)` is True.

```
pts = [[random.uniform(-5,20), random.uniform(-2,25)] for _ in range(10000)]
sec = cr2dt([[2,1,.1],[7,5,2],[5,7,3],[-5,7,2],[-7,5,3]])
inside = pies1(sec, pts)
show(p_line3d(sec, .05, 0, 1))
show(color(points(inside, .2), "magenta"))
```

sample_curve()

```
sample_curve(shape, n)
```

Exposed to scripts | backend/brep.py:6185 | Used in: hook.py

`n` evenly-parametrized points along a Wire/Edge (or a filled Face's own boundary wire, pulled out automatically -- same "grab the outline" step sweep_sec2path()/p_line3d() use internally) -- the general-purpose way to turn a real curve into a plain point list, for when `to_points()` isn't enough: `to_points()` only grabs each EDGE's own START point, so a Wire made of few but long/curved edges (an `intersection()` result, most commonly -- often just ONE single curved edge) comes back as far too few points from `to_points()` to be useful as a station list. `sample_curve()` instead walks the WHOLE wire's own combined parametrization evenly, regardless of how many edges it's actually built from.

Reach for this whenever you need a real curve (an `intersection()` result, an `arc_edge()`, any Wire/Edge) as a point list for one of `brep.py`'s point-list functions -- `convert_3lines2fillet()`'s own `spine/line_a/line_b`, most commonly:

```
curve = intersection(a, b)
pts = sample_curve(curve, 40)
```

On a CLOSED wire (a full loop, e.g. `mixed_wire()`'s own output, or one of a solid's own `wires()`), `n` distinct points are spread evenly around the WHOLE loop without ever repeating the start point -- unlike an open curve, where `t=0` and `t=1` are genuinely different endpoints worth keeping both of, `t=0` and `t=1` on a closed one are the exact same physical point, so including both would silently duplicate it. That matters downstream: feeding a duplicated closing point into `interp_spline(..., closed=True)` (`cap_wire()`'s own fallback for a raw point list) gives OCCT's periodic interpolator a zero-length final span, which fails with a low-level "BSplCLib::Interpolate" error rather than a clean Python one.

UNVERIFIED against a real `build123d` install (see module docstring) -- `Wire.position_at(t)` is expected to parametrize evenly by ARC LENGTH across the whole wire (`t=0.5` landing on the true geometric midpoint) rather than by the wire's raw underlying parameter space (which could bunch points unevenly if the wire's edges have very different natural parameter ranges); if station spacing looks uneven in practice, that assumption is what to revisit here.

sec2vector()

```
sec2vector(v1=(1, 0, 0), sec=())
```

Not yet exposed to scripts | backend/brep.py:3317 | Used in: —

Rotates the 2D section `sec` so that the direction its own "up" (+Y in the flat 2D sketch) used to point now points along the 3D vector `v1` instead -- ocad.py's own `sec2vector()`, ported onto this file's own `rot_p()`. Handy for orienting a flat profile (a `circle()/polygon()/etc`'s raw point list) to face an arbitrary direction before feeding it to `solid_from_closed_rings()/surface_from_points_grid()/etc`.

section_view()

```
section_view(shapes, axis, offset, keep='positive')
```

internal: cuts each shape in `shapes` with a plane perpendicular to `axis` ('x'/'y'/'z') at `offset` along that axis, keeping the 'positive' ($\geq$ offset) or 'negative' ($\leq$ offset) side -- the exact same plane()+split() idiom already verified working in examples/21_import_and_section_step_library.py's own _safe_quarter_cut(), just parametrized on a caller-chosen axis/offset/side (from the Drawing tab's own Section tool) instead of that example's fixed through-the-center quarter cut.

Returns (halves, cam_dir, up, errors): halves: the surviving half of each shape that could be cut (one

per input shape that didn't raise -- see `errors` below)

cam_dir: unit-ish direction (center-to-camera, same convention as

_drawing_view_directions()) that looks straight at the freshly exposed cut face -- the "positive" side keeps material whose cut face points -axis, so the camera needs to sit on the -axis side looking back along +axis to face it head-on (and vice versa for "negative")

up: this axis's viewport_up (see _AXIS_UP above) errors: one message per shape the cut plane missed entirely (a

real, expected outcome on a disconnected multi-body assembly -- see _safe_quarter_cut()'s own docstring -- not a bug), so the caller can report a partial result instead of silently losing shapes with no explanation.

sinewave()

```
sinewave(length, cycles, amplitude, n)
```

Exposed to scripts | backend/brep.py:8230 | Used in: —

A 2D point list tracing a sine wave: x runs 0 to `length`, y = `amplitude` * $\sin(2\pi \cdot \text{cycles} \cdot x / \text{length})$, sampled at `n` points. ocad.py's own sinewave(), ported. Feed it to polygon()/ linear_extrude(), or wrap it into 3D yourself (translate/rotate) for a rippled profile.

turtle2d()

```
turtle2d(offsets)
```

Exposed to scripts | backend/brep.py:367 | Used in: —

Converts a list of relative [dx, dy] offsets into absolute [x, y] points via a running cumulative sum -- "turtle graphics" style path building: each entry means "move this far from wherever you just were", not an absolute coordinate. Feed the result into polygon() to sketch a closed outline from turtle-style steps instead of typing out absolute coordinates by hand:

```
sec = turtle2d([[0,0],[10,0],[0,5],[-10,0]]) # a 10x5 rectangle
show(linear_extrude(polygon(sec), h=3))
```

Plain Python -- no build123d involved, just produces the point list polygon() then turns into a real B-rep face.

turtle3d()

```
turtle3d(offsets)
```

Exposed to scripts | backend/brep.py:390 | Used in: —

Converts a list of relative [dx, dy, dz] offsets into absolute [x, y, z] points via a running cumulative sum -- the 3D counterpart of turtle2d() (see its docstring). Each entry means "move this far from wherever you just were", not an absolute coordinate:

```
path = turtle3d([[0,0,0],[10,0,0],[0,0,10],[0,10,0]])
# -> [[0,0,0],[10,0,0],[10,0,10],[0,10,10]]
```

Plain Python -- no build123d involved. There's no 3D polyline/sweep primitive in brep.py yet to feed this straight into, but it's handy for building a 3D point path to pass to your own build123d curve construction in a script, e.g.:

```
from build123d import Polyline
wire = Polyline(*turtle3d(offsets))
```

or for sanity-checking a path's shape with `echo()/show_edges()`-style inspection before using it elsewhere.

xyc()

```
xyc(pnts)
```

Exposed to scripts | [backend/brep.py:2904](#) | Used in: —

Zeroes the z-coordinate of every point in a 3D point list -- `ocad.py`'s own `xyc()`, ported: projects onto the XY plane while keeping the points 3D (unlike `c32()`, which drops z entirely and returns 2D points).

```
xyc([[1, 2, 3], [4, 5, 6]]) # -> [[1, 2, 0], [4, 5, 0]]
```

xzc()

```
xzc(pnts)
```

Exposed to scripts | [backend/brep.py:2916](#) | Used in: —

Zeroes the y-coordinate of every point in a 3D point list -- `ocad.py`'s own `xzc()`, ported: projects onto the XZ plane.

```
xzc([[1, 2, 3], [4, 5, 6]]) # -> [[1, 0, 3], [4, 0, 6]]
```

yzc()

```
yzc(pnts)
```

Exposed to scripts | [backend/brep.py:2926](#) | Used in: —

Zeroes the x-coordinate of every point in a 3D point list -- `ocad.py`'s own `yzc()`, ported: projects onto the YZ plane.

```
yzc([[1, 2, 3], [4, 5, 6]]) # -> [[0, 2, 3], [0, 5, 6]]
```

Polygons, Wires & Faces (13)

c_polygon()

```
c_polygon(s1, pitch=0.1, interpolation=1)
```

Exposed to scripts | [backend/brep.py:9335](#) | Used in: —

A filled Face from a rough, unevenly-spaced closed point list -- smoothed and evened out first, unlike `polygon()/mixed_polygon()` (straight segments exactly through the given points, however unevenly they're spaced). Two steps under the hood:

1. `o.homogenise(s1, pitch, 1)` (the opt-in `ocad` bridge -- see the module docstring) re-samples `s1` to a fine, EVEN `pitch` (default 0.1) around the closed loop, so a boundary with long straight runs and tight clustered corners (e.g. raw output from a CAD import, or a coarse hand-picked outline) gets uniform point density before it's fit with a curve. Lower `pitch` = more points = a closer-fitting spline but slower -- raise it for a coarse `s1` where extra smoothness isn't needed. 2. `interp_spline(..., 1)` fits one true closed (periodic) spline exactly through those resampled points (see `interp_spline()` above), and `cap_wire()` fills that spline boundary as a real Face.

The result is a smooth, rounded-corner face that follows `s1`'s general shape without `polygon()`'s straight-segment faceting -- reach for this instead of `polygon()/mixed_polygon()` when `s1` is a rough/organic outline you want smoothed rather than traced exactly, e.g.:

```
s1 = [[0, 0], [40, 5], [45, 30], [10, 40], [-5, 15]]
show(linear_extrude(c_polygon(s1), 5))
```

Note the extra pass through `ocad` makes this noticeably slower than `polygon()` for large point counts -- prefer `polygon()/mixed_polygon()` when `s1` is already the exact boundary you want.

cap_wire()

```
cap_wire(boundary, surface_points=None)
```

Exposed to scripts | backend/brep.py:6127 | Used in: —

Caps a closed boundary with a real, potentially NON-planar Face -- build123d's own native Face.make_surface() (OCCT's BRepOffsetAPI_MakeFilling under the hood: a genuine curvature-aware surface fill through the given boundary, not a flat cap). The non-planar counterpart to face_from_wire() above -- reach for this one instead whenever the ring you need to cap does NOT lie in one flat plane (face_from_wire()/polygon()/mixed_polygon() all require planarity; this doesn't).

`boundary` is either a closed loop of raw [x,y,z] points (built into one periodic interpolating spline Edge via interp_spline() first) or an existing Wire/Edge/list of Edges to use directly -- e.g. one of a Shape's own wall.wires(), for the closest possible edge match when sewing with solid_from_faces() (an independently built spline through the same points is still a genuinely different curve, so it won't sew edge-exact the way face_from_wire() does for a planar cap).

`surface\_points`, if given, are extra raw [x,y,z] points the fill is pulled through in addition to the boundary -- use this to keep a bulging/non-planar cap closer to the true shape in the middle instead of just bridging flat-ish between the boundary points.

```
wall = surface_from_points_grid(grid)    # a tube, open at both ends
cap_a = cap_wire(grid[0])                # first ring, however curved
cap_b = cap_wire(grid[-1])              # last ring
solid = solid_from_faces(wall, cap_a, cap_b)
```

concave_hull()

```
concave_hull(points, n=3, engaging_angle=270)
```

Not yet exposed to scripts | backend/brep.py:3464 | Used in: —

The concave hull of a scattered list of 2D `points`, via a k-nearest-neighbours boundary-walking algorithm -- ocad.py's own concave_hull(), ported: `n` is the starting neighbourhood size (grown automatically if the walk gets stuck), `engaging\_angle` caps how sharp a turn is allowed when picking the next boundary point. See also convex_hull() for the convex-only case.

contiguous_chains()

```
contiguous_chains(seg_list)
```

Exposed to scripts | backend/brep.py:1316 | Used in: —

Stitches a flat, unordered list of 2-point line segments `seg\_list` into one or more separate continuous chains/loops of points -- ocad.py's own contiguous_chains(), ported as plain point-list math (no build123d needed). A real build123d boolean's own Wire(intersection())'s result, say) already comes back as a proper ordered point list, so you won't need this for that -- but two_solids_intersection() below (the mesh-based, more robust alternative to a real B-rep boolean for genuinely hard cases) DOES return raw, disconnected triangle-intersection segments that genuinely need stitching -- exactly the pairing this was ported for. Also handy for the rare case of feeding in your own disconnected segment list from an external data source. It is NOT a way to reduce the edge count of an existing B-rep Shape -- for that, see arc_3p()'s `s=` parameter or interp_spline().

Repeatedly pulls out one connected run of segments (matching endpoints to 4 decimal places) until every segment has been consumed, so the result may contain more than one chain if the input isn't all one connected network. A chain whose two ends meet back up comes back as a closed loop (first point == last point).

```
segs = [[[0,0],[10,0]], [[10,10],[10,0]], [[0,0],[0,10]]]
contiguous_chains(segs) => [[[10,10],[10,0],[0,0],[0,10]]]
```

convex_hull()

```
convex_hull(pnts)
```

Exposed to scripts | backend/brep.py:3573 | Used in: —

The convex hull of 2D point list `pnts`, as an ordered point list of just the hull's own vertices (the outline you'd get wrapping a rubber band around every point). Andrew's monotone chain algorithm.

face from wire()

```
face_from_wire(wire)
```

Exposed to scripts | backend/brep.py:6097 | Used in: —

Wraps an EXISTING closed Wire directly into a Face -- build123d's own native Face(wire) constructor. The point of this over mixed_polygon()/polygon() (which both build a brand new Wire from a plain point list) is EXACT edge matching: when you need a cap Face whose boundary is guaranteed to coincide exactly with another real shape's own boundary edges (e.g. one end of a surface_from_points_grid() surface), rebuilding that boundary independently from raw points (even the SAME points) traces a close-but-not-identical curve -- a fitted NURBS boundary and a fresh straight-line polygon through the same points are two genuinely different curves, and OCCT's exact-topology sewing (solid_from_faces()) needs edges to coincide, not just look close, or it fails to zip them into one watertight shell.

```
wall = surface_from_points_grid(surf)      # a curved wall Face
w = wall.wires()                          # its own boundary wire(s)
cap_a = face_from_wire(w[0])              # exact match to wall's own edge
cap_b = face_from_wire(w[1])
solid = solid_from_faces(wall, cap_a, cap_b)
```

Raises whatever build123d itself raises if `wire` isn't planar (a Face can only be built from a wire that lies in one flat plane -- same restriction polygon()/mixed_polygon() have).

hull()

```
hull(*shapes)
```

Exposed to scripts | backend/brep.py:6257 | Used in: —

The convex hull enclosing all the given shapes -- real B-rep, built from build123d's own ConvexPolyhedron (which computes the hull of a point cloud itself; there's no need to compute hull faces here).

Every shape is tessellated first (to_mesh(), default tolerance) and every triangle vertex from all of them is fed into the hull together. For a shape made only of flat faces (box(), a union() of boxes, etc), tessellating doesn't add or move any true corner -- triangulating a flat face just adds a diagonal across it -- so the result is exact. For a shape with curved faces (cylinder(), sphere(), a fillet(ed) edge), it's only an approximation, accurate to roughly that default tessellation tolerance -- tessellate the shape yourself first with a tighter to_mesh(shape, tolerance=...) and pass the resulting points through bd.ConvexPolyhedron directly if you need closer than that.

i_p2d()

```
i_p2d(l1, l2)
```

Exposed to scripts | backend/brep.py:2691 | Used in: —

The intersection point of two 2D lines l1=[p0,p1], l2=[p2,p3], extended infinitely (not clipped to the segments) -- e.g. i_p2d([(0,0],[1,4]], [[10,0],[7,2]]) -> [1.4286, 5.7143]. Raises if the lines are parallel.

mixed_polygon()

```
mixed_polygon(pieces)
```

Exposed to scripts | backend/brep.py:9301 | Used in: —

A B-rep 2D filled polygon like polygon(), but each item in `pieces` can be EITHER a plain 2D/3D point ([x,y] -- a straight-line vertex, same as polygon()) OR a real build123d Edge (interp_spline(), arc_edge(), tangent_arc() output, etc) used as-is for that stretch of the boundary instead of a straight line -- see mixed_wire() above for the exact chaining rules (this is just mixed_wire(pieces) capped with bd.Face(), which -- unlike cap_wire() -- REQUIRES the resulting wire to be exactly planar; see mixed_wire()'s own docstring for what goes wrong if it isn't and how to route around it with mixed_wire()+cap_wire() instead).

The main use: swapping out a heavily-faceted arc_3p(...)-built curve (many short straight edges -- see arc_3p()'s own `s` parameter) for ONE real curved Edge, when the rest of the profile should stay sharp-cornered -- e.g. instead of `arc_3p(p1,

p2, p3, s=30) + [c1, c2]` (a ~30-point faceted arc plus 2 straight corners, all straight edges once polygon()-ed), use `[interp_spline([p1, p2, p3]), c1, c2]` here: same overall profile shape, but the curved portion becomes a single smooth Edge -- and that's ~29 fewer edges once extruded/booleaned.

```
curve = interp_spline([[10, 0], [0, 50], [10, 100]])
sec = mixed_polygon([curve, [-10, 100], [-10, 0]])
show(linear_extrude(sec, 10))
```

Raises if `pieces` doesn't chain up into exactly one closed loop (e.g. a given Edge points the wrong way, or there's a real gap).

mixed_wire()

```
mixed_wire(pieces)
```

Exposed to scripts | backend/brep.py:8879 | Used in: —

Chains `pieces` into ONE closed Wire -- the exact-points half of mixed_polygon() below, pulled out on its own so you can cap the result with something OTHER than mixed_polygon()'s planar-only bd.Face(wire). Each item in `pieces` can be EITHER a plain 2D/3D point ([x,y] -- a straight-line vertex) OR a real build123d Edge (interp_spline(), arc_edge(), tangent_arc() output, etc) used as-is for that stretch of the boundary instead of a straight line. Walks `pieces` in order, connecting consecutive plain points with straight lines and dropping in each given Edge directly (adding a straight line from the previous point to the Edge's own start if there's a gap), then closes the loop from the last item back to the first the same way -- combines the whole chain into one Wire (bd.Wire.combine()) and returns it (NOT a Face -- see below).

WHY THIS EXISTS, separately from mixed_polygon(): mixed_polygon() caps its wire with bd.Face(wire), which -- like face_from_wire()/ polygon() -- requires the wire to be exactly PLANAR (flat, no matter how slightly it bows out of one plane). Feed it a boundary that isn't quite flat (an easy thing to end up with by the time points have been through several rot()/translate()/curve-fitting steps in a real pipeline) and the result is NOT a clean error -- it's a silently wrong/degenerate Face, which shows up downstream as loft()/linear_extrude() only picking up a small sliver of the boundary instead of the whole thing you fed in, with no obvious cause at the mixed_polygon() call site itself.

cap_wire() doesn't have that restriction -- it fills a boundary with a genuine curvature-aware surface (build123d's Face.make_surface(), the same one a non-planar boundary from raw points already goes through there) and, unlike mixed_polygon(), accepts an existing Wire directly. So for the exact-points-AND- real-Edges input mixed_polygon() takes, but with cap_wire()'s non-planar-safe result instead of mixed_polygon()'s planar-only one:

```
w = mixed_wire([curve, [-10, 100], [-10, 0]])
sec = cap_wire(w)          # instead of mixed_polygon(...)
show(linear_extrude(sec, 10))
```

mixed_polygon() itself is unchanged (still bd.Face(mixed_wire(...)), still planar-only) -- reach for mixed_wire()+cap_wire() instead the moment mixed_polygon() gives you a suspiciously small/wrong result, without needing to first prove the boundary really is non-planar.

Raises if `pieces` doesn't chain up into exactly one closed loop (e.g. a given Edge points the wrong way, or there's a real gap) -- same check, same message, as mixed_polygon().

polygon()

```
polygon(points)
```

Exposed to scripts | backend/brep.py:304 | Used in: —

A B-rep 2D filled polygon through `points` ([[x,y], ...] -- the same shape ocad.py's own circle()/square() return, and what OpenSCAD's polygon(points) takes). Unlike those, the winding order of `points` controls which way the resulting face's normal points (counter-clockwise -> +Z-facing normal, clockwise -> -Z-facing) -- which matters for linear_extrude()'s default extrude direction.

The loop is closed automatically (the last point connects straight back to the first) -- don't append `points[0]` again to "close" it yourself, that creates a zero-length closing edge (invalid geometry, since the polygon is then closed twice) rather

than actually closing anything; if ``points`` does end with a near-exact repeat of its own first point, that repeat is dropped for you.

That auto-added closing edge has to not cross ``points``' own outline anywhere, or the result is a self-intersecting (invalid) shape -- easy to hit by accident when ``points`` comes from an open curve (`bezier()`/`bspline_open()`/etc) whose two ends don't line up with its own overall direction. Checked and raised here with a clear error pointing at which two points cross, rather than silently building invalid geometry that only fails much later, cryptically, when the viewer tries to tessellate it.

Only builds a FLAT profile in the XY plane -- a nonzero z on any point RAISES rather than being silently dropped (a real user hit this: `axis_rot_1_pts()`-ing a point list out of the XY plane, then handing it to `polygon()` expecting the bend to carry through -- it didn't, since a flat Face has no way to represent it, and the z values were just discarded with no warning). If ``points`` is already positioned/bent in 3D, use `p_line3d(points, ...)` to trace it directly as a tube instead (no flattening); if you actually want a flat profile repositioned in 3D, build it flat here first and move/rotate the resulting real Shape afterward with `translate()/rotate()/axis_rot_1()` instead of pre-rotating the raw points.

square()

```
square(size=1, center=False)
```

Exposed to scripts | backend/brep.py:260 | Used in: —

A B-rep 2D rectangle -- same call convention as OpenSCAD's `square()`: ``size`` is either one number (a square) or `[x,y]`, ``center`` follows OpenSCAD's meaning (False: one corner sits at the origin; True: centered on the origin).

wire_from_edges()

```
wire_from_edges(edges, tol=None)
```

Exposed to scripts | backend/brep.py:8973 | Used in: hook.py

Stitches an UNORDERED, mixed-direction bag of real build123d Edges into ONE closed Wire -- e.g. a handful of edges picked one at a time with the probe list tool, in whatever order they happened to be clicked, not necessarily the order they connect in around the loop.

THIS IS NOT `mixed_wire()`: `mixed_wire()` walks ``pieces`` in the EXACT order given, connecting each item's end to the next item's start with a straight line if there's a gap -- correct for a hand-built profile (`[curve, corner_pt, corner_pt, ...]`) where you already know the walking order, but wrong for a set of Edges picked off an existing shape by clicking, where nothing guarantees the click order matches the geometric order, or that each Edge's own start/end (`position_at(0)/(1)`) points the way you happened to walk it. Feeding an out-of-order/mixed-direction Edge list to `mixed_wire()` doesn't fail cleanly -- it silently inserts a spurious straight "bridge" line between mismatched endpoints for every out-of-sequence Edge, which is exactly what breaks the result into multiple separate wires instead of raising something more informative at the actual problem.

`wire_from_edges()` skips that cursor-walking step entirely and hands ``edges`` straight to build123d's own `Wire.combine()`, which stitches by matching COORDINATES at shared endpoints regardless of input order or individual edge direction -- the same underlying call `mixed_wire()` itself ends with, just without the point-based bridging in front of it. Only takes real Edges (no plain `[x, y]` points mixed in, unlike `mixed_wire()/mixed_polygon()`) -- if you need to mix in straight corner points too, reorder ``edges`` into a genuine walk and use `mixed_wire()` instead.

```
e2 = [s3.edges()[i] for i in [3, 2, 0, 1]] # <- probe list tool: e2
w = wire_from_edges(e2)
pts = sample_curve(w, 50)
```

Raises the same way `mixed_wire()` does if ``edges`` doesn't chain into exactly one closed loop -- here that means a REAL gap (the edges given don't actually share endpoints all the way around), since ordering is no longer a factor. UNLESS ``tol`` is given (see below), that's still true: `Wire.combine()`'s own coordinate matching uses a very tight built-in tolerance, tight enough that edges from genuinely different construction paths (e.g. one edge sampled off a NURBS surface fit, another a real edge off a solid that's been through `split()/union()/mirror()`) can be visibly touching in the viewer and still fail to combine, off by some small but real amount from floating-point drift accumulated along two different paths to "the same"

point.

`tol`, if given, is a SECOND ATTEMPT that runs only after the exact-match attempt above has already failed -- it does NOT loosen or replace that first attempt (so a script that already worked keeps working identically, byte-for-byte, whether or not you pass `tol`).

With `tol` set, every possible (edge end) <-> (edge end) pairing across the WHOLE selection is considered up front and accepted shortest-distance-first (a standard "greedy edge" cycle-building heuristic) -- NOT a single-direction walk that only ever compares candidates against wherever it currently happens to be. That distinction matters on exactly the case `tol` exists for: several edges clustered close together (e.g. picking edges off a solid that's been union()ed with a near-duplicate mirror copy of itself only a fraction of a unit away almost everywhere) can make a single cursor's own nearest candidate the WRONG one, even though a different, unambiguous, genuinely-shortest pairing exists elsewhere in the same selection -- processing shortest-first instead locks in every unambiguous short match before a longer, more ambiguous one is ever even considered, regardless of which edge got looked at first. Once every edge has a matched partner at each end, a short straight bridge line is added at every matched pair that isn't ALREADY touching at build123d's own tight exact-match precision, and the ORIGINAL edges (never copied, reversed, or reordered) plus those bridges are handed to build123d's own Wire.combine() one more time -- letting real topology (not this function's own guess at direction) resolve the final connectivity, the same as the plain exact-match attempt at the very top of this function already does.

Pick `tol` a little larger than the actual gaps you're seeing (start small -- e.g. 0.01-1 depending on your model's own scale -- and increase only if it still fails) rather than something huge: too large a `tol` still risks matching an edge to the wrong neighbor when several candidates are ALL within `tol`, even with the shortest-first ordering above (it reduces, but can't fully eliminate, the risk when the true gap and a wrong-neighbor distance are close to each other) -- silently building a wrong loop instead of raising a clear error.

```
e3 = [s6.edges()[i] for i in [2]] + [sol2.edges()[i] for i in [11, 15, 19, 20]] # <- probe list tool: e3
s7 = wire_from_edges(e3, tol=0.5) # small real gaps between s6's and sol2's edges
```

Raises ValueError if, even with `tol` given, the edges still don't all chain into one connected sequence (a gap bigger than `tol` somewhere, or a genuinely missing/extra edge in the selection).

Planes & Coordinate Frames (3)

equation_of_plane()

equation_of_plane(pts)

Exposed to scripts | backend/brep.py:3627 | Used in: —

The equation of the plane through `pts` (3+ coplanar points), as [a,b,c,d] where $ax+by+cz=d$ -- built on normal_vector().

plane()

plane(normal, size=(100, 100), intercept=(0, 0, 0))

Exposed to scripts | backend/brep.py:8195 | Used in: —

A flat rectangular Face of the given `size` ([width, height]), lying in the plane through `intercept` with the given `normal` -- e.g. plane([1,0,0]) is a rectangle in the YZ plane; plane([0,0,1]) is a rectangle in the XY plane. ocad.py's own plane(), ported -- real B-rep, build123d's own Plane+Rectangle rather than the hand-built surface-from-2-vectors construction the mesh version needed. Mostly useful as a visual reference plane, or as the `target` for project_curve_on_face()/a cutting tool for split()- style operations you build yourself with build123d's own Shape methods.

Returns a genuine bare Face, not the Sketch that `Plane(...) \* Rectangle(...)` actually produces -- build123d's own Rectangle() returns a Sketch (and Sketch IS-A Compound, per build123d's own source: `class Sketch(Compound)`), so multiplying it onto a Plane still comes back as a Compound-wrapping-one-Face, not a bare Face. That distinction matters a lot for split(): passing that Compound straight through as `bisect\_by` reaches OCCT's own cut internals, which expect a real TopoDS_Face and fail there with a confusing "UVBounds_s(): incompatible function arguments ... invoked with TopoDS_Compound" -- unwrapping it with .faces()[0] here avoids that for every caller, not just split()'s own callers.

plane from equation()

```
plane_from_equation(e, size=(100, 100))
```

Exposed to scripts | backend/brep.py:3637 | Used in: hollow_cylinder_with_star.py, hook.py

The inverse of `equation_of_plane()`: builds a real rectangular Face lying exactly in the plane $ax+by+cz=d$, given $e=[a,b,c,d]$ (the same 4-number form `equation_of_plane()` returns). `ocad.py`'s own `plane_from_equation()`, ported onto this file's own real B-rep `plane()` -- e.g. `plane_from_equation(equation_of_plane(pts))` rebuilds a Face through the same plane as arbitrary `pts` (3+ coplanar points), and `plane_from_equation([0,0,1,40])` is the flat $z=40$ plane (handy as a `nlos()/psos()` target when "snap onto a flat plane at a given height" is what `nlos()/psos()` should be projecting against).

`size` is the rectangle's [width, height], same as `plane()`'s own `size` -- centered on the plane's own closest point to the origin, i.e. the foot of the perpendicular from (0,0,0) onto the plane.

Measurement & Analysis (11)

area()

```
area(shape)
```

Exposed to scripts | backend/brep.py:10003 | Used in: —

The shape's total surface area (every face's area, summed) -- `build123d`'s own `shape.area`, exposed here as a plain function to match `volume()`'s own convention (`volume(shape)` -> `shape.volume`, `area(shape)` -> `shape.area`).

This is the shape's REAL surface area, not its "shadow" -- a curved or folded surface's own area is always $\geq$ the area of the flat outline it casts when viewed from any one direction; those only coincide for a flat face viewed exactly head-on. For that "shadow" instead, see `projected_area()` below.

bb()

```
bb(prism)
```

Exposed to scripts | backend/brep.py:1132 | Used in: —

The 3D bounding-box size [width, height, depth] of a point list or nested list of point lists (e.g. `to_mesh()`'s vertex list, or a stack of sections) -- OR of a real B-rep Shape (`import_step()`'s own result, `cube()/cylinder()/etc`, any `split()/fillet()/etc` result): `build123d`'s own `shape.bounding_box().size`, unwrapped into this same plain [w, h, d] list so both kinds of input work with one function. Use `shape.bounding_box()` directly instead (see its own `.min`/`.max`/`.center`) if you need more than just the overall size -- e.g. `import_step()`'s own docstring above uses `.center()` to center a cutting plane on an imported part.

bb2d()

```
bb2d(sec)
```

Exposed to scripts | backend/brep.py:1153 | Used in: —

2D counterpart of `bb()` -- [width, height] of a 2D point list.

cog()

```
cog(sol)
```

Exposed to scripts | backend/brep.py:1161 | Used in: —

Center of gravity (plain centroid, not area/volume-weighted) of a point list or nested list of point lists.

draft_check()

```
draft_check(shape, direction, min_draft=1.0, tolerance=0.2)
```

Exposed to scripts | backend/brep.py:13029 | Used in: —

Checks every face of `shape` for UNDERCUTS and INSUFFICIENT DRAFT relative to a chosen pull direction -- the same analysis a real mold/die design package's "draft angle analysis" does, scriptable here instead.

`direction`: the pull direction ([x, y, z], e.g. [0, 0, 1] for a

top die pulling straight up) -- doesn't need to already be unit length, normalized internally.

`min\_draft`: the minimum acceptable draft angle in degrees

(default 1.0 -- a common injection-molding minimum; a deeper die-cast draw or a shallow snap-fit wall might need something different -- pass whatever your process actually requires).

`tolerance`: passed straight through to this app's own `to_mesh()`

to control how densely each face gets sampled -- a face's normal can vary a lot across a curved surface, so checking only its center can miss a locally-undercut patch on an otherwise-fine face; a smaller tolerance samples more points (slower, more thorough).

A face's DRAFT ANGLE here is measured between the face and the pull direction itself (not the face's normal): a wall running exactly PARALLEL to the pull direction has 0 degrees of draft (it will scuff/gall on ejection, no clearance at all); a wall that tilts outward, away from the core, as you go in the pull direction has some POSITIVE draft (the more positive, the more clearance); a face that tilts the other way -- inward, back over itself -- has NEGATIVE draft, meaning the tool would have to pass back through material it already cleared to release: a genuine UNDERCUT, needing a slider/side-action (or an entirely different pull direction) rather than just "more draft", the same class of problem
17_slider_extraction_from_cavity_wall.py's example works around.

Returns a list of dicts, one per face, SAME ORDER/INDICES as `shape.faces()` (so they line up with every other probe/pick tool in this app):

```
{"index": i, "min_draft_deg": ..., "status": "undercut" |  
  "insufficient" | "ok"}
```

"min_draft_deg" is the WORST (smallest) draft angle found anywhere on that face -- negative means "undercut", 0 up to `min\_draft` means "insufficient" (releases, but shallower than your minimum), at or above `min\_draft` means "ok".

```
e1 = [c.faces()[i] for i in [3, 7]] # pulled straight from the  
                                     # Pick tool, faces=kind  
report = draft_check(c, [0, 0, 1], min_draft=1.5)  
for r in report:  
    if r["status"] != "ok":  
        print(r)
```

IMPORTANT LIMITATION -- one status covers the WHOLE face: a face that's mostly fine but dips bad somewhere on it still gets that one worst status for its entire area. Usually harmless (most real part walls/ribs are already their own separate, small faces), but on one large curved face whose normal sweeps through a wide range of directions -- a plain sphere is the extreme case, a single Face spanning nearly the whole surface -- this can mark the ENTIRE face as one uniform status even though roughly half of it is actually fine. Use `draft_check_points()/draft_check_markers()` instead for an accurate picture on shapes like that (or just to see a smooth per-point gradient rather than hard face boundaries).

To see THIS (per-face) version, color each face by its own status and `show()` them one at a time (this app's `show()` only ever takes one shape per call, so a loop of individually-colored `show()` calls is how anything multi-colored gets displayed here; see e.g. `mirror_surface()`'s own example):

```
colors = {"undercut": "red", "insufficient": "orange", "ok": "green"}  
faces = list(c.faces())  
for r in report:  
    show(color(faces[r["index"]], colors[r["status"]]))
```

UNVERIFIED against a real build123d install (see module docstring) -- built on `Face.normal_at(point)` and this app's own `to_mesh()` (itself already flagged UNVERIFIED for the same reason). If every draft angle comes back roughly 180 degrees off from what you'd expect (e.g. a flat top face facing straight along `direction` reading as -90 instead of +90), that's a face-

orientation sign flip -- the first thing to check here.

draft_check_markers()

```
draft_check_markers(shape, direction, min_draft=1.0, tolerance=0.2, marker_size=None)
```

Exposed to scripts | backend/brep.py:13160 | Used in: —

Convenience wrapper around `draft_check_points()`: builds a small sphere marker at every sample point, grouped by status, ready to `show()` directly (3 `show()` calls total -- one per status -- instead of one per point):

```
markers = draft_check_markers(c, [0, 0, 1], min_draft=1.5)
colors = {"undercut": "red", "insufficient": "orange", "ok": "green"}
for status, m in markers.items():
    if m is not None:
        show(color(m, colors[status]))
```

This is the accurate, per-point picture -- on a plain sphere, for instance, you'll see roughly one hemisphere's worth of green dots and the other red, meeting in a clean band around the "equator" relative to `direction``, instead of `draft_check()`'s single whole- shape worst-case status (see its docstring's own limitation note).

`marker_size``, if not given, defaults to ~0.8% of the shape's own largest bounding-box dimension -- small enough not to obscure the surface, big enough to actually see at normal zoom.

Returns {"undercut": shape_or_None, "insufficient": ..., "ok": ...} -- a status with no points at all (e.g. a shape with zero undercuts) comes back None rather than an empty/invalid group().

draft_check_points()

```
draft_check_points(shape, direction, min_draft=1.0, tolerance=0.2)
```

Exposed to scripts | backend/brep.py:13126 | Used in: —

The same draft-angle check as `draft_check()`, but returns every individual SAMPLE POINT instead of one aggregated worst-case status per face -- reach for this (not `draft_check()`) whenever a shape has a large curved face that's fine over MOST of its area but dips bad somewhere on it. A plain sphere is the extreme example: its outward normal sweeps through every possible direction across ONE single Face, so `draft_check()`'s per-face worst-case marks the WHOLE sphere as one uniform status even though roughly half its surface is actually fine -- this instead keeps each sample point's own true status, so you get a smooth, accurate boundary between good and bad regions rather than one hard face-wide cutoff.

Same `direction`/min_draft`/tolerance`` meaning as `draft_check()`.

Returns a flat list across every face, one entry per sample point:

```
{"point": [x, y, z], "draft_deg": ..., "status": "undercut" |
"insufficient" | "ok"}
```

Meant to be visualized as a "heat map" of small colored dots rather than colored whole-faces -- see `draft_check_markers()` for a ready-to-show() version of exactly that.

max_fillet_radius()

```
max_fillet_radius(shape, edges, tolerance=0.1, max_iterations=10)
```

Exposed to scripts | backend/brep.py:6384 | Used in: —

The largest radius that `fillet(shape, edges, radius)` will actually succeed with, for this exact `shape`/edges`` selection -- a thin wrapper around `build123d's` own `Shape.max_fillet()` (a binary search over candidate radii, each checked by actually attempting the fillet). Directly answers the question `fillet()`'s own error message points you at ("...try a smaller value or use `max_fillet()` to find the largest valid fillet radius") without you having to guess-and- check radii by hand:

```
edges = [c.edges()[i] for i in [-4, -8, -9, -10, -11, -12]]
r = max_fillet_radius(c, edges)
show(fillet(c, edges, r))
```

`tolerance`/`max\_iterations` control the binary search's precision and effort, same meaning as build123d's own parameters -- the defaults are usually fine, but see the important caveat below for models with small features relative to their overall size.

IMPORTANT (verified against build123d's own `max_fillet()` source): the binary search's STARTING window is always (0, 2 * shape's own overall `bounding_box().diagonal`) -- the whole model's size, not anything related to the specific edges/feature you're testing. On a model where the actual achievable radius is small relative to the whole shape (a thin wall on a much bigger part, e.g. a 2mm wall on an 80mm-tall tube), the search can spend most/all of its `max_iterations` budget just narrowing that huge starting window down into the right neighborhood, and run out before ever converging -- reporting "couldn't find ANY radius" even though a real small radius likely DOES exist, it just wasn't found in time. This is a genuine false negative, not proof of unfilletability -- raising `max_iterations` well above the default (e.g. 30-40) CAN and often does fix this, and is the first thing to try before concluding an edge is really unfilletable:

```
max_fillet_radius(shape, edges, max_iterations=40)
```

If THAT still raises "Failed to find the max value within <tolerance> in <max_iterations>", NOW it's more likely a genuinely different, more fundamental problem: at least one edge in your selection can't be filleted no matter how small the radius is -- not "1 was too big", but "this particular edge is unfilletable" (a seam edge on a periodic/cylindrical surface, or an edge at a vertex where too many filleted faces would need to meet, are the two most common real causes). The fix at that point is to find out WHICH edge in the group is the problem by testing them one at a time (still with a generous `max_iterations`):

```
edges = [c.edges()[i] for i in [-4, -8, -9, -10, -11, -12]]
for i, e in enumerate(edges):
    try:
        print(i, max_fillet_radius(c, [e]))
    except Exception as exc:
        print(i, "UNFILLETABLE:", exc)
```

then drop whichever index(es) print UNFILLETABLE from the group (or fillet them separately with `chamfer()` or a different radius/edge choice instead).

`edges` can also be a single bare Edge (not wrapped in a list) -- e.g. `max_fillet_radius(shape, e)` works the same as `max_fillet_radius(shape, [e])`, so a per-edge testing loop like the one above doesn't need to wrap each `e` itself -- same convenience as `fillet()/show_edges()/label_edges()`. (Forgetting this wrapping used to surface as a confusing "'Edge' object is not iterable" buried inside the generic "couldn't find ANY radius" message below -- a plain Python usage error masquerading as a geometric one; now it just works instead.) A bare Wire/Shell/Solid/Compound/Face (e.g. straight out of `mixed_wire()/wire_from_edges()`) also works directly -- expanded to its own real constituent edges automatically. Passing a bare Wire specifically used to surface as a much more confusing OCP-internals error deep inside the binary search ("Add(): ... Invoked with: ... TopoDS_Wire") -- fixed the same way.

projected_area()

```
projected_area(shape, direction, n_samples=16, strict=True, weld_tol=None)
```

Exposed to scripts | [backend/brep.py:10112](#) | Used in: —

The area of the "shadow" `shape` would cast on a screen perpendicular to `direction` -- its silhouette area as seen looking straight along `direction`, NOT its real surface area (see `area()` above for that; they're only equal for a flat face viewed head-on). `direction`: [x, y, z], the direction you're looking ALONG (e.g. [0, 0, -1] to see the shape from above, looking down).

Built on the SAME hidden-line-removal this app's own `technical_drawing_views()` uses for engineering drawings (`Shape.project_to_viewport()`, OCCT's real HLRBRep_Algo) rather than a boolean union of every face's own individual projection. That alternative -- `project()` every face onto one flat target plane, then `union()` the results to avoid double-counting overlaps -- sounds simpler, but every one of those projected faces would land on the exact same plane, and `union()`'s own docstring documents a CONFIRMED real OCCT fragility fusing large, exactly coplanar shapes ("AttributeError: 'NoneType' object has no attribute 'NbNodes'") -- precisely what that approach guarantees on every call. Using HLR instead sidesteps that risk entirely: `project_to_viewport()` already resolves true visibility (which parts of the shape are actually visible from `direction`, correctly leaving a genuine hole in the silhouette where the shape is actually

hollow from that view, never double-counting a near/ far surface pair), so this just samples the VISIBLE edges it returns into 2D polylines (same `position_at(t)` sampling `_hlr_polylines()` uses, but skipping the sampling entirely on straight edges -- their own 2 endpoints already describe them exactly), stitches them into closed loops with the $O(n)$ `_stitch_loops_fast()` (NOT this app's shared `contiguous_chains()`, which is $O(n^2)$ and a real, reported hang on complex geometry -- see `_stitch_loops_fast()`'s own docstring), and sums each loop's own area.

Silhouette loops from HLR can legitimately nest (an outer boundary plus one or more hole boundaries, e.g. a flange's own bolt holes, or a pipe/torus viewed straight down its axis) -- but a raw edge's own parametric direction has no reliable relationship to whether it's tracing an outer boundary or a hole from THIS particular view, so this does NOT just add up each loop's own signed shoelace area and trust the sign (that would only work if every loop's winding direction reliably meant the same thing, which isn't a safe assumption for arbitrary HLR edges). Instead each loop's UNSIGNED area is added or subtracted based on actual geometric nesting depth (`point_in_polygon()` against every other loop, standard even-odd fill rule: contained in an even number of other loops -- 0, 2, 4, ... -- counts as solid material; an odd number means it's a hole, or a hole inside a boss inside a hole, etc.), which is correct regardless of any individual edge's own winding.

``n_samples`` controls how finely curved visible edges are sampled into polyline points before computing area -- same tradeoff as `technical_drawing_views()`'s own ``n_samples`` (higher = smoother curves = more accurate area on anything that isn't all straight edges, at the cost of more sampling work).

``strict`` (default `True`): raises `ValueError` if any of the visible- edge chains `_stitch_loops_fast()` built didn't actually stitch back into a CLOSED loop (its own first and last point coinciding) -- REAL BUG FIX (user report: `projected_area()` on a complex organic part -- a steering wheel, multiple spokes/blended surfaces -- came back roughly 1/3 of the true manually-computed minimum area). The loop-building step used to accept ANY chain with ≥ 3 points, closed or not, straight into the area sum -- an OPEN chain has no real "enclosed area" at all; folding it into the shoelace formula anyway implicitly closes it with a straight line from its last point back to its first, which has nothing to do with the actual shape's boundary and can make the total badly wrong in either direction (usually an undercount in practice, since the same numerical fragmentation that broke one loop into several open pieces tends to scatter the true boundary's own area across those pieces' bogus closing-line contributions rather than the real silhouette). This false-closure fabrication is now never done: a chain has to genuinely close before it's treated as a polygon at all. Complex organic/heavily-blended geometry (lots of curved surfaces meeting at shared vertices) is exactly where a handful of edges can fail to weld under the stitcher's own coordinate-rounding match (see `_stitch_loops_fast()`'s own docstring) -- when that happens, ``strict=True`` raises rather than silently returning a number that LOOKS plausible but may have entire real regions of the silhouette dropped or double-counted; the error message reports how much unstitched edge length was found. Pass ``strict=False`` only if you specifically want the old best-effort behavior back (excluding the open chains from the sum, same as ``strict=True`` computes internally, just without raising) -- e.g. for a quick approximate number on a model you already know has imperfect geometry.

``weld_tol``: how close 2 sampled endpoints need to be (in ``shape``'s own units) to be treated as the SAME point when stitching visible edges back into loops (see `_stitch_loops_fast()`'s own ``tol``). ``None`` (the default) derives it from ``shape``'s own bounding-box diagonal (roughly `diagonal * 1e-5`, floored at `1e-6`) rather than a fixed constant -- ANOTHER REAL FIX from that same customer STEP import: even after the ``strict`` fix above stopped silently fabricating area from open chains, that model's own visible edges still failed to weld into closed loops at all (hundreds of open chains, ``strict=True`` correctly raising every time) under the OLD hardcoded 0.0001 absolute tolerance -- a real STEP import from another CAD system, combined with OCCT's own HLR splitting edges at visibility transitions (`position_at()` evaluated independently on each resulting piece), can easily accumulate more floating-point noise than that at a shared vertex, and that noise naturally scales with the model's own coordinate magnitudes (a large part's transform pipeline has more room for it than a small one) -- a fixed constant can't track that, but ``diag``-relative can. Pass this explicitly (try something a bit looser, e.g. 10x the auto value) if ``strict=True`` still raises with the default; going too loose risks welding 2 genuinely DIFFERENT nearby points together instead (silently wrong topology, no error) -- so widen it gradually rather than jumping straight to something huge.

projected area mesh()

```
projected_area_mesh(shape, direction, resolution=600, tolerance=0.1,
angular_tolerance=0.3)
```

Exposed to scripts | backend/brep.py:12268 | Used in: —

A ROBUST fallback for `projected_area()` -- reach for this one when THAT one is too slow or genuinely hangs on a particular shape.

REAL, CONFIRMED motivation (real customer STEP part): `projected_area()` (built on OCCT's own `Shape.project_to_viewport()` hidden-line-removal -- see its own docstring) was still taking 30+ minutes / never finishing on a real automotive bracket even AFTER union()-ing its 5 originally-separate solids into one -- and that fused shape had only 39 faces, ruling out "just too many faces/edges" as the explanation. That points at OCCT's own HLR algorithm having a genuine pathological slowdown on this shape's specific geometry (near-tangent surfaces, an ill-conditioned trim curve from the union, etc.) -- something NO amount of pre/post-processing in `projected_area()`'s own Python code can fix, since it happens inside OCCT's own C++ implementation, with no way to safely interrupt it short of killing the whole process (see `kernel_manager.py`'s `restart()` for why THAT'S the only thing that reliably works on a truly stuck native call).

This sidesteps `project_to_viewport()` entirely -- it never calls it, so it can't inherit its fragility. Instead it uses the SAME tessellation pipeline the 3D viewer already displays this exact shape with just fine (`to_mesh()` -- if `show()` renders it, this can tessellate it): every triangle gets projected onto a 2D plane perpendicular to `direction`, then rasterized onto a `resolution` x `resolution` grid covering the projected bounding box. A grid cell counts as covered the instant ANY triangle overlaps it -- boolean, not additive -- which handles occlusion/overlap for free: a near surface and a far surface covering the same 2D region (or two triangles from the same curved face overlapping at the raster scale) is never double-counted the way naively summing every triangle's own projected area would be.

This is an APPROXIMATION -- accuracy is bounded by `resolution` (the grid cell size), unlike `projected_area()`'s own exact loop-geometry answer -- so try `projected_area()` FIRST and only reach for this if that one is unreasonably slow or raises. The default `resolution` (600, i.e. up to 360,000 cells) is usually accurate to a small fraction of a percent for a normally-proportioned part; raise it for a very thin/elongated silhouette, where 600 cells across the LONG axis still leaves the short axis coarse (this always keeps the grid's own aspect ratio matched to the shape's own projected bounding box, so `resolution` is "cells across the longer side").

`tolerance`/`angular_tolerance`: passed straight through to `to_mesh()` -- finer tessellation costs more triangles to rasterize but tracks curved surfaces more closely.

volume()

```
volume(shape)
```

Exposed to scripts | backend/brep.py:9990 | Used in: —

The shape's EXACT enclosed volume, computed by OCCT from the real B-rep geometry -- not a discrete/mesh estimate like the divergence- theorem-over-triangles trick used to sanity-check `mcm()`/ `repair_mesh()` output elsewhere in this project. Useful as a ground- truth check if you ever want to compare a B-rep shape's true volume against a mesh reconstruction of "the same" shape.

Primitive Solids (6)

box()

```
box(size=1, center=False)
```

Exposed to scripts | backend/brep.py:161 | Used in: —

A B-rep rectangular solid -- `size` is either one number (a cube) or `[x,y,z]` (a box), `center` follows real OpenSCAD's meaning (False: one corner sits at the origin; True: centered on the origin). Builds a build123d Box under the hood.

c_solid()

```
c_solid(s1, planarize=True)
```

Exposed to scripts | backend/brep.py:9480 | Used in: —

A solid lofted through an explicit LIST of cross-sections, each given as a raw `mixed_polygon()`-style piece list rather than an already-built Face -- the "many independent sections" counterpart to `solid_line_vector()` above (which pushes a SINGLE section along one vector) and the `mixed_polygon()`-based counterpart to `c_polygon()`-driven lofts (which smooth a rough/organic outline instead of tracing it exactly).

``s1`` is a list of cross-sections; each item ``p`` in it is itself a list of plain 2D/3D points and/or real build123d Edges -- exactly what `mixed_polygon()` takes (straight lines between consecutive points, any given Edge dropped in as-is for that stretch of the boundary) -- see `mixed_polygon()`'s own docstring for the full rules, including that each ``p`` must chain into exactly one closed loop or `mixed_polygon()` raises. Each section needs to already be positioned where it belongs, same requirement as `loft()`'s own sections -- but since ``p`` is a raw POINT list here, not a Face yet, that means shifting the points themselves (plain Python/numpy math, or the ocad bridge's `o.translate()`, which -- unlike `brep.py`'s own `translate()/rotate()`, which only operate on already-built Shapes -- works directly on raw point lists like these).

Under the hood this is just ``loft([mixed_polygon(p) for p in s1])`` -- a straight-edged Face per section, smoothly lofted (`loft()`'s default ``ruled=False``) between them. If you need a piecewise- straight ("ruled") loft instead, or want to mix `mixed_polygon()` sections with sections built some other way, skip `c_solid()` and call ``loft(*faces, ruled=True)`` directly -- `c_solid()` always uses the smooth default and always builds every section via `mixed_polygon()`.

```
diamond = [[10, 0], [0, 10], [-10, 0], [0, -10]]
square = [[7, 7], [-7, 7], [-7, -7], [7, -7]]
at_z = lambda pts, z: [[x, y, z] for x, y in pts]
rings = [at_z(diamond, 0), at_z(square, 15), at_z(diamond, 25)]
show(c_solid(rings))
```

With `planarize=True` (the default), each section that's a plain point list (no Edge mixed in) is first run through `planarize_ring()` -- projected onto its own best-fit plane, see that function's own docstring -- before `mixed_polygon()` is tried, so a section that's only a fraction of a unit out of plane (independently computed points drifting station to station, same real case `solid_from_wedge_grid()` handles) still builds instead of raising "wires not planar". A section containing a real Edge is left untouched either way (`planarize_ring()` can't project an Edge's own control points the same way a plain point does, so it's skipped automatically -- not something you need to steer around by hand). Pass `planarize=False` to restore the old always-exact behavior.

cube()

```
cube(size=1, center=False)
```

Exposed to scripts | backend/brep.py:176 | Used in: —

Alias for `box()` under OpenSCAD's own name for this primitive.

cylinder()

```
cylinder(r1=None, r2=None, h=1, r=None, center=False, s=None)
```

Exposed to scripts | backend/brep.py:181 | Used in: —

A B-rep cylinder or (truncated) cone -- same `r1/r2/h/r/center` parameters as real OpenSCAD's `cylinder()`. ``s`` (segment count / \$fn) is accepted only for drop-in call compatibility and is IGNORED: this is exact geometry, there's no facet count to choose -- a cylinder here really is round, not a many-sided prism approximating one.

`r1/r2` both given (and different) -> `build123d Cone(r1, r2, h)`. Otherwise (`r`, or matching `r1==r2`) -> `build123d Cylinder(radius, h)`.

helix()

```
helix(radius=10, pitch=10, turns=1, center=(0, 0, 0), direction=(0, 0, 1),
lefthand=False)
```

Exposed to scripts | backend/brep.py:8834 | Used in: drill.py, tap.py

A helical Wire (a coil/spring/thread path) -- real B-rep, build123d's own native Helix curve (exact, not a polyline approximation), of the given `radius` and `pitch` (rise per full turn), `turns` long, starting at `center` and winding along `direction`. ocad.py's own helix(), ported (its `number\_of\_coils`/`step\_angle` become `turns`/build123d's own exact curve -- no facet-angle step needed). Feed it straight to sweep_sec2path() as path3d for a coil spring or a screw thread:

```
show(sweep_sec2path(circle(1), helix(radius=10, pitch=5, turns=4)))
```

sphere()

```
sphere(r=1)
```

Exposed to scripts | backend/brep.py:205 | Used in: hook.py

A B-rep sphere -- same `r` meaning as ocad.py's sphere().

Extrude, Sweep, Loft & Revolve (9)

extrude_wave2path()

```
extrude_wave2path(w1, c1)
```

Not yet exposed to scripts | backend/brep.py:3342 | Used in: —

Extrudes the 2D/3D wave profile `w1` (its own X read as "height off the path", Y as "distance travelled along the path") along the 3D path `c1` -- ocad.py's own extrude_wave2path(), ported onto this file's own rot_p(): at each path point, tilts+turns w1's local cross-section to match that point's own tangent direction, using the same y/z rotation-angle decomposition the original used.

linear_extrude()

```
linear_extrude(profile, h=1, center=False, twist=0, scale=1, dir=None)
```

Exposed to scripts | backend/brep.py:3791 | Used in: —

Extrudes a 2D profile (from circle()/square()/polygon(), or any build123d Sketch/Face) into a real solid by height `h` -- same call convention as ocad.py's own linear_extrude(sec, h, a, steps), just real B-rep in and out. `center` follows OpenSCAD's linear_extrude(..., center=...) meaning (False: extrudes from the profile's own plane up to h; True: centered on that plane instead).

Extrudes along +Z by default -- but that's really "the profile's own local normal", which only happens to be +Z because circle()/square()/polygon() build flat in the XY plane by default. If you've since rot()/translate()d the profile (or its winding order makes its normal point the "wrong" way), the actual extrusion direction moves with it -- pass `dir` ([x,y,z], only its direction matters, `h` still controls the distance) to pin the extrusion to a specific direction regardless of the profile's own orientation, e.g. to match a surface_line_vector() sweep vector exactly:

```
a = rot('x90', polygon(cr2dt(...), s=10)))
wall = surface_line_vector(a, [0, 100, 0]) # side surface only
solid = linear_extrude(a, h=100, dir=[0, 1, 0]) # the real solid
```

twist/scale (OpenSCAD's linear_extrude(twist=, scale=) -- a helical/ tapered extrusion) are NOT implemented yet: build123d's own extrude() has no direct equivalent for either (that shape needs a loft between rotated/scaled copies of the profile, or a genuine helical sweep, neither wired up here yet) -- passing a non-default value raises rather than silently extruding straight with no explanation.

`profile` can also be a plain closed point list instead of an already-built Shape -- e.g. Sketch on Face's own "<name>_3d" companion variable (built by to_plane()), a profile already placed on an arbitrary plane in 3D but not yet a real build123d object. Same "either a Shape or a raw point list" support solid_line_vector()/surface_line_vector() already offer -- and for exactly the same reason: a raw list is delegated straight to solid_line_vector() below, which (for a raw list) runs it through step_export.py's own exact B-rep reconstruction first, so a faceted 2D-sketch circle comes out as a round prism here too, not a visibly N-gon one -- see solid_line_vector()'s own docstring for the full explanation.

loft()

```
loft(*sections, ruled=False)
```

Exposed to scripts | backend/brep.py:8693 | Used in: —

A solid lofted through a sequence of 2D cross-sections -- e.g. ``loft(square(20, center=True), translate([0,0,20], circle(8)))`` smoothly blends a square base into a round top 20 units up. Real B-rep, build123d's own native `loft()` (an exact ruled/smooth surface between the section boundaries, not a stack of thin mesh slices) -- accepts 2 or more sections, each already positioned where you want it (`translate()`/`rotate()` them into place first, same as `sweep_sec2path()`'s section). This is what `ocad.py`'s `swp_prism_h()` and the various "stitch two rail curves together" part-building tricks are standing in for by hand -- build123d's `loft()` does the same underlying job natively and more robustly, so those weren't ported as separate point-list functions. (`ocad.py`'s `prism()` itself IS ported below, as a real-B-rep override that calls this `loft()` internally -- see `prism()`'s own docstring.)

``ruled`` controls how the surface behaves AT each intermediate section, which matters a lot as soon as there are 3+ sections:

```
False (default): a smooth spline surface threading through every
section with continuous tangency -- if section 2's radius
jumps up and section 3's jumps back down, the surface eases
into and out of that bulge rather than kinking, so you will
NOT see a sharp corner exactly at an intermediate section's
height even though its radius is a local max/min. This is
almost certainly why a profile built from rings of varying
radius (e.g. a stack of circles at different heights/radii)
comes out looking rounded/blobby instead of angular.
```

True: straight-line ("ruled") surface between each consecutive

```
pair of sections instead -- ends up as a stack of conical
frustums, with a visible sharp corner exactly at every
intermediate section's height. This is what you want if the
rings/sections were meant to define a piecewise-straight
profile (a vase with angular shoulders, a stepped cone, etc).
```

```
rings = [translate([0,0,h], circle(r)) for h, r in
          [(0,10),(10,14),(20,9),(30,15),(40,8)]]
show(loft(*rings, ruled=True)) # sharp corner at each ring
```

loft_fillet()

```
loft_fillet(line_a, line_b, spine, s=20, closed_loop=0, ruled=False)
```

Exposed to scripts | backend/brep.py:4463 | Used in: —

Builds a real B-rep SOLID filling the corner between two surfaces, by lofting through MANY small closed cross-section wedges (one per station) instead of fitting a single open surface patch (`convert_3lines2fillet()`+`surface_from_points_grid()` above) -- the real B-rep equivalent of `ocad.py`'s own `swp_c()`-based closed- section sweep. Same 3-line input (`spine/line_a/line_b` -- see `convert_3lines2fillet()`'s own docstring for the full recipe for deriving all 3, including the mesh-based `two_solids_intersection()` route for genuinely hard cases where a real B-rep boolean struggles).

At every station, builds a small CLOSED wedge profile: the bezier arc from `line_a[i]` through `spine[i]` (pulled toward it) out to `line_b[i]`, then back to `spine[i]`, closed back to `line_a[i]` (`mixed_polygon()` does the explicit closing build123d has no auto-close for, matching `swp_c()`'s own real auto-close-last-to-first behavior). Lofting through the whole stack of these wedges fills exactly the corner between the two original surfaces, pinched down to the sharp spine at every cross-section.

```
blend = 2
n = 40
spine = sample_curve(intersection(a, b), n)
line_b = sample_curve(intersection(offset_face(a, blend), b), n)
line_a = sample_curve(intersection(a, offset_face(b, blend)), n)
solid = loft_fillet(line_a, line_b, spine, s=10)
show(solid)
```

Each station's wedge needs to be at least CLOSE to planar for `mixed_polygon()`'s underlying `Face(Wire)` construction to succeed -- true for most reasonable blend widths relative to how sharply the spine curves, but a spine that twists tightly relative to ``blend`` can produce a station too warped to build (`mixed_polygon()` or `loft()` itself will raise). If you hit that, `surface_from_points_grid(convert_3lines2fillet(...))` is the more robust fallback -- fits one smooth surface through the whole grid at once instead of building/lofting individual station Faces, trading the solid result for an open patch you'd thicken() yourself.

``closed_loop`/`ruled`` mean the same as `convert_3lines2fillet()`'s/ `loft()`'s own (see their docstrings) -- ``ruled=True`` if you want a piecewise-straight fillet between stations instead of the smooth default.

Builds the wedge grid then hands it straight to `solid_from_wedge_grid()` to actually loft it -- so every fix that applies there applies here too, automatically: each station is `planarize_ring()`-projected onto its own best-fit plane first (a spine that twists tightly relative to ``blend`` producing a station that's only SLIGHTLY off-plane no longer needs to hit the `surface_from_points_grid()` fallback below -- it just gets planarized like any other borderline station), and `closed_loop=True` gets the sliver-free tube+overlapping-closer construction instead of a single `loft()` through a station repeated at both ends.

`p_line3d()`

```
p_line3d(path, d=1, closed=0, transition='round')
```

Exposed to scripts | [backend/brep.py:8092](#) | Used in: [drill.py](#)

A tube/rope of diameter ``d`` following ``path``, real B-rep -- `ocad.py`'s own `p_line3d()/p_line3dc()` modules, ported. Handy for turning a bare 3D point list (`turtle3d()`'s output, `cr3dt()`'s output, any hand-built polyline) straight into a real solid you can `show()` directly, without setting up `sweep_sec2path()`'s 2D- section-and-path call yourself -- e.g. a quick tube/wireframe-style rendering of a 3D path, or an actual rope/cable model. ``closed=1`` also connects the last point back to the first (`ocad.py`'s separate `p_line3dc()`) -- same `closed_loop` convention as `turtle3d()/sweep_sec2path()`.

``path`` can also be a real Shape instead of a raw point list -- a Wire/Edge (e.g. `arc_edge()`, or a Face's own `.wires()[0]`), or a filled 2D shape like `polygon()/circle()/cr2dt()+polygon()`'s output (its boundary outline is traced automatically, same "pull the outline out of a filled Face" step `sweep_sec2path()` already does for a Shape `path3d` -- see `_as_path_wire()`). A genuinely closed Shape (a closed Wire, or a filled Face's boundary -- `polygon()`'s output, say) skips the end caps automatically, the same as ``closed=1`` does for a raw point list, since there's no open end to round off either way.

REAL PERFORMANCE FIX: this used to build a capsule (hull) of two spheres, or two cubes with `rec=1` -- that parameter's gone now, see below) for EVERY consecutive pair of points, then `union()` them all together -- one `hull()` plus one boolean per segment, chained. That was a real user-reported slowdown: unioning N solids one at a time is $O(N)$ booleans, each re-solving against the whole accumulated result so far, and a path with even a few dozen points made this genuinely slow. Now it's ONE `sweep_sec2path()` call along the whole path plus 2 small `sphere()` caps unioned on at the very ends -- same rounded-tube-with-rounded-ends look, a handful of real B-rep operations total instead of one per segment (the exact fix applied by hand in `examples/21_drill_bit.py` before it was folded back into this function directly).

No more ``rec`` (square vs round caps) -- a round tube with round end-caps is what this function is actually for now; if you specifically want flat/square caps instead, call `sweep_sec2path(square(d, center=True), path)` yourself.

``transition`` is passed straight through to `sweep_sec2path()` (see its own docstring for what it does) -- "round" (the default) is fine for most paths, but a path with many points and/or tight, sharp turns relative to ``d`` can hit the same fragility any long multi-segment sweep can (see `examples/20_3d_knots.py` for the full story on that failure mode) -- try "right_corner" or "transformed" if you hit it here too. CONFIRMED via a real user report: a many-point, non-planar path (e.g. `axis_rot_1_pts()`-bent) with the default "round" raised `AttributeError: 'NoneType' object has no attribute 'NbNodes'` -- a real OCCT/OCF error in "round"'s per- corner fillet construction -- and `transition="right_corner"` fixed it. If you see that exact error, this is almost certainly why.

``closed=1`` skips the end caps entirely (there's no exposed end left to round off once the loop connects back to itself) -- just the one sweep, closed all the way around.

prism()

```
prism(sec, path, type=1, ruled=False)
```

Exposed to scripts | backend/brep.py:8739 | Used in: drill.py, tap.py

A solid built by offsetting a 2D section by a sequence of (offset, height) steps and lofting through the resulting stack of cross-sections -- real B-rep.

Same call signature and offset math as ocad.py's own prism(sec, path, type) -- `path` is a list of [offset, height] pairs (e.g. from corner_radius()/pts1()), `type` selects between offset()'s two offset methods (1 or 2, see ocad.py's offset() docstring) -- so an existing script written against ocad.py's prism() needs no changes to get a real B-rep solid back automatically; this overrides that mesh-only version by the same name (see python_eval.py's make_namespace()), it doesn't need to be piped through loft() by hand.

`sec` is usually itself a discretized point list -- e.g. a circle sketched in the 2D Sketch panel, which is really a many-sided polygon (60 points is a typical default), not a genuine curve. Building each cross-section as a literal straight-edged polygon() Face and lofting between them (the naive approach) is real B-rep, but bakes every one of those points in as its OWN permanent vertical seam edge the whole height of the shaft -- e.g. a 60-point circle section produces 60 visible facet lines running top to bottom, no matter how the sections are lofted, which looks "not pure" even though it's exact. So this tries a smarter reconstruction FIRST: the same exact-STEP-round-trip step_export.py uses for solid_line_vector()/swp()/swp_c() -- fitting real curves through long runs of near-collinear points (a fine circle approximation's points are all only a few degrees apart) and only keeping a sharp corner where `sec` actually has one -- across the WHOLE stack of sections at once, so a faceted circle section comes back as one genuinely round shaft, not 60 flat strips. This needs every section (each offset() result, one per `path` entry) to have the exact same number of points -- true whenever `type`'s offset math doesn't add/ remove points, i.e. almost always -- and falls back to a plain loft() between literal straight-edged Faces (see `ruled` below) if it doesn't, or if the exact reconstruction otherwise isn't possible for this particular profile.

`ruled` only affects that fallback path (the smart reconstruction above doesn't take it -- it always fits smooth curves through non-corner points): passed straight through to loft() (see its own docstring). Defaults to False (one smooth, tangent-continuous surface threading through every section) rather than loft()'s own default of True, since `path` is often corner_radius()'s own dense output -- ruled=True there would put a separate flat/conical face between every one of those many closely-spaced sections. Pass ruled=True explicitly if `path` genuinely only has a few, deliberately-placed [offset, height] steps and you DO want a sharp corner at each one (a stepped/shouldered profile, not a smooth taper).

example: sec=corner_radius(pts1([[-15,0,2.4],[0,15,3],[30,0,3],[0,-15,2.4],[5,0,2.4],[0,20,7],[-40,0,7],[0,-20,2.4]]),10)
path=corner_radius(pts1([[-2,0],[-2,0,2],[0,7,2],[-2,0]]),10) sol=prism(sec,path) show(sol)

rotate_extrude()

```
rotate_extrude(profile, angle=360)
```

Exposed to scripts | backend/brep.py:3870 | Used in: —

Revolves a 2D profile around the Z axis into a solid -- same idea as OpenSCAD's rotate_extrude(angle=...): `profile`'s local X becomes the resulting solid's radius, local Y becomes its height, same convention real OpenSCAD's rotate_extrude has. `profile` should lie on the +X side of the Z axis (a profile straddling or crossing the axis produces self-intersecting/invalid geometry).

REAL BUG FIX (found via a real user report -- "rotate_extrude did not work"): circle()/square()/polygon()/cr2dt()+polygon() all build `profile` flat on the XY plane (z=0 everywhere), same as they need to be for linear_extrude(). But build123d's own revolve() sweeps a Face rigidly around the literal 3D axis given -- per its own docs, "the most common use case is when the axis is in the same plane as the face to be revolved". Axis.Z is PERPENDICULAR to the XY plane, not in it, so hand build123d's revolve() a profile straight off circle()/polygon()/etc and every point just spins in place within its own z=0 plane -- a degenerate, zero-height "solid" (or an outright OCCT error), not the swept solid OpenSCAD's rotate_extrude() produces. The fix: rotate the profile 90 degrees about X first, carrying it from the XY plane onto the XZ plane (local (x,y) -> world (x,0,y)) -- NOW Axis.Z lies IN the profile's plane, exactly the case revolve() is built for, and local X/Y map onto

radius/height exactly like real OpenSCAD.

sweep_sec2path()

```
sweep_sec2path(section2d, path3d, mirror=0, orientation=0, closed_loop=0, is_frenet=None, transition='round')
```

Exposed to scripts | backend/brep.py:3926 | Used in: tap.py

Sweeps a 2D cross-section along a 3D path to build a solid -- the B-rep counterpart of `ocad.py`'s `sweep_sec2path()`. Calls `build123d`'s own native `sweep()` (OCCT's pipe-shell sweep) directly -- this went through two home-rolled reimplementations first (manually extruding+unioning each straight path segment, to work around `sweep()` apparently choking on sharp path corners) before landing back here, because the actual problem was a single missing argument, not `sweep()` itself: ``transition=``.

``transition`` controls how OCCT joins two straight (or curved) stretches of ``path3d`` at a corner: "round" (default here -- see below): inserts a rounded fillet-like

surface at the joint. The one that reliably produces valid geometry through a sharp corner, which is why it's the default here -- `build123d`'s own default (used if you pass `transition="transformed"` instead) is the "transformed" one below, which is fine for gentle bends but is a well-known way to get an invalid/self-intersecting solid through a genuinely sharp corner (a mitered/extended-face joint whose faces cross each other), which is almost certainly what a couple of earlier real bug reports against this function actually were.

"transformed": each cross-section is rigidly carried along the

path and the resulting faces are extended/trimmed to meet at a flat mitered joint. Sharper-looking corners on GENTLE bends; prone to invalid geometry on sharp ones (see above).

"right_corner": a hard-edged corner join, no rounding. If your path has any tight turns and the default doesn't look right, this is the first thing to try changing.

`section2d`: the cross-section to sweep -- either a `[[x,y], ...]` point

list (same shape `circle()/square()/turtle2d()` produce; turned into a `polygon()` automatically) or an already-built 2D shape (`circle()`, `square()`, `polygon()`, ...). Defined once in the XY plane; this function moves/orients a copy of it to the start of ``path3d`` for you -- you don't need to position it there yourself.

`path3d`: the path to sweep along -- either a `[[x,y,z], ...]` point

list (same shape `turtle3d()` produces; turned into a straight-line-segment Wire automatically, i.e. `build123d`'s Polyline -- 2D `[[x,y], ...]` points, like `arc()`'s or `sinewave()`'s output, are accepted too and padded to `z=0` automatically, so a path built entirely in the XY plane doesn't need `c23()`'d by hand first), an already-built `build123d` Edge/Wire (e.g. from your own `build123d` Spline/Helix/ThreePointArc, for a smoothly curved path instead of straight segments), or a filled 2D shape like `circle()/square()/polygon()` -- its boundary is used as the path automatically (e.g. `sweep_sec2path(circle(2), circle(20))` sweeps a small circle around a big one, a torus -- `circle()/etc` return a filled face, not just its outline, but a face isn't itself a valid sweep path, so the outline is pulled out for you here). Only reliable for a simple outline with a single boundary (like `circle()/square()/a plain polygon()`) -- for anything with holes or multiple boundaries, build your own explicit Wire/Edge and pass that instead.

mirror: 0 (default) or 1 -- flips the section across its own local

X axis before sweeping (same as `ocad.py`'s `mirror= flag`).

orientation: 0/1/2/3 -- rotates the section 0/90/180/270 degrees

about the path's own tangent direction before sweeping. A 2D cross-section has no inherent "up" once it's swept along an arbitrary 3D path -- there's no single correct answer for which way it should face, only a family of 4 that differ by which edge points which way -- so, same as `ocad.py`'s own `orientation= flag`, this is a "try 0, look at it, bump to 1/2/3 if it's facing the wrong way" knob, not something computed automatically.

closed_loop: 0 (default, open path) or 1 -- set 1 if ``path3d``'s

last point should connect back to its first (a closed-loop path, e.g. swept around a ring) rather than stopping there. Only meaningful when ``path3d`` is a point list; if you pass your own Edge/Wire, close it yourself before calling this.

is_frenet: passed straight through to `build123d`'s own

`sweep(is_frenet=...)` -- controls which frame carries ``section2d`` along the path: False uses OCCT's twist-minimizing "corrected" frame (no guaranteed relationship to "radially inward" -- it just avoids twisting, so a profile meant to point inward/outward can end up facing sideways instead); True uses a strict Frenet frame instead, whose principal normal always points straight at the path's own center of curvature -- exactly "radially inward" for a circle/helix, but prone to flipping abruptly at inflection points on a path that isn't uniformly curving one way (a wavy or S-shaped path, say).

Default (None) picks automatically: True if ``path3d`` is this file's own `helix()` (a real `build123d` Helix, whose curvature never changes sign, so there's no inflection point for Frenet to flip at -- and this is also what makes a thread/groove cut come out right, see below), False for every other path shape. This matches `ocad.py`'s own `sweep_sec2path()`, which has no separate Frenet/non-Frenet mode to begin with -- it always computes each cross-section's frame straight from the path's own local curvature, i.e. effectively always "Frenet". Pass `is_frenet` explicitly (True or False) to override this auto choice either way -- e.g. force True on a hand-built spiral path that isn't built via this file's `helix()` but still never changes curvature sign.

A tight helix (pitch smaller than ``section2d``'s own size along the path direction) with the wrong frame is the single most common cause of a "should be a thin thread but looks like a stack of thick overlapping discs/washers" result: if the profile's "point" isn't actually facing inward because of a wrong frame, sweeping it produces much WIDER coils than intended, and adjacent turns visibly overlap/merge. If that's what you're seeing and ``path3d`` ISN'T this file's own `helix()` (so the auto-True above didn't kick in), pass `is_frenet=True` by hand first before assuming the pitch/profile size themselves are the problem.

For an actual helical thread/groove CUT (used in a `difference()`), if the cut still comes out inverted (cutting outward instead of in) even with the right frame, try `mirror=1` or `orientation=2` on top -- ``is_frenet`` alone only fixes which way the frame points "in", not which side of the profile is the "point".

IMPORTANT -- this has previously been through several rounds of real bug fixes against a real build123d install (invalid geometry and a crash with orientation != 0; then, after switching to a hand-rolled segment-union approach, visibly disjointed segments; then switching to native sweep() plus transition="round"; then a filled 2D shape like circle() passed as path3d raising AttributeError, since a Face isn't itself a valid sweep path -- fixed by extracting its boundary wire automatically) but this exact version has NOT itself been re-verified against that real install (no network access in this environment to pip install build123d) -- only wiring-checked with a mock. Please report back what you see -- if "round" still doesn't look right for your geometry, try transition="right_corner" or "transformed" before assuming something else is wrong.

CONFIRMED real failure mode (real user report): a many-segment, NON-PLANAR path3d (e.g. one bent out of flat via axis_rot_1_pts() or similar) with the default transition="round" can raise `AttributeError: 'NoneType' object has no attribute 'NbNodes'` -- a real OCCT/OCF error deep in the "round" transition's per-corner fillet-surface construction, not a bug in this wrapper. Switching to transition="right_corner" (no fillet surfaces at all) fixed it. Same fragility class documented in p_line3d()'s own docstring (and examples/20_3d_knots.py's story) -- many segments + sharp/ varying turns + "round" is the pattern to watch for; try "right_corner" or "transformed" first before digging further.

swp_c()

```
swp_c(bead2, tol=1e-06)
```

Not yet exposed to scripts | backend/brep.py:11823 | Used in: —

Direct B-rep port of ocad.py's own swp_c() -- builds the exact same triangulated side-wall mesh (straight quads between consecutive rows, each split into 2 triangles the same way, every row's own point list wrapping around as a closed ring in the column direction) as a real sewn B-rep Shell/Face, instead of an OpenSCAD polyhedron() string.

`bead2`: a list of R rows ("stations"), each row a same-length list of [x,y,z] points -- one CLOSED ring per row (point 0 connects back to the last point automatically; don't repeat the first point at the end of a row). Consecutive rows are connected by a straight- faceted band; the ROW direction itself is left OPEN (station 0 and station R-1 are NOT connected) -- same as ocad's swp_c(), which needs bead2[0] repeated as the LAST row yourself if you want the row direction to close into a loop too (works here exactly the same way -- the final band then connects the true last row back to row 0, same as any other band, no special-casing needed).

No caps are added either way, matching swp_c()'s own actual behavior (it computes cap face data internally but never returns it -- see its own source). Close the ends yourself (cap_wire() + solid_from_faces()), or by union()-ing against adjoining geometry that already covers them -- e.g. slider_housing()'s own wedge-grid wall is exactly this technique, just built with surface_from_points_grid_closed() instead for a smoother per-rail spline fit rather than swp_c()'s own raw straight facets; reach for THIS function instead whenever you specifically want that raw faceted triangulation (matching an existing ocad script's swp_c() output facet-for-facet), not a smoothed re-fit of it:

```
rows = [[l1[i], l4[i], l3[i], l2[i], l1[i]] for i in range(n)]
rows.append(rows[0])          # close the row direction too
wall = swp_c(rows)             # a closed Shell, no open ends
housing = bd.Solid(wall)
```

Returns a single Face if `bead2` has exactly 2 rows (one straight- ruled band, 2*m triangles sewn as one Face), otherwise a Shell of every band's triangles sewn together. Raises ValueError if `bead2` has fewer than 2 rows, or rows of inconsistent length.

Surfaces (17)

face_islands()

```
face_islands(faces)
```

Exposed to scripts | backend/brep.py:6888 | Used in: —

Splits ``faces`` into its separate topologically-CONNECTED groups -- "how many disjoint pieces is this face list actually made of" as a real function. Two faces are in the same group if you can walk from one to the other over a chain of shared edges (exactly the same shared-edge adjacency `faces_within_boundary()` floods over, but with no boundary walls here -- this just reports the natural islands the faces already fall into on their own).

Directly answers "why won't `shell_from_faces()/solid_from_faces()` stitch these together": both require ONE single connected Shell, and a normal-direction/region selection like `faces_facing()` has NO guarantee of returning one connected patch -- on a curved or organic shape, faces sharing a similar normal direction can easily fall into several separate islands (e.g. a shape's main dome PLUS a few disconnected slivers elsewhere that happen to face the same way). If `shell_from_faces()` is failing, run this first:

```
groups = face_islands(f1)
[len(g) for g in groups]      # sizes of each disconnected piece
show(*groups[0], 'red')       # just the largest connected piece

f1_clean = groups[0]           # keep only the largest island
shell = shell_from_faces(f1_clean)
```

Returns a list of lists of Faces, largest group first -- a single connected input comes back as one group containing everything.

faces_facing()

```
faces_facing(faces, direction, angle_tol=90.0)
```

Exposed to scripts | backend/brep.py:6965 | Used in: —

Selects every face whose own outward normal points TOWARD ``direction`` -- "select all faces visible from this direction" as a real function:

```
top_faces = faces_facing(sol.faces(), [0, 0, 1])
show(*top_faces, 'red')
```

Tests each face's `normal_at()` (sampled at its own center()) against ``direction``: a face qualifies if the angle between its own normal and ``direction`` is $\leq$ ``angle_tol`` degrees. The default (90) keeps anything that faces at least partly TOWARD the direction rather than strictly away from it -- narrow it (e.g. 30-45) for only faces pointing MORE directly toward it, e.g. a genuinely flat "top" versus a steeply-angled side that technically still has a sliver of upward component.

This is a NORMAL-DIRECTION test, not a true occlusion-aware visibility test -- it doesn't ray-cast to check whether some OTHER part of the same shape sits in front and actually blocks a given face from that viewpoint (a real concave pocket can have a face that technically points toward ``direction`` but is still hidden behind a wall). Fine for convex-ish or mildly concave shapes (this project's own mouse-body dome included); for a shape with deep pockets/cavities where that distinction actually matters, filter the result further by hand. Real ray-cast visibility is genuinely expensive PER-FACE work -- same performance class as `seam_edges()`'s own `.distance_to()` cost this session -- deliberately not attempted here.

``direction`` doesn't need to be a unit vector -- only its own direction matters, not its length.

Returns a list of Faces (a subset of ``faces``).

faces_in_box()

```
faces_in_box(faces, box, mode='inside', tolerance=1e-06)
```

Exposed to scripts | backend/brep.py:6707 | Used in: —

`faces_in_box()`'s own counterpart for face selectors -- the exact same probe-box picking workflow as `edges_in_box()`, but for `shape.faces()` instead of `shape.edges()`. Useful for things like `offset()`'s ``openings=`` argument or picking which faces to `fillet()`/`chamfer()` by drawing a box around/through them instead of reading `label_faces()`'s numbers.

``box``: any B-rep shape (usually `box([dx, dy, dz])`, `translate()`'d/ `rotate()`'d into place) -- only its own `bounding_box()` is used, its actual geometry never matters.

``mode``: "inside" -- (default) only faces FULLY CONTAINED in the probe

box's extent -- draw a box that ENCLOSES just the faces you want and nothing else.

"crossing" -- faces whose own bounding box overlaps the probe box

AT ALL, including ones only partly inside it -- draw a (usually thin) box that CUTS THROUGH the faces you want, even if it doesn't fully contain them.

```
a = cube([20, 20, 20])
show(a, alpha=0.2)
probe = translate([-10, -10, 18], box([20, 20, 4]))
show(probe, 'red', alpha=0.15) # eyeball/adjust probe first
top_face = faces_in_box(a.faces(), probe)
hollow = offset(a, -2, openings=top_face)
```

Pass the SAME list you'd index into for `offset()`/`fillet()`/`chamfer()` -- almost always `shape.faces()` itself, unfiltered -- and the returned list is already the plain list of Face objects those functions expect, no further indexing needed. Combine with the other `filter_by()`/`sort_by()`/`label_faces()` techniques as needed -- e.g. ``[f for f in faces_in_box(...) if f.area > 50]`` works fine on top of this.

faces within boundary()

```
faces_within_boundary(faces, boundary_edges, seed_point=None)
```

Exposed to scripts | backend/brep.py:6780 | Used in: —

Selects every face reachable from a starting face WITHOUT crossing any of ``boundary_edges`` -- "select the region enclosed by this edge loop" as a real function, the face counterpart to `seam_edges()`'s own "find the boundary" direction: once you already have a boundary (`seam_edges()`, a `fillet()`'s own edge list, edges picked by hand through this app's Pick Edges panel, or any other list of real Edges), this fills in everything INSIDE it.

```
seam = seam_edges(sol, s1, b_2)
patch = faces_within_boundary(sol.faces(), seam, seed_point=[0, 0, 40])
show(*patch, 'red')
```

``boundary_edges`` doesn't need to be one single closed loop -- works just as well with several separate closed loops at once (e.g. an inner + outer ring bounding an annular band -- pass both loops' edges together in one list), or an OPEN chain that doesn't fully close on its own. Every face's own boundary edge is either (a) one of ``boundary_edges`` (never crossed), (b) shared with exactly one other face and NOT in ``boundary_edges`` (crossed freely -- this is how the flood fill spreads), or (c) owned by only that one face at all (a genuinely free/open edge -- e.g. a Shell's own outer rim -- with no "other side" to flood into regardless). An open boundary chain still works as long as it, together with the shape's own real free edges, fully encloses the region you want -- the flood fill simply has nowhere else to go once it reaches either kind.

``seed_point``: [x, y, z] on any ONE face inside the region you want -- resolves which side of the boundary is "inside" without guessing (the boundary generally splits ``faces`` into two pieces, same as a closed curve splits a plane). If omitted, returns whichever resulting piece has FEWER faces -- a reasonable default (the enclosed patch is usually the minority of the shape), but pass `seed_point` explicitly whenever that guess might be wrong, e.g. picking a large region bounded by a small hole.

Returns a list of Faces (a subset of ``faces``).

fill face holes()

```
fill_face_holes(faces, all_faces, max_size=None)
```

Exposed to scripts | backend/brep.py:7024 | Used in: —

Adds back any small "hole" in a face selection -- a connected group of faces that got EXCLUDED from ``faces`` but is completely enclosed by faces that ARE in ``faces`` (never touching ``all_faces``' own real outer/free edge), so it's really just an island punched straight through an otherwise simply-connected patch.

Directly fixes the failure mode `close_faces_to_plane()` (and anything else expecting one clean boundary loop) hits on a face- NORMAL selection like `faces_facing()`'s own output, confirmed directly in this app: on a fine, many-facet mesh,

individual flat facets can flip in and out of a normal-angle cutoff right at the threshold -- especially along a carved groove/fillet, where the true surface's normal keeps crossing back and forth near the cutoff as it winds around -- punching dozens of tiny single-facet holes through an otherwise-connected patch. `face_islands()` still reports the patch as ONE connected piece (every face IS reachable from every other), but each of those tiny holes is its OWN separate boundary loop, and `wire_from_edges()` can't (and shouldn't) silently guess how to chain several genuinely separate closed loops into one.

```
f1 = faces_facing(sol.faces(), [0, 0, 1]) # riddled with holes
f1_filled = fill_face_holes(f1, sol.faces())
f1_clean = face_islands(f1_filled)[0]
shell = shell_from_faces(f1_clean)
slider = close_faces_to_plane(shell, [0, 0, 80], [0, 0, 1])
```

This does NOT loosen `faces_facing()`'s own `angle_tol` -- it keeps the selection criterion exactly as given and only patches over the facet-level noise that criterion leaves behind, so this is the right fix when the selection angle itself is correct/required and just needs cleaning up, not widening.

``all_faces``: the full pool ``faces`` was drawn from (e.g. ``sol.faces()`` -- same shape ``faces``' own items came from). Needed to tell a genuinely enclosed hole apart from an excluded region that's simply part of the real outside (e.g. `faces_facing()`'s own excluded "pointing away" hemisphere) -- an island only gets filled if NONE of its own edges are free (single-owner) across the WHOLE ``all_faces`` pool, meaning it never actually reaches the outside at all.

``max_size``, if given, only fills a hole with AT MOST that many faces -- leaves a bigger enclosed region alone (more likely a real, intentional window/cavity through the patch than facet noise). Default (None) fills every enclosed hole regardless of size.

``faces``/``all_faces`` each accept either a plain list of Faces or a single Shape (Shell/Solid/Compound) with its own `faces()`.

Returns a new list of Faces: everything in ``faces``, plus every enclosed hole's own faces added back in.

mirror_surface()

```
mirror_surface(shape, n1, loc=(0, 0, 0))
```

Exposed to scripts | backend/brep.py:5556 | Used in: —

Mirrors a real ``shape`` (solid/face/wire/edge) across the plane through ``loc`` with normal ``n1`` -- ocad.py's own `mirror_surface()`, ported. The original mirrored a mesh "surface" (a grid of point-list rows) row-by-row via `mirror_line()`; here it's just build123d's own native `mirror()` applied straight to a real shape, with an arbitrary ``loc`` offset -- the general form of this app's own `mirror(v, shape)` above, which always mirrors through the origin. For mirroring a raw 2D/3D point list instead of a real shape, see `mirror_line()/mirror_point()`.

```
s1 = translate([0, -30, 0], sphere(10))
s2 = mirror_surface(s1, [1, 1, 0], [0, -30, 0])
show(s1)
show(color(s2, "cyan"))
```

offset_face()

```
offset_face(face, amount)
```

Exposed to scripts | backend/brep.py:5818 | Used in: —

Moves a Face (or Shell) ``amount`` along its own surface normal -- build123d's own native `Face.offset()/Shell.offset()` INSTANCE method. This is deliberately a DIFFERENT function from `offset()` above, because they do genuinely different things for a Face: `offset()` (the top-level function above) offsets the face's OUTER WIRE sideways, WITHIN its own plane (bigger/smaller outline, same plane) -- `offset_face()` instead pushes the whole face OFF its own plane, along its normal (a plain flat plate pushed straight up).

EXACT for a planar Face (`plane()`'s own output, say): every point genuinely moves ``amount`` along the same constant normal, so this is a true parallel copy of the plane.

APPROXIMATE for a curved Face (a cylinder's lateral surface, a fitted NURBS patch): build123d implements this as one RIGID TRANSLATION by (the face's own reference-point normal) * amount, not a true "every point grows along ITS OWN local normal" offset surface (that operation isn't exposed as a simple one-line call in build123d). Good enough as a nearby stand-in surface for finding a shifted intersection curve (see convert_3lines2fillet()'s own docstring for the full recipe this is built for) -- not a substitute for a real, precise offset surface.

```
line_b = sample_curve(intersection(offset_face(a, 1), b), 40)
line_a = sample_curve(intersection(a, offset_face(b, 1)), 40)
```

project_curve_on_face()

```
project_curve_on_face(curve, target, direction, closest=True)
```

Exposed to scripts | backend/brep.py:9579 | Used in: —

Projects `curve` (a real build123d Edge/Wire, e.g. from interp_spline()/turtle3d()+a helper, or `shape.edges()[i]`) onto `target` (a Face/Solid -- e.g. `some\_solid.faces()[0]`) along `direction` ([x,y,z]) -- exact B-rep projection, computed by OCCT (build123d's own Edge/Wire.project()), not a nearest-point search over a triangulated approximation. For projecting a raw point list or point grid instead of an actual curve, see psos() below.

`closest=True` (default) returns the single nearest result if the curve's shadow lands on the target more than once; `closest=False` returns every hit (a list).

project_face_on_shape()

```
project_face_on_shape(face, target, direction)
```

Exposed to scripts | backend/brep.py:9597 | Used in: —

Projects a whole Face (or every face of a Solid/Shape) onto `target` (a Face/Solid) along `direction` ([x,y,z]) -- extends project_curve_on_face()'s same idea (exact B-rep projection, computed by OCCT, no sampling) up one dimension, from a single curve to an entire surface. Uses build123d's own native Face.project_to_shape(): `face` is swept along `direction` far enough to clear `target` entirely, then intersected with it -- unlike psos()+surface_from_points_grid() (project a raw point grid, then fit a NURBS surface back through the results), this returns the REAL projected Face(s)/Shell(s) directly, exact, with no sampling or fitting step at all.

```
a = sphere(10)
b = plane([0, 0, 1], [50, 50])
c = project_face_on_shape(a, b, [0, 0, 1])
show(a, alpha=0.2)
show(color(c, "cyan"))
```

`face`: a real Face (circle()/polygon())/a shape's own

`.faces()[i]`, OR any Shape with `.faces()` (e.g. a Solid like sphere()/box()'s output) -- every one of its faces is projected in turn and all results combined into one list.

`target`: the real Face/Solid to project onto.

Returns a list of the projected Face(s)/Shell(s) -- there can be more than one per source face (e.g. the "front" and "back" of a closed target Solid both catch a projection); faces that would land "behind" the projection direction are never returned.

surface_cap_solid()

```
surface_cap_solid(shape, direction, side='top', plane_point=None, n_lines=40,
points_per_line=20, angle=0.0, ruled=True, tol=1e-06)
```

Exposed to scripts | backend/brep.py:12044 | Used in: —

Builds one smooth solid for the volume between a flat reference plane and `shape`'s own REAL curved surface -- the fast, hole-free alternative to mesh_cap_solid() for when THAT function's own documented cost (build+fuse one solid PER TRIANGLE) turns out to matter in practice (CONFIRMED real numbers, this session: ~430s, and the result still came back with a couple of faces missing from its own on-screen preview).

`mesh_cap_solid()`'s slowness and the missing faces share one root cause: it works off to `_mesh()`'s RE-TESELLATED copy of ``shape``, so it pays for potentially thousands of individual triangle-sized solids, each with its own sewing/fuse cost and its own chance to trip to `_mesh()`'s documented per-face tessellation fragility (see that function's own docstring). This function never tessellates ``shape`` at all -- it projects a modest number of straight sample LINES onto ``shape``'s own exact analytic/NURBS surface via `psos()` (build123d's real `Face.find_intersection_points()`, the same exact projection `project_curve_on_face()/project_face_on_shape()` use), and lofts directly through the results with `solid_from_closed_rings()` (already this app's own proven "many parallel closed rings -> one smooth solid" tool -- see `organic_fillet()`'s own ribbon technique for the same idea used elsewhere).

HOW: the flat outline (`shape_outline_raster()`'s own "outer" loop, reused directly -- not recomputed) is filled with ``n_lines`` evenly spaced scan lines (like hatching, at ``angle`` degrees), each line clipped to wherever it's actually inside the outline (same ray-crossing rule `point_in_polygon()` already uses, generalized to return every crossing instead of just True/False -- a concave footprint correctly produces more than one line SEGMENT at a given scan position, each handled as its own separate ring, not merged; a shape with real HOLES in its outline -- `shape_outline_raster()`'s own "holes" list, e.g. a washer or a plate with a bore -- is handled the same way, by the same even-odd rule, so scan points are never generated inside a hole where ``shape`` has no material at all). Each segment is resampled to ``points_per_line`` points and projected in ONE `psos()` call onto ``shape``'s real surface -- since every sample point starts out ON the flat plane INSIDE the outline (so genuinely inside ``shape``'s own volume at that height), the nearest hit going further along ``direction`` (or its opposite, for ``side`="bottom"`) is exactly the skin on that side, found exactly, not approximated off a mesh.

Each flat line and its own projected twin are joined end-to-end into one closed ring (out along the flat line, up to the surface, back along the curved line, down to close) -- ``n_lines`` (or more, on a concave scan row) of these, in scan order, go straight into `solid_from_closed_rings()` as its ``rows``, which lofts them into one watertight solid in a SINGLE OCCT call, not thousands of small ones.

```
top_solid = surface_cap_solid(sol, [0, 0, 1], side="top", plane_point=[0, 0, 40])
trimmed = intersection(sol1, top_solid)
show(trimmed, 'red')
```

``side``: "top" (default) or "bottom" -- which of ``shape``'s two skins (as seen along ``direction``) to build the volume up/down to.

``plane_point``: same convention as `shape_outline_raster()`'s own -- must be a height that actually passes THROUGH ``shape`` (so the scan points genuinely start out inside it); defaults to ``shape``'s own bounding-box center. Also reused as `shape_outline_raster()`'s own ``plane_point`` internally, so the flat outline and the scan plane are always the exact same plane.

``n_lines`/`points_per_line``: resolution knobs -- ``n_lines`` scan positions across the footprint, ``points_per_line`` samples along each one. Both directly drive how closely the loft follows ``shape``'s real curvature; raise them for a tighter fit, at some cost (still one `solid_from_closed_rings()` call either way, so nowhere near as steep as `mesh_cap_solid()`'s own per-triangle cost curve).

``angle``: hatch angle in degrees (0 = scan lines run along the plane's own local x/u axis). Try a different angle if the footprint's own shape happens to align badly with the default (e.g. a long, thin diagonal outline scanned only along its short axis needlessly produces many tiny segments).

``ruled`/`tol``: passed straight through to `solid_from_closed_rings()` -- ``ruled=True`` (default here, unlike that function's own default) to match this app's own established practice of preferring it for genuinely organic/non-primitive ring stacks (see that function's docstring); try ``ruled=False`` afterward for a smoother result once this is confirmed working.

Returns one Solid.

Raises `ValueError` if fewer than 2 usable scan-line segments were found at all (the outline is smaller than ``n_lines`/`points_per_line`` can usefully resolve, or ``plane_point`` doesn't actually pass through ``shape``), or whatever `psos()/solid_from_closed_rings()` themselves raise.

surface_from_2_waves()

```
surface_from_2_waves(p0, p1, amplitude=1)
```

A wavy real B-rep surface combining two wave-like point lists `p0`/`p1` (each meant to run along its OWN axis, perpendicular to the other -- see the example below) -- ocad.py's own `surface_from_2_waves()`, ported as a real fitted NURBS Face (`surface_from_points_grid()` above) instead of a triangle-mesh grid.

For every combination of a point `i` from `p0` (the grid's "row" direction) and a point `j` from `p1` (the "column" direction), builds a grid point at `[i.x, j.y, i . j]` (i and j's own 3D dot product -- so the two waves' heights multiply together, the classic combined-wave "basket weave"/ripple look), then rescales the whole grid's height so its max is exactly `amplitude`.

`p0`/`p1` are typically `sinewave()`/`cosinewave()` output laid out directly in their own plane (NOT rotated afterward with `rot()` -- that only works on real Shapes, not point lists -- build the wave straight into the plane you want instead, same trick `surface_line_vector()`'s own docstring uses):

```
l1 = [[x, 0, y] for x, y in sinewave(50, 3, 2, 50)]      # XZ plane
l2 = [[0, x, y] for x, y in cosinewave(50, 3, 2, 50)]    # YZ plane
s1 = surface_from_2_waves(l1, l2, 2)
show(s1)
```

A ripple with a small `amplitude` relative to `p0`/`p1`'s own length (e.g. `amplitude=2` over a `length=100` wave -- only a ~2% grade) will genuinely look almost flat at normal zoom, in any renderer -- that's not a bug, just perspective. Use a bigger `amplitude:length` ratio to see it clearly, or check `s1.bounding_box().size.Z` (should be close to 2*amplitude, since the surface genuinely swings +amplitude to -amplitude) to confirm the ripple is really there before assuming something's wrong.`

surface from closed rings()

```
surface_from_closed_rings(rows, ruled=False, tol=1e-06)
```

A genuinely SMOOTH, minimal-patch closed-loop B-rep surface, built from an ORDERED list of closed cross-section rings -- e.g. exactly the ring-per-sweep-station data a manual point-grid pipeline naturally produces (`o.solid_from_2surfaces()`'s own output, or anything shaped like it), in its ORIGINAL [station][ring point] layout -- no `o.cpo()` transpose needed, unlike `surface_from_points_grid_closed()` (see below for why).

WHY THIS EXISTS ALONGSIDE `surface_from_points_grid_closed()`: that function is a real, working closed-loop surface, but it's built from N-1 separate RULED strip Faces (one per pair of adjacent rail curves) sewn into a Shell -- correct, watertight topology, but genuinely a multi-facet Shell, not one smooth surface, and its facet/edge COUNT scales directly with however many stations you feed it (a few hundred stations really does mean a few hundred real B-rep edges -- not a display artifact; see `fillet()`/`thicken()`'s own performance notes elsewhere in this file, and reducing the input resolution only ever reduces the facet count, never removes the faceting itself).

This function instead calls build123d's own loft engine directly on periodic section wires -- OCCT's `BRepOffsetAPI_ThruSections`, `ruled=False` == its "smoothed" mode, the exact same engine FreeCAD's own Part Design Loft feature uses with "Ruled" unchecked. Given ordered PERIODIC closed wires, it blends them into ONE genuinely smooth, typically SINGLE-Face surface (a real multi-span NURBS surface -- C2-continuous across every station internally, but one topological Face, not N sewn ones), with no per-station facet edges left behind at all.

`rows`: ordered list of R closed rings (station 0 first, station R-1 last), each ring a same-length list of `[x,y,z]` (or `[x,y]`, `z=0`) points -- same point-grid convention as `surface_from_points_grid()`/`surface_from_points_grid_closed()`, just NOT transposed: every row here already IS one ring, so this closes the ring axis directly rather than needing the column-becomes-row trick `surface_from_points_grid_closed()` relies on.

`ruled=False` (default) is the smoothed/single-surface mode this function exists for. `ruled=True` instead facets straight between adjacent rings (visually similar to `surface_from_points_grid_closed()`, just built via `ThruSections` instead of manual rail-ruling) -- mostly useful as a diagnostic: if a smoothed fit fails but `ruled=True` succeeds, the rings themselves are fine and it's specifically the smoothing that's struggling (e.g. two adjacent rings differing wildly in shape).

Returns a Shell open at its two ends (the first/last ring's own boundary edge, same as `surface_from_points_grid_closed()`) -- `cap_wire()` + `solid_from_faces()` close it into a solid exactly the same way:

```
wall = surface_from_closed_rings(rows)
e_start, e_end = wall.edges()[0], wall.edges()[-1] # probe-pick
                                                    # for real

cap_a, cap_b = cap_wire([e_start]), cap_wire([e_end])
solid = solid_from_faces(wall, cap_a, cap_b)
```

Raises `RuntimeError` if OCCT's `ThruSections` can't build a valid surface through the given rings at all (self-intersecting rings, fewer than 2 rings, or rings that vary too wildly station to station for any single smooth fit to follow) -- try ``ruled=True`` first as a diagnostic (see above), or loosen ``tol``.

surface from points grid()

```
surface_from_points_grid(points, tol=0.01, min_deg=1, max_deg=3)
```

Exposed to scripts | [backend/brep.py:4897](#) | Used in: —

Fits a real B-rep surface (a Face, generally non-planar) through a 2D grid of 3D points -- `build123d`'s own native `Face.make_surface_from_array_of_points()` (GeomAPI `PointsToBSplineSurface` under the hood: a genuine NURBS surface approximation through the whole grid), instead of a triangle mesh. ``points`` is a list of rows, each row a same-length list of `[x,y,z]` (or `[x,y]`, `z=0`) points -- a rectangular `V x U` grid.

The general-purpose building block behind `surface_from_2_waves()` below -- reach for this one directly whenever you already have your own 2D point grid (your own height-field/parametric math, a grid sampled off some other surface, etc) and just want it as one real curved Face rather than assembling it from wave curves.

``tol`/`min_deg`/`max_deg`` control the fitting accuracy/degree -- same meaning as `build123d`'s own parameters, defaults are fine for most cases. Raises `ValueError` ("B-spline approximation failed") if the grid is too irregular/self-overlapping for a single surface to fit through at the given tolerance -- try loosening ``tol`` first.

surface from points grid closed()

```
surface_from_points_grid_closed(points)
```

Exposed to scripts | [backend/brep.py:4926](#) | Used in: —

Like `surface_from_points_grid()` above, but for a point grid that should wrap into ONE continuous, seamless loop along its ROW direction (the direction `ocad.py`'s `path_extrude_open()`/ `path_extrude_closed()` sweep along) instead of stopping at an open start/end -- e.g. a ribbon swept all the way around a closed path (`o.arc(10, 0, 360, s=49)`, a full circle) that should genuinely meet itself with no seam, not just end up visually touching.

WHY `surface_from_points_grid()` ITSELF CAN'T DO THIS: it's a thin wrapper around `build123d`'s `Face.make_surface_from_array_of_points()` (OCCT's `GeomAPI_PointsToBSplineSurface`), which only ever fits an OPEN (non-periodic) B-spline surface -- there's no "closed" mode to ask for. The usual-looking workaround -- appending a duplicate of the first row onto the end of ``points``, which is exactly what `o.path_extrude_closed()` does differently from `o.path_extrude_open()` -- LOOKS closed once rendered, but isn't: you get two separate, exactly-coincident rows of geometry sitting flush against each other at the seam instead of one shared boundary. That's precisely the "flush/coincident contact" case OCCT's booleans and fillets are documented elsewhere in this file to be fragile on (see `union()`'s own docstring) -- which is why `fillet()` can behave differently, or fail outright, right at that seam even once edge indices are re-probed for the new topology.

THE FIX: don't fit one big surface through a duplicated grid at all. Treat each COLUMN of ``points`` (station 0 across every row, station 1 across every row, ...) as its own rail curve running along the path, close EACH rail into a genuine single closed Wire (looping back to its own first point -- ONE piece of geometry, not two coincident ones sitting on top of each other), then rule a real B-rep surface directly between each pair of ADJACENT rails (`build123d`'s own native `Face.make_surface_from_curves()` -- the same call `surface_line_vector()` above already uses successfully) and sew the resulting strips into one Shell via the same OCCT-level sewing shell `from_faces()/solid_from_faces()` use. For the common case of a flat 2-point cross-section (a ribbon -- `o.path_extrude_open(d, c)` with `d=[[-2,0],[2,0]]`) that's just ONE

ruled strip between the 2 rails, genuinely periodic with no seam artifact anywhere, and this returns that single Face directly rather than a 1-face Shell.

Each rail tries a smooth periodic interpolating spline first (build123d's `Edge.make_spline(pts, periodic=True)` -- matches `surface_from_points_grid()`'s own NURBS-fit smoothness along this direction), falling back automatically to a plain closed Polyline (straight facets between path samples -- still one single continuous Wire, never two coincident ones) if this build123d version's `make_spline()` doesn't accept `periodic=True`, or the fit fails for some other reason (e.g. a self-tangent or too-short column). Either way the loop closure itself is real, not visual.

``points``: same shape/convention as `surface_from_points_grid()` takes -- rows of same-length `[x,y]` or `[x,y,z]` points -- and the SAME convention `o.path_extrude_open()` returns. Feed it `o.path_extrude_open()`'s output directly, NOT `o.path_extrude_closed()`'s (and don't duplicate the first row onto the end yourself either) -- this function closes the loop on its own, from an ordinary open grid:

```
a = o.rot('x90', o.sinewave(50, 3, 2, 50))
b = o.c23(o.arc(10, 0, 360, s=49))
c = o.extrude_wave2path(a, b)
d = [[-2, 0], [2, 0]]
sr1 = o.path_extrude_open(d, c)          # NOT path_extrude_closed
wall = surface_from_points_grid_closed(sr1)
sr1 = thicken(wall, -1)
show(sr1)
```

Returns a real Face (2-column grid) or Shell (3+ columns) -- feed it straight to `thicken()` same as a plain `surface_from_points_grid()` Face; `thicken()` already accepts either (see `shell_from_faces()`'s own docstring/example). Since the loop is now genuinely closed all the way around, `thicken()` has no start/end cap edges left to wall off at all -- the only boundary left is the rail curves at each end of the grid's own COLUMN range (e.g. the first and last rail, for a grid wider than 2 columns), which `thicken()` walls same as always.

Raises `ValueError` if ``points`` has fewer than 2 rows, fewer than 2 columns, or rows of inconsistent length (need a genuine rectangular grid, same as `surface_from_points_grid()`), or whatever build123d/ OCCT itself raises if a rail is too irregular for either the spline or the polyline fallback to rule a valid surface from.

surface_normals()

```
surface_normals(grid, direction=1)
```

Not yet exposed to scripts | [backend/brep.py:9837](#) | Used in: —

B-rep equivalent of `ocad.py`'s own `surface_normals()` -- for each point in the rectangular point grid ``grid``, averages the face normal of every triangle (of the SAME open-grid triangulation `_surface_to_triangles()` itself builds) touching that point, then normalizes. ``direction``: 1 (default) or -1, flips which way every normal points.

``grid`` must be a genuine 2D grid (rows of same-length point rows) -- a flat point list has no neighbors to compute a normal from.

Returns a flat list of `[x,y,z]` unit vectors, one per point, in the same row-major order as ``grid`` flattened.

surface_solid_closed_intersection()

```
surface_solid_closed_intersection(surface1, solid1)
```

Exposed to scripts | [backend/brep.py:2393](#) | Used in: —

Same as `surface_solid_open_intersection()` above (see its own docstring for the full explanation of why both `ocad.py` names collapse to the same real-B-rep call here) -- `ocad.py`'s own `surface_solid_closed_intersection()`, ported.

surface_solid_open_intersection()

```
surface_solid_open_intersection(surface1, solid_open, ear=0)
```

Exposed to scripts | [backend/brep.py:2362](#) | Used in: —

The raw intersection LINE SEGMENTS between ``surface1`` and ``solid_open`` -- real B-rep, just two `_solids_intersection()` under a different name (see its own docstring for the full picture, including ``contiguous_chains()`` to stitch the result into continuous lines).

ocad.py's own `surface_solid_open_intersection()/ surface_solid_closed_intersection()` are really the SAME operation under two names, differing only in which internal mesh- triangulation helper each used for ``solid1`/`solid_open`` (`triangulate_solid_closed()` vs `triangulate_solid_openx()/ triangulate_solid_open()`) -- a distinction about how a MESH happens to be represented that has no equivalent for a real build123d Shape (always a well-defined solid either way), so both names are ported here as the exact same call. ``ear`` (ocad.py's own `triangulate_solid_open()` vs `triangulate_solid_openx()` choice) is accepted for call-signature compatibility but has no effect here.

This is what a script written for ocad.py's own `plane()+surface_solid_open_intersection()` needs to keep working once ``plane`` resolves to this file's own `plane()/ plane_from_equation()` (a real Face, not a mesh grid) -- ocad.py's version expects to triangulate its surface argument itself, which fails outright on a real Shape (``TypeError: object of type 'Face' has no len()``, from deep inside its own `triangulate_surface()`).

Fillets & Chamfers (8)

chamfer()

```
chamfer(shape, edges, length)
```

Exposed to scripts | backend/brep.py:6485 | Used in: —

Bevels ``edges`` of ``shape`` by ``length`` -- a real, exact B-rep chamfer, same idea as `fillet()` above but a flat angled cut instead of a rounded one. See `fillet()`'s docstring for how to select ``edges`` (e.g. ``shape.edges()`` for all of them) -- ``edges`` can also be a single bare Edge, same convenience as `fillet()`.

convert_3lines2fillet()

```
convert_3lines2fillet(line_a, line_b, spine, s=20, closed_loop=0)
```

Exposed to scripts | backend/brep.py:4373 | Used in: hook.py

Builds a fillet surface (as a 2D point grid) by blending 3 matching-length point lists station-by-station -- ocad.py's own `convert_3lines2fillet()`, ported: ocad.py called these positional args `l2`, `l3`, `l1` respectively (matching that exact positional order if you're porting an old script), describing the order as "starting line, ending line, anchor line" -- ``spine`` here is that 3rd "anchor" argument.

The manual, explicit-geometry way to build a fillet when `fillet()/max_fillet_radius()` genuinely can't find one -- a deep or sharp cut, a seam edge, edges too close together (see `fillet()`'s own docstring for the real failure modes this is the fallback for).

``spine`` is the actual sharp edge you're rounding off -- e.g. a real intersection curve between two surfaces ``a`/`b``, sampled into a point list. ``line_a`/`line_b`` are two curves tracing where the fillet should blend INTO each of the two original surfaces -- pulled a little way off ``spine``, onto each surface respectively. All 3 lists must be the SAME length, in the SAME station order -- station `i` on ``spine`` must correspond to station `i` on ``line_a`/`line_b``.

The general recipe for getting all 3 from two real surfaces ``a`/`b``, using ONLY real B-rep booleans (no surface-projection math needed): nudge one surface off its own plane with `offset_face()`, then intersect it with the OTHER (un-nudged) surface -- the resulting curve lies exactly on the un-nudged surface, shifted away from the true spine by roughly the nudge amount:

```
blend = 2                                # how far the fillet reaches
n = 40                                    # station count
spine = sample_curve(intersection(a, b), n)
line_b = sample_curve(intersection(offset_face(a, blend), b), n)
line_a = sample_curve(intersection(a, offset_face(b, blend)), n)
grid = convert_3lines2fillet(line_a, line_b, spine, s=15)
patch = surface_from_points_grid(grid)
```

```
show(patch)
```

(`project_curve_on_face()`/`psos()`) are the other real B-rep tools for getting a curve to lie on a specific surface, if you already have one from elsewhere instead of deriving it via `offset_face()`+`intersection()` above.)

At every station, builds a small `bezier()` arc through `[line_a[i], spine[i], line_b[i]]` (`s`` points, pulled toward the spine point in the middle, same as any `bezier()` control point). The result is a 2D point grid (one row per station) -- feed it straight into `surface_from_points_grid()` for the real B-rep fillet Face:

```
grid = convert_3lines2fillet(line_a, line_b, spine, s=15)
patch = surface_from_points_grid(grid)
show(patch)
```

DIFFERS from `ocad.py`'s own version in one deliberate way: the original appended the spine point again at the end of every station (``+[p1]``), closing each cross-section into a wedge shape for its own mesh-based `swp_c()` (a closed-section sweep -- see `swp_c()`'s own real behavior: it auto-closes the LAST point of every row back to the FIRST, so appending the spine point again at the end there actually built a closed, pinched-at-the-spine wedge profile per station, not just a cosmetic extra point). This function returns the OPEN row instead (no reclosing point) since it's meant for `surface_from_points_grid()` below, which fits one smooth surface through the whole grid directly and doesn't need (or want) a closed per-row loop. If you want the actual closed- wedge-per-station behavior for a REAL solid via `loft()` -- the `build123d` equivalent of `swp_c()` -- see `loft_fillet()` right below instead, which reconstructs that exact reclosing.

``closed_loop=1`` also repeats the FIRST station as one more row at the end -- for a fillet that wraps all the way around a closed spine (a full loop, not two open ends), same `closed_loop` convention as `turtle3d()/p_line3d()/sweep_sec2path()`.

corner_n_radius_list()

```
corner_n_radius_list(p0, r_l, n=10)
```

Exposed to scripts | `backend/brep.py:790` | Used in: —

Rounds each corner of the ABSOLUTE 2D point list ``p0`` (a closed loop) using the matching radius from the parallel list ``r_l`` (same length as `p0`; 0 means "leave this vertex sharp") -- `cr2dt()`'s sibling for when you already have a fixed set of absolute corner points (e.g. a rectangle's 4 known corners) rather than a chain of turtle-style relative offsets: `ocad.py`'s own `corner_n_radius_list()`, ported. ``n`` is the number of segments per rounded corner (a sharp vertex, `r=0`, is always just 1 point either way -- same convention as `cr2dt()`'s ``s``).

```
sec = corner_n_radius_list([[0,0],[10,0],[10,10],[0,10]], [1,2,3,4], 10)
show(linear_extrude(polygon(sec), h=3))
```

Like `cr2dt()`, always treats ``p0`` as a CLOSED loop -- corner 0's rounding uses the LAST point as its "previous" neighbor, and vice versa, so leave a point's radius at 0 if you want it left exactly as given (e.g. the end of what's conceptually an open path). Raises if a requested radius doesn't fit between a vertex and either neighboring edge -- same check as `cr2dt()`.

`ocad.py` also has a `corner_n_radius_list_3d()` 3D counterpart (absolute 3D points, corners rounded in each one's own local plane, same idea as `cr3dt()` vs `cr2dt()`) -- not ported here, ask if you want it too.

fillet()

```
fillet(shape, edges, radius)
```

Exposed to scripts | `backend/brep.py:6320` | Used in: `hollow_cylinder_with_star.py`, `hook.py`

Rounds ``edges`` of ``shape`` with the given radius -- a real, exact B-rep fillet (a genuine curved/cylindrical blend surface computed by OCCT), not a faceted approximation built out of extra geometry the way OpenSCAD scripts usually fake a fillet (e.g. unioning in a `rotate_extrude()`'d quarter-circle profile, or `minkowski()` with a sphere).

``edges`` is a selection of edges FROM ``shape`` itself -- most simply ``shape.edges()`` for every edge (real `build123d` Shape objects, which is what `box()/cylinder()/union()/etc` all return here, have this method directly), or narrowed down first, e.g. ``shape.edges().filter_by(Axis.Z)`` for only the vertical ones (needs ``from build123d import Axis``), or ``shape.edges().sort_by(Axis.Z)[-4:]`` for just the top 4. See `build123d`'s own docs for the full edge- selector API -- `brep.py` doesn't wrap that part, ``shape`` is a real `build123d` object so its own methods are already there to use directly.

``edges`` can also be a single bare Edge (not wrapped in a list) -- e.g. ``fillet(a, a.edges()[3], 2)`` works the same as ``fillet(a, [a.edges()[3]], 2)`` -- same convenience as `show_edges()/label_edges()`. A bare Wire/Shell/Solid/Compound/Face also works directly (e.g. straight out of `mixed_wire()/wire_from_edges()`) -- expanded to its own real constituent edges automatically, rather than being mistaken for one single (invalid) Edge.

Example:

```
a = cube([20,20,20])
rounded = fillet(a, a.edges(), 3)
```

If ``radius`` is too big for the local geometry at one of ``edges`` (blend surfaces from adjacent edges would collide/overlap), OCCT raises "Failed creating a fillet with radius of X" -- a real geometric limit, not a bug: some edges in your selection just can't take that big a rounding given how close they are to other features. Try a smaller radius, fillet a smaller edge subset at a time, or call `max_fillet_radius(shape, edges)` first to get the largest radius that actually works for that exact selection.

UNVERIFIED against a real build123d install, same caveat as the rest of this file (see module docstring) -- `fillet()/chamfer()` are thin passthroughs to build123d's own top-level `fillet()/chamfer()` functions, so if something's wrong here it's most likely just the argument order/names, not the underlying operation.

fillet3points()

```
fillet3points(sec, r=1, s=20)
```

Exposed to scripts | backend/brep.py:3662 | Used in: —

The fillet arc rounding the corner at the middle point of the 3-point path ``sec`=[p0,p1,p2]`, radius ``r``, as an ``s``-segment point list -- the single-corner building block `cr2dt()` applies at every vertex of a whole profile at once (see its docstring for the underlying math).

organic_fillet()

```
organic_fillet(solid_a, solid_b, radius_a, radius_b=None, samples=100, fillet_samples=20,
closed_loop=True, thickness=None, ruled=False, max_stations=None, ear_a=0, ear_b=0)
```

Exposed to scripts | backend/brep.py:4535 | Used in: —

One-call fillet between two solids for the HARD case where the native OCCT `fillet()` -- and even `loft_fillet()`'s own B-rep-native bezier-station approach -- aren't viable, because the seam between ``solid_a`` and ``solid_b`` doesn't chain into one clean OCCT edge loop (a real, not-uncommon failure mode on organic/sculpted/spline geometry: seen directly on a mouse-body-style shape, see examples/ 26_mouse_body_wedge_fillet_ribbon_technique.py, whose own comment reads "the seam between s1 and b doesn't chain into one clean OCCT edge loop, so a plain `fillet()` call isn't viable here"). If your two shapes DO produce a clean `intersection()/offset_face()` result, prefer `loft_fillet()` (or plain `fillet()`) instead -- both are cheaper and stay fully inside the B-rep kernel; reach for this one only after those have failed or thrown, the way example 26 does.

Packages the "ribbon fillet" workaround from that example -- until now hand-assembled as 5+ separate mesh-domain calls with a couple of easy-to-get-backwards details (which two args are lines vs. solids, the (line_b, line_a, spine) argument order `convert_3lines2fillet()` actually wants) -- into one call operating directly on the two solids:

1. Tessellate ``solid_a`/`solid_b`` and find every triangle-triangle crossing segment between them (`two_solids_intersection()`).
2. Stitch those into one continuous seam chain (`contiguous_chains()`, first/longest chain used as the spine) and resample it to ``samples`` evenly spaced points (`equidistant_pathc()`).
3. Walk that seam outward across each solid's own surface by ``radius_a`/`radius_b`` (`o_3d()`, the surface-following offset -- NOT a straight-line offset) to get the two outer rails of the fillet ribbon.
4. Fit a smooth grid of cross-section arcs between those two rails, pulled through the spine (`convert_3lines2fillet()`).

5. Thicken that (still paper-thin) ribbon into a real volume (`solid_from_fillet_closed()`, ``thickness`` -- defaults to `-radius_a`, matching example 26's own `-3` with radius `7/-3`) so it can be booleaned against your model.
6. Loft the thickened grid into a genuine B-rep Solid (`solid_from_wedge_grid()` -- the exact, already-designed counterpart for this grid shape).

Returns that solid, sized and positioned to sit right over the seam -- `difference()` it against the union/intersection of your two original (unfilleted) solids to actually carve the round, the same way example 26's own final ``sol = difference(sol, f3)`` does. Does NOT do that union/difference itself, since how the two solids should first be combined (union vs. a boolean `intersection()`, as example 26 uses) is model-specific.

``radius_a`/`radius_b``: fillet blend width measured outward across each solid's own surface from the seam (signs matter -- see `o_3d()`'s own docstring; example 26 uses `+7` and `-3`, i.e. the two rails don't have to be symmetric). ``radius_b`` defaults to ``radius_a`` if not given.

``samples`/`fillet_samples``: spine resampling density / cross-section arc resolution -- same knobs as `equidistant_pathc()`'s ``s`` and `convert_3lines2fillet()`'s own ``s``.

``thickness``: how far to thicken the fillet ribbon into a volume before lofting (`solid_from_fillet_closed()`'s own ``d``) -- defaults to `-radius_a`` (matching example 26's own `-3-with-radius-7/-3` convention); pass an explicit value if that default doesn't carve deep enough (or too deep) for your two solids.

``ruled`/`max_stations``: passed straight through to `solid_from_wedge_grid()`. Left at `ruled=False` (smooth loft) here DELIBERATELY, unlike example 26's own `ruled=True` -- `ruled=True` produces a visibly dense, flat-faceted result (every station becomes its own straight-walled facet instead of blending into its neighbors), which is almost certainly why that example's own mouse-body came out looking faceted rather than smoothly filleted when opened elsewhere. Pass `ruled=True` yourself only if you specifically want the piecewise-straight look (or need the speed).

``ear_a`/`ear_b``: passed straight through to `two_solids_intersection()`'s own ``ear1`/`ear2`` (set 1 if the default `triangulate_solid_openx()` tessellation of that particular solid comes out wrong -- see its own docstring).

Raises `ValueError` if the two solids don't actually intersect (no crossing triangles found at all -- check they truly overlap along a seam first).

UNVERIFIED end-to-end against a real `build123d/OCCT` install (see module docstring) for the `solid_from_wedge_grid()` conversion at the very end, same as every other OCCT-calling function here -- but every step through the thickened point grid is pure point-list math with no OCCT call at all, and was verified standalone this session by importing `ocad.py` directly (all of `two_solids_intersection()/contiguous_chains()/equidistant_pathc()/o_3d()/convert_3lines2fillet()/solid_from_fillet_closed()` import and run with no OCCT dependency whatsoever); this function's only job is orchestrating those exact same calls, in the exact same order and argument convention example 26 already proves out by hand, so the only genuinely new/unverified step is the final `solid_from_wedge_grid()` hand-off, which is the same well-established conversion `loft_fillet()` above already relies on.

solid from fillet()

```
solid_from_fillet(fillet_1, d)
```

Exposed to scripts | `backend/brep.py:5034` | Used in: `hook.py`

Real-B-rep override of `ocad.py`'s own `solid_from_fillet()` -- `ocad.py` built its "cutting edge from a fillet" by hand: offsetting the raw point grid with `surface_offset()`, then stitching each original row to the REVERSED matching row of the offset copy into a closed per-station polygon (``cpo(fillet_1)[i] + flip(cpo(fillet_2)[i])``), producing its own point-list "solid" representation -- exactly the kind of pre-B-rep mesh data this whole file exists to promote past (see `prism()`'s own docstring for the same class of fix). That representation was never a real `Shape` to begin with, so a script working entirely in real B-rep (`fillet()`, `difference()`, etc.) had no way to actually use it -- feeding it into a real boolean like `difference()` would fail immediately.

``fillet_1`` here is the grid `convert_3lines2fillet()` returns (a list of same-length point rows -- see its own docstring for how to derive one from two real surfaces via `offset_face()+intersection()`). Builds the real fillet `Face` through that grid with

`surface_from_points_grid()` (exactly the "fallback" path `convert_3lines2fillet()`'s and `loft_fillet()`'s own docstrings already point to), then `thicken()`s it by `d` into a genuine solid -- same sign convention as `thicken()` itself (positive/negative picks which side gets material; if it comes out on the wrong side, flip the sign of `d`), so the result is a real Shape you can pass straight into `difference()/union()/etc.`

```
spine = sample_curve(intersection(a, b), n)
line_b = sample_curve(intersection(offset_face(a, blend), b), n)
line_a = sample_curve(intersection(a, offset_face(b, blend)), n)
grid = convert_3lines2fillet(line_a, line_b, spine, s=15)
cutter = solid_from_fillet(grid, -1)
r1 = difference(r1, cutter)
```

If your fillet is naturally a full closed loop (not two open ends), build the grid with `closed_loop=1` on `convert_3lines2fillet()` and use `solid_from_fillet_closed()` below instead -- same reasoning as `surface_from_points_grid_closed()`'s own docstring for why a duplicated-first-row grid isn't actually the same as a genuinely closed one.

solid from fillet closed()

```
solid_from_fillet_closed(fillet_1, d)
```

Exposed to scripts | backend/brep.py:5080 | Used in: —

Like `solid_from_fillet()` above, but for a fillet grid that wraps all the way around a closed spine (build it with `closed_loop=1` on `convert_3lines2fillet()`) -- `ocad.py`'s own `solid_from_fillet_closed()`, ported the same way:

`surface_from_points_grid_closed()` (the real, seamless closed-loop fit -- see its own docstring for why a plain `surface_from_points_grid()` on a duplicated-first-row grid ISN'T the same thing) instead of `surface_from_points_grid()`, then `thicken()` exactly as `solid_from_fillet()` does.

Offsetting & Shelling (5)

erode()

```
erode(shape, depth, kind='arc', openings=None)
```

Exposed to scripts | backend/brep.py:5736 | Used in: —

Uniformly shrinks a SOLID inward by `depth` from every face, returning a smaller SOLID -- not a hollow shell. `offset()` alone can't do this directly (for a solid it always SHELLS, producing the hollow material near the boundary, not a smaller solid on its own); `erode()` gets there by shelling and then subtracting that shell back out of the original shape.

Reach for this to approximate which region of a casting solidifies/coolS LAST: the material with the most surrounding mass (farthest from every outer surface) survives the deepest erosion, so progressively eroding a part and watching what core remains is a rough geometric proxy for where shrinkage porosity risk concentrates -- see `erosion_series()` for the stepwise version of exactly that.

`depth` should be POSITIVE (how far to erode inward) -- unlike `offset()`'s own `amount` (where negative means inward, positive outward), this only ever erodes, so it reads naturally without needing to remember a sign convention.

`kind`/'openings': passed straight through to `offset()` -- see its own docstring for what they mean.

IMPORTANT, confirmed on this app's own test shape (the filleted sphere/cube cavity from `08_filletted_cube_sphere_cut.py`): a sharp CONCAVE corner (where several faces meet cupping inward -- this shape's own 3-flat-wall cavity is exactly that) combined with small fillets right at that corner is a genuinely hard case for OCCT's offset algorithm and can fail outright (`offset()`'s own `RuntimeError`), even with one face opened and a small depth. Two fixes, in order of how likely they are to help: 1. Pass `openings=` with SEVERAL of the concave region's own

```
faces, not just one -- e.g. the same face list you'd hand
shell_from_faces() for that cavity -- giving OCCT more room
to resolve the shell around the tight corner.
```

2. Erode the shape BEFORE any small fillets are applied at that

corner, not after -- fillets sitting right at an already-tight concave junction are what tends to destabilize this, and for a hot-spot/rough solidification check, doing it on the simplified unfilleted geometry is usually accurate enough anyway (fillets are a small local detail that rarely changes which broad region solidifies last).

```
raw = difference(b, a) # before fillet()
concave_faces = [raw.faces()[i] for i in [...]] # pick with the Pick tool
core = erode(raw, 2, openings=concave_faces)
show(core)
```

erosion series()

```
erosion_series(shape, depths, kind='arc', openings=None)
```

Exposed to scripts | backend/brep.py:5791 | Used in: —

`erode()` at every depth in `depths`, each computed independently from the SAME original `shape` (not compounded from the previous step's result, so there's no accumulated error and you can jump straight to any single depth) -- the stepwise "which region survives the deepest peel" scan, ready to color/show in one loop:

```
raw = difference(b, a)
concave_faces = [raw.faces()[i] for i in [...]]
cores = erosion_series(raw, [1, 2, 3, 4], openings=concave_faces)
palette = ["red", "orange", "yellow", "green"]
for core, col in zip(cores, palette):
    show(color(core, col))
```

Returns a list of solids, same order/length as `depths`. If one depth fails (too deep for this shape's own tightest feature -- see `erode()`'s own docstring), that `RuntimeError` propagates immediately rather than being silently skipped -- knowing exactly which depth stopped working is itself useful here, since it marks how deep material extends before hitting this shape's own geometric pinch point.

offset()

```
offset(shape, amount, kind='arc', openings=None)
```

Exposed to scripts | backend/brep.py:5659 | Used in: —

Grows (`amount > 0`) or shrinks (`amount < 0`) `shape` by `amount` -- same idea as OpenSCAD's `offset()`, just calling `build123d`'s own native `offset()` directly rather than a hand-rolled polygon-offset algorithm (2D polygon offsetting -- correctly handling corners, self-intersections after a large inward offset, holes, etc -- is a genuinely hard problem in its own right, exactly the kind of thing this app leans on the real OCCT kernel for rather than reimplementing; see `sweep_sec2path()`'s own docstring for a case where a hand-rolled attempt at something similar didn't go well).

`shape` can be: - a 2D shape (`circle()`/`square()`/`polygon()`/etc): grows/shrinks its

outline -- e.g. `offset(square(10, center=True), 2)` is a 14x14 square; `offset(..., -2)` a 6x6 one. A shape with holes has its holes offset the opposite direction automatically (so growing the outer boundary shrinks the holes, like OpenSCAD's own `offset()`), so e.g. offsetting a ring outward makes it a wider ring, not a filled disc.

- a 3D solid: SHELLS it -- hollows it out to a uniform wall

thickness `amount`, sealed on every face by default (a closed hollow shell) unless you open some of them via `openings=` (e.g. `openings=box.faces().sort_by(Axis.Z)[-1]` leaves the top face open, like a hollow box with no lid -- same `edges()`/`faces()` selector API `fillet()`/`chamfer()` use; see the README's "Manually selecting edges" section).

A FULLY SEALED shell (`openings=None`) of a solid that's entirely curved with no flat face to anchor on -- a sphere or a capped cylinder are the classic cases -- is a known-fragile

operation in OCCT itself: the thicken algorithm can fail outright there (an "offset Error"), and unlike a mitered-vs-rounded-corner problem, trying a different `kind` often does NOT fix it (it's not a corner-handling issue at all -- it's that a fully closed offset of a shape with no flat "seam" to start stitching from is a genuinely hard case for the algorithm). If you hit that, don't bother cycling through `kind` values -- either open at least one face via `openings=` (even a small one), or, most robust of all, skip offset() entirely and build the hollow shape directly out of two nested copies with difference() instead -- e.g. difference(sphere(10), sphere(8)) for a hollow sphere with a 2-unit wall. That's a plain boolean, not dependent on the thicken algorithm at all, so it always works.

kind picks how corners are handled where the offset outline can't stay parallel to a sharp original corner: "arc" (default): rounds the corner. "intersection": a sharp, mitered corner (extends both offset

edges until they meet -- can fail on a corner sharp/tight enough for that extension to self-intersect, same class of issue as sweep_sec2path()'s "transformed" transition).

"tangent": a straight-line transition between the two edges.

offset 3d()

```
offset_3d(sec, r, direction=None, type=1)
```

Exposed to scripts | backend/brep.py:11499 | Used in: —

B-rep equivalent of ocad.py's own offset_3d() -- grows/shrinks a closed 3D loop `sec` that lies in (or close to) one plane, using a real, self-intersection-trimmed 2D offset (ocad's own offset(), NOT build123d/OCCT's own offset() -- confirmed this session that OCCT's offset is fragile on exactly this kind of organic concave boundary at ordinary margins, self-intersecting where ocad's own trims cleanly instead; slider_housing()'s own l3 uses exactly this technique inline -- this factors that same pattern out into a standalone, reusable function).

HOW: build a local 2D (u, v) frame on `sec`'s own plane (bd.Plane's own x_dir/y_dir at that origin/normal), flatten `sec` into it, offset in 2D via o4.offset(), then map the result back into 3D.

```
l6 = [[p[0], p[1], 40.0] for p in l5] # l5 flattened to z=40
l7 = offset_3d(l6, 40, direction=[0, 0, 1])
show(*[line3d(l, 0.3) for l in (l6, l7)], 'blue')
```

`direction`: the plane's own normal, if you already know it (e.g. the same `direction` a mesh_boundary_loop()/mesh_cap_ribbon() call used to produce `sec` in the first place). Skips the best-fit step below and uses exactly this direction instead -- useful when `sec` is only APPROXIMATELY planar (real mesh boundary noise) and you want the offset computed relative to the TRUE intended plane, not whatever slightly-off plane best-fits the noisy points exactly. Defaults to a best-fit normal (SVD of the centered points) when left None -- fine for a genuinely (near-)planar loop.

`r`: offset distance, +ve grows outward, -ve shrinks inward (same sign convention as ocad's own offset()/offset_3d()).

`type`: passed straight through to ocad's own offset() (`type=1` default; ocad's own advice is to try `type=2` yourself if a particular loop needs it). Unlike slider_housing()'s own inline version, this does NOT silently auto-retry with type=2 on a collapse -- call it again yourself with type=2 if type=1 comes back too short, so a genuinely-too-large `r` doesn't get masked by a silent fallback.

Returns a plain list of [x,y,z] points, same convention `sec` used -- NOT necessarily the same length as `sec` (ocad's own offset() can add/trim points where it resolves self-intersections). Resample with _resample_closed_ring() first if you need it to match `sec`'s own point count/indexing station-for-station, e.g. for building a wedge grid alongside the original loop.

Raises RuntimeError if the offset collapses to fewer than 3 points at this `r`/`type` (too large a shrink, or an outward margin that still crosses itself even after trimming) -- try a smaller `abs(r)`, the other `type`, or check `direction` is actually right for `sec`.

offset_shell()

```
offset_shell(shell, amount, both=False, validate=True)
```

Exposed to scripts | backend/brep.py:5446 | Used in: —

A safer `thicken()` for exactly the failure mode that motivated it: `thicken()` (`bd.thicken()` above -- OCCT's `BRepOffsetAPI_MakeThickSolid` under the hood) doesn't reliably RAISE when the requested `amount` can't actually exist as a valid, non-self-intersecting solid everywhere on `shell` -- by far the most common real cause is a concave fillet somewhere on the shell with a radius SMALLER than `abs(amount)`: a uniform wall that thick would need to fold back through itself right at that fillet (its own offset radius would have to go negative there, which isn't geometrically possible). OCCT is just as likely to hand back a technically-non-null but topologically INVALID or self-intersecting solid as it is to raise a clean exception for this -- which is exactly what "the result looks wrong" (a spiky/self-crossing/garbled shape, not a crash) usually means in practice, and is easy to not notice until much later (export, a boolean against it, a slicer).

Calls the exact same `bd.thicken()` `thicken()` does, then (unless `validate=False`) runs the real result through OCCT's own `BRepCheck_Analyzer` -- the standard validity checker some `build123d` versions expose as `Shape.is_valid()`, called directly via raw OCP here instead since this app has no confirmed real `build123d` install to check whether every version wraps it the same way (see this module's own header docstring; `import_brep()`'s own `_heal_imported_shape()` above uses this exact same raw-OCPC-access pattern for the same reason) -- and raises a clear, actionable error instead of silently handing back bad geometry if it comes back invalid.

`amount`/`both` mean exactly what they do for `thicken()` -- see its own docstring first for the basics; this only adds the validity check and a more specific error message on top.

```
f2 = shell_from_faces(f1)
f3 = offset_shell(f2, -30)  # raises a clear error naming the
                           # likely cause instead of silently
                           # handing back a bad solid if some
                           # fillet on f2 is tighter than 30
```

If it raises: the two real fixes are increasing whichever fillet(s) on the ORIGINAL solid (before `shell_from_faces()`) are smaller than `abs(amount)` so they can actually accommodate a wall that thick, or reducing `abs(amount)` to fit the tightest fillet that's actually on `shell`. There's no way to make an offset that thick valid AT that fillet without changing one of those two numbers -- it's a real geometric conflict, not a bug to work around.

Pass `validate=False` to skip the check and get `thicken()`'s raw result straight back, e.g. if you already know it's fine and just want to avoid the extra check's cost on a very large shape.

Transformations (20)

axis_rot_1()

```
axis_rot_1(sol, ax1, loc1, theta)
```

Exposed to scripts | backend/brep.py:4242 | Used in: —

Rotates `sol` by `theta` degrees about an ARBITRARY axis -- the axis passes through point `loc1` with direction `ax1`, unlike `rotate()/rot()` above, which only ever rotate about an axis through the origin. `ocad.py`'s own `axis_rot_1()`, ported: the original built this out of a `translate-to-the-solid's-own-centroid/apply-a-rotation-matrix/translate-back` dance; here it's just `build123d`'s own native `Shape.rotate(axis, angle)`, which already IS "rotate about an arbitrary axis" directly -- no centroid math needed.

```
c1 = cylinder(r=5, h=20)
c2 = axis_rot_1(c1, [1, 1, 0], [0, 0, 20], -90)
show(c1)
show(color(c2, "blue", alpha=0.5))
```

axis_rot_1_pts()

```
axis_rot_1_pts(pts, ax1, loc1, theta)
```

Exposed to scripts | backend/brep.py:4263 | Used in: —

Rotates a raw point list `pts` ([x,y] or [x,y,z] points -- 2D points are padded to z=0 first) by `theta` degrees about an ARBITRARY axis (direction `ax1` through point `loc1`) -- the point-list counterpart of `axis_rot_1()` (which rotates a real build123d Shape the same way, via its own native `.rotate()`), for when you have a plain point list rather than a Shape (no build123d needed, same spirit as `fit_pline2line()/xrot4d()` above).

Always returns 3D points, even for 2D input (see `fit_pline2line()`'s own note on why -- rotating about an arbitrary axis generally needs to leave whatever plane the input started in, so flattening back to 2D would just throw the rotation away).

Handy for rotating a profile's points BEFORE building a real shape from them -- e.g. ``polygon(axis_rot_1_pts(pts, ax, loc, theta)`` -- instead of building the shape first and rotating it afterward with `axis_rot_1()`; this also sidesteps a real build123d/OCCT rough edge some rotated Face boundary wires can hit later (`position_at()` raising `""NoneType" object has no attribute 'NbNodes'"` when passed to `p_line3d()/sweep_sec2path()`).

```
pts = arc_3p([10, 0], [0, 50], [10, 100]) + [[-10, 100], [-10, 0]]
pts = axis_rot_1_pts(pts, [0, 1, 0], [10, 0, 0], 60)
d = polygon(pts)
show(p_line3d(d, 0.1))
```

flip()

```
flip(sec)
```

Exposed to scripts | backend/brep.py:1091 | Used in: —

Reverses the order of a list -- e.g. `flip([1,2,3]) -> [3,2,1]`. Handy for reversing a point list's winding direction.

mirror()

```
mirror(v, shape)
```

Exposed to scripts | backend/brep.py:4295 | Used in: —

Mirrors `shape` across the plane through the origin whose normal is `v` -- same convention as OpenSCAD's `mirror(v, ...)`, e.g. `mirror([1, 0, 0], shape)` flips it across the YZ plane.

mirror_line()

```
mirror_line(pts, n1, loc)
```

Exposed to scripts | backend/brep.py:2733 | Used in: —

Mirrors every point in `pts` across the plane through `loc` with normal `n1` -- e.g. `mirror_line(circle_pts, [1,1,0], [0,0,0])` reflects a 2D point list across the plane $x+y=0$. 2D points are treated as $z=0$ while mirroring; the result comes back 2D too unless mirroring genuinely moves a point off the $z=0$ plane.

mirror_point()

```
mirror_point(pnt, n1, loc)
```

Exposed to scripts | backend/brep.py:2754 | Used in: —

Mirrors a single point `pnt` -- see `mirror_line()`.

resize_solid()

```
resize_solid(shape, rsz)
```

Exposed to scripts | backend/brep.py:5593 | Used in: hook.py

Real-B-rep override of `ocad.py`'s own `resize_solid()` (`=rsz3d`) -- NOT the same as the bare `resize` name: `resize` with no suffix is NOT one of this app's functions at all -- `ocad.py` does ``from numpy import *`` at its own top, and Layer 1 flattens every public name out of `ocad.py` bare into every script's namespace (see `prism()`'s own docstring for how the two-layer namespace works), which includes numpy's own re-exported `resize` right along with it. So calling `resize(some_shape,`

[w,h,d]) silently calls numpy's array-resize instead -- it doesn't raise, it just returns an ndarray, which is exactly why show()'s own type-check on the result is what actually catches the mistake (numpy has no way to know `some\_shape` wasn't meant as array data). Use `resize_solid()` (or `resize_solid_centered()` below) instead -- neither name collides with anything numpy exports.

Non-uniformly scales `shape` so its bounding box becomes exactly `rsz` = [x, y, z] -- same per-axis independent scale ocad.py's own `rsz3d()` computed by hand off min/max coordinates, just driven by build123d's real `bounding_box()` instead of scanning a raw point list. The shape's MIN corner (`bounding_box().min` -- the "bottom left" corner ocad.py's own docstring describes) stays fixed in place; growth/shrink happens away from that corner. Use `resize_solid_centered()` instead if you want the shape's CENTER to stay fixed instead.

```
sol = box([10, 5, 3])
sol1 = resize_solid(sol, [5, 6, 7]) # sol's min corner unmoved
```

resize_solid_centered()

```
resize_solid_centered(shape, rsz)
```

Exposed to scripts | backend/brep.py:5635 | Used in: —

Like `resize_solid()` above, but the shape's CENTER (`bounding_box().center()`, not its min corner) stays fixed in place instead -- ocad.py's own `resize_solid_centered()` (=rsz3dc), ported the same way.

rot()

```
rot(a, shape)
```

Exposed to scripts | backend/brep.py:4216 | Used in: drill.py

A string-based axis+angle rotation DSL -- e.g. `rot('z30x70y-90', shape)` rotates first about Z by 30 degrees, then X by 70, then Y by -90, strictly in the order the axis letters appear in the string (not always x-then-y-then-z like `rotate()` above -- whatever order you write it in).

Like `rotate()` above, each individual step here is a pure single- axis rotation (see `_rotate_single_axis()`), so there's no dependence on build123d's own multi-axis composition convention.

`shape` can be a real build123d Shape OR a raw 2D/3D point list -- see `_rotate_single_axis()`'s own docstring for the point-list fix (ocad.py scripts commonly call `rot(...)` directly on `sinewave()`/ `cosinewave()` output, e.g. `surface_from_2_waves()`'s own docstring).

rot2d()

```
rot2d(a, sec)
```

Exposed to scripts | backend/brep.py:2759 | Used in: —

Rotates 2D point list `sec` by `a` degrees counter-clockwise around the origin (any extra coordinates beyond x,y pass through unchanged).

rot_p()

```
rot_p(a, points)
```

Exposed to scripts | backend/brep.py:2865 | Used in: —

Rotates a point list `points` (2D or 3D -- `sinewave()`/ `cosinewave()` output, `cr2dt()`'s output, etc) by axis letters and angles packed into a single string `a` -- the point-list sibling of `rot()`/ `rotate()`, which only work on real Shapes. Same string convention as ocad.py's own `rot()`/brep.py's own `rot()`: each axis is applied in the ORDER it appears in the string.

```
rot_p('x90', [[0, 0], [10, 0]])
# rotates around X by 90 deg

rot_p('z30x70y-90', points)
# rotates z by 30, then x by 70, then y by -90, in that order
```

2D points are padded to z=0 first (via `c23()`); extra trailing values on a point (e.g. a corner-radius column) pass through unchanged, same as `xrot4d()/yrot4d()/zrot4d()` (which this is built from). This is the tool for the `sinewave/cosinewave-into-a-plane` trick used throughout the `surface_from_2_waves()` examples -- instead of building each wave directly in its target plane (`'[[x, 0, y]` for `x, y` in `sinewave(...)]'`), you can now do:

```
l1 = rot_p('x90', sinewave(60, 3, 8, 60))
l2 = rot_p('x90z90', cosinewave(60, 3, 8, 60))
```

rotate()

```
rotate(angles, shape)
```

Exposed to scripts | [backend/brep.py:4189](#) | Used in: —

Rotates `shape` by `[x, y, z]` degrees -- same convention as real OpenSCAD's `rotate([x,y,z], ...)`: applied as three separate single- axis rotations in sequence, first `x` degrees about `X`, then `y` about `Y`, then `z` about `Z` (OpenSCAD's own documented order for the vector form). Implemented as three chained single-axis rotations (see `_rotate_single_axis()`) specifically so this doesn't depend on `build123d`'s own internal convention for a combined multi-axis `Rotation(x,y,z)` call, which isn't pinned down clearly enough in its docs to rely on directly.

`shape` can be a real `build123d` Shape OR a raw 2D/3D point list (e.g. `sinewave()/turtle3d()` output) -- see `_rotate_single_axis()`'s own docstring for the point-list fix and why it matters.

scale()

```
scale(v, shape)
```

Exposed to scripts | [backend/brep.py:5579](#) | Used in: —

Scales `shape` about the origin -- same convention as OpenSCAD's `scale(v, ...)`: `v` is either one number (uniform scale) or `[x,y,z]` (independent per-axis scale).

translate()

```
translate(v, shape)
```

Exposed to scripts | [backend/brep.py:4136](#) | Used in: —

Moves `shape` by `[x,y,z]` -- same convention as `ocad.py`'s own `translate(p, sec)`, just with `build123d`'s B-rep Shape instead of a prism/section point list. Returns a new shape; doesn't mutate the original (`build123d` shapes are immutable under these operators).

xrot()

```
xrot(theta)
```

Exposed to scripts | [backend/brep.py:2773](#) | Used in: —

The 3x3 rotation matrix for rotating a point around the `X` axis by `theta` degrees (right-hand rule) -- `ocad.py`'s own `xrot()`, ported as a plain nested Python list (3 rows of 3 numbers) instead of a numpy array, matching `brep.py`'s other point-list math (no numpy dependency needed inside `brep.py` itself). Apply it to a point as a row vector times this matrix, same convention the original used (see `xrot4d()` below for a ready-made point-rotation helper that does exactly that, or write your own with numpy if you already have `np`/'numpy' imported in your script: `array(v) @ array(xrot(theta))`).

xrot4d()

```
xrot4d(theta, v)
```

Exposed to scripts | [backend/brep.py:2820](#) | Used in: —

Rotates a point `v` (a `[x,y,z]` point, or a longer one like `[x,y,z,extra,...]` -- e.g. `pts3()/corner_radius3d()`-style points with a trailing corner-radius value) around the `X` axis by `theta` degrees, using `xrot()` -- `ocad.py`'s own `xrot4d()`, ported. Any elements beyond the first 3 (`x,y,z`) pass through unchanged.

yrot()

```
yrot(theta)
```

Exposed to scripts | backend/brep.py:2794 | Used in: —

Same as xrot(), but the rotation matrix for the Y axis.

yrot4d()

```
yrot4d(theta, v)
```

Exposed to scripts | backend/brep.py:2831 | Used in: —

Same as xrot4d(), but rotates around the Y axis (via yrot()).

zrot()

```
zrot(theta)
```

Exposed to scripts | backend/brep.py:2804 | Used in: —

Same as xrot(), but the rotation matrix for the Z axis.

zrot4d()

```
zrot4d(theta, v)
```

Exposed to scripts | backend/brep.py:2836 | Used in: —

Same as xrot4d(), but rotates around the Z axis (via zrot()).

Booleans & Assembly Operations (8)

difference()

```
difference(base, *shapes)
```

Exposed to scripts | backend/brep.py:6083 | Used in: drill.py, hollow_cylinder_with_star.py, hook.py, tap.py

CSG difference: `base` minus every shape in `shapes` (the `` operator) -- same argument order as ocad.py's usual difference(first_child, remaining_children...) convention.

group()

```
group(*shapes)
```

Exposed to scripts | backend/brep.py:12953 | Used in: —

Bundles two or more shapes into one Compound "assembly" -- unlike union(), this does NOT boolean-fuse them into a single solid; each shape stays its own separate, distinct solid inside the Compound, exactly as it was. Mainly for export_step()/export_brep(): passing a group() of several shapes writes them all into ONE file as separate parts (an assembly), same idea as `to\_export` being "object or assembly" per build123d's own export_step() docstring -- rather than calling export_step() once per shape into separate files, or fusing everything into one solid with union() just to get a single file out.

```
a = cylinder(r=10, h=40)
b = box(30, 30, 5)
export_step(group(a, b), "both_parts.step") # 2 distinct
                                           # solids, 1 file
```

show() only ever takes ONE shape per call (call it once per shape, same as every other example does, to see them all in the viewer) -- group() is specifically for bundling shapes together for export, not for display. show(group(a, b)) still works (it tessellates the whole Compound into one combined display mesh), but you lose each part's own separate color/selectability in the viewer that way, so prefer plain show(a); show(b) for viewing and save group() for the

`export_step()/export_brep()` call itself.

intersection()

```
intersection(*shapes)
```

Exposed to scripts | backend/brep.py:6173 | Used in: —

CSG intersection of two or more B-rep shapes (the ``&`` operator).

split()

```
split(shape, bisect_by, keep='top')
```

Exposed to scripts | backend/brep.py:5853 | Used in: hook.py

Bisects ``shape`` with a plane (or a Face) and keeps one side, the other, or both -- build123d's own `split()`, exposed directly here so ``split(...)`` and ``Keep.X`` both just work in your script without ``from build123d import split, Keep`` first (build123d's own docs/ examples always show it written that way, since build123d itself doesn't auto-expose its own names into scripts the way this app's whole function library does).

``bisect_by`` is the cutting surface -- typically a real Plane (see `plane()/plane_from_equation()`) for a flat cut, or a Face for a curved one, e.g. carving the top off a block along an arbitrary wavy surface instead of a flat plane:

```
import ocad as o
block = cube([30, 30, 10])
wavy = surface_from_points_grid(o.plane_from_equation([1, 1, 0, 7], [50, 50]))
bottom_half = split(block, bisect_by=wavy, keep="bottom")
```

``keep`` selects which side survives -- accepts either a plain string ("top"/"bottom"/"both", case-insensitive -- same convention as this app's own `offset()`'s ``kind`` argument) or build123d's own `Keep` enum directly (`Keep.TOP/Keep.BOTTOM/Keep.BOTH`, already in scope, no import needed -- matches how build123d's own examples write it): "top" (the default): the half on ``bisect_by``'s positive-normal

side.

"bottom": the other half. "both": BOTH halves at once, combined into a single returned

shape -- same convention as this app's other multi-piece results (e.g. `offset()`'s hollowed shell); pull the two halves back apart with e.g. ``solids()`` on the result if you need them separately.

two_solids_intersection()

```
two_solids_intersection(a, b, tolerance=0.1, angular_tolerance=0.3)
```

Exposed to scripts | backend/brep.py:2282 | Used in: —

The raw intersection LINE SEGMENTS between two real B-rep shapes ``a`/`b`` -- ocad.py's own `two_solids_intersection()`, ported: tessellates each into a triangle mesh (`to_mesh()`, same tolerance/ angular_tolerance meaning as there) and tests every pair of nearby triangles (after a bounding-sphere prefilter) for a genuine triangle-triangle crossing. Returns the raw, DISCONNECTED list of 2-point segments -- feed it through `contiguous_chains()` to stitch them into one or more ordered curves:

```
segs = two_solids_intersection(a, b)
spine = contiguous_chains(segs)[0] # the first (often only) chain
```

WHY REACH FOR THIS instead of `intersection()` (the real B-rep boolean, exact and usually the right first choice): OCCT's exact boolean intersection can be numerically fragile on genuinely hard curved cases (see `fillet()`'s own docstring for the same class of real failure mode) -- a mesh-vs-mesh triangle intersection test doesn't care about exact analytic surfaces at all, just triangle soup, so it tends to be far more forgiving when `intersection()` itself struggles or errors. The tradeoff: the result is a faceted APPROXIMATION of the true curve (as accurate as ``tolerance`/`angular_tolerance`` make the underlying tessellation), not an exact analytic curve.

The full manual-fillet recipe this exists for (see `convert_3lines2fillet()`'s own docstring for the rest of the pipeline once you have ``spine``):

```
spine = contiguous_chains(two_solids_intersection(a, b))[0]
line_b = sample_curve(intersection(offset_face(a, blend), b), len(spine))
line_a = sample_curve(intersection(a, offset_face(b, blend)), len(spine))
grid = convert_3lines2fillet(line_a, line_b, spine, s=15)
patch = surface_from_points_grid(grid)
```

(mixing a mesh-derived ``spine`` with real-B-rep-derived ``line_a``/``line_b`` like this is fine -- they're just point lists by the time `convert_3lines2fillet()` sees them.)

REAL PERFORMANCE NOTE: this is a plain $O(\text{len}(\text{mesh_a}) * \text{len}(\text{mesh_b}))$ pair of nested loops (with a cheap bounding-sphere distance check to skip clearly-far pairs) -- `brep.py` deliberately has no numpy dependency (see `xrot()`'s own docstring), so this doesn't get `ocad.py`'s fully vectorized `einsum` version. Fine for the moderate mesh sizes to `_mesh()` produces at normal tolerances; if it's slow, tessellate coarser first (a bigger ``tolerance`` here) -- fewer, larger triangles means far fewer pairs to test.

union()

```
union(*shapes)
```

Exposed to scripts | backend/brep.py:5901 | Used in: —

CSG union of two or more B-rep shapes (the ``+`` operator).

CONFIRMED real failure mode (real user report): fusing several already-unioned shapes against one more shape that meets them across a large EXACTLY COPLANAR shared area (e.g. a flat base plate whose top face sits flush with several standing panels' bottom edges, all resting at the same z) can raise ``AttributeError: 'NoneType' object has no attribute 'NbNodes`` deep inside OCCT -- same fragility class already documented in `sweep_sec2path()`'s/ `p_line3d()`'s own docstrings, here triggered inside the boolean fuse itself rather than a sweep. It reliably showed up only once SEVERAL pieces' coincident contact area had accumulated (each piece unioned individually against the base worked fine) -- OCCT's fuse is well known to be far more fragile on large flush/coplanar contact than on a genuine volumetric overlap or a small contact patch.

FIX: don't let two pieces meet across an exactly flush shared plane -- give the piece real volumetric overlap into the others instead (a thicker base shifted to poke slightly INTO whatever sits on top of it, say), e.g.:

```
base = translate([0, 0, -0.5], linear_extrude(square(70), 2))
# instead of: linear_extrude(square(70), 1) # flush at z=0
```

``clean()``-ing operands right before a fuse that's struggling is also worth trying, but didn't fix this particular case on its own -- the real fix was removing the flush/coincident contact.

union_fuzzy()

```
union_fuzzy(*shapes, fuzzy_value=0.001)
```

Exposed to scripts | backend/brep.py:5942 | Used in: —

Same job as `union()` -- CSG union of two or more B-rep shapes -- but drops past `build123d`'s ``+`` operator to call OCCT's `BRepAlgoAPI_Fuse` directly so a "fuzzy value" can be set explicitly. Reach for this INSTEAD of `union()` specifically when you hit the large-flush/coplanar-contact fragility `union()`'s own docstring documents (``AttributeError: 'NoneType' object has no attribute 'NbNodes`` or similar deep-OCCT failures on pieces that meet across an exactly-shared flat area) and giving the pieces real volumetric overlap isn't practical.

WHAT FUZZY VALUE ACTUALLY DOES: OCCT's boolean engine normally treats two vertices/edges as "the same" only within its ordinary geometric tolerance (tiny, near machine precision). A large flush contact multiplies how many near-coincident vertex/edge pairs the algorithm has to disambiguate at once, and it's exactly that disambiguation step that goes fragile. `SetFuzzyValue(fuzzy_value)` widens the distance within which OCCT is willing to merge near-coincident vertices/edges BEFORE doing the boolean -- the same knob FreeCAD's own Part Boolean feature exposes as "Fuzzy value" for this identical class of problem. Too large a value can weld together geometry that was only meant to be close,

not coincident, so start small (the 1e-3 default is in the same units as the model, i.e. 0.001 mm for a millimeter-scale part) and raise it only as far as actually needed.

``shapes`` accepts the same `union(a,b,c)` / `union([a,b,c])` calling conventions as `union()` (via the same `_flatten()` helper).

UNVERIFIED against a real build123d/OCCT install (see this module's own header docstring) -- `BRepAlgoAPI_Fuse`'s constructor/argument `shapes` and `SetFuzzyValue()/Build()/IsDone()/Shape()` are documented OCCT API (unchanged across recent OCCT versions), and ``shape.wrapped`` / reconstructing ``type(shape)(topods)`` are the same established build123d patterns already used elsewhere in this file (see `_heal_imported_shape()`), but this exact call sequence hasn't been run against a real install. Falls back to plain `union()` (and therefore to whatever `union()` itself would have done, fuzzy value or not) if anything about the raw OCP path fails, so this is safe to try -- worst case it behaves exactly like calling `union()`.

union_multi()

```
union_multi(*shapes, fuzzy_value=0.001)
```

Exposed to scripts | backend/brep.py:6017 | Used in: —

Fuses THREE OR MORE shapes in a single combined OCCT boolean pass (`BOPAlgo_Builder`) instead of `union()`'s/`union_fuzzy()`'s pairwise chain (`a+b`, then `(a+b)+c`, then `((a+b)+c)+d`, ...). Falls back to `union_fuzzy()` for 1-2 shapes, where there's no "chain" to avoid.

WHY THIS EXISTS -- CONFIRMED real failure mode (real user report, `solid_from_mesh_slices(branches=True)` below): fusing many similarly-shaped solids that each share a near-coincident boundary with a common neighbor (adjacent branch "threads" built from overlapping/jittered copies of the same borrowed rings -- see `_jitter_ring()`'s own docstring) came back as multiple disconnected solids/faces instead of one watertight result once enough pieces accumulated (`n=200` station count), even after per-piece `.clean()` and `ShapeFix_Shape` healing. That symptom -- OCCT reporting several separate pieces after what should be one continuous fuse -- is a known weak spot of chaining `BRepAlgoAPI_Fuse` pairwise: each step only ever compares TWO already-fused results against each other, so any tolerance/rounding slop from an early step compounds into the next one, and a piece that only barely (near-coincidentally) touches its neighbor can end up on the wrong side of that slop by the time it's chained through several prior fuses.

`BOPAlgo_Builder` is OCCT's own lower-level, general N-ary boolean framework -- the thing `BRepAlgoAPI_Fuse` itself is built on for the 2-argument case -- run here directly across ALL the shapes AT ONCE in one combined pass instead of pairwise. OCCT's own documentation recommends exactly this (`BOPAlgo_Builder` over chained `BRepAlgoAPI_Fuse`) for fusing many shapes together for this same accumulated-tolerance reason. Whether it actually resolves the `n=200` branching case specifically hasn't been confirmed (no live build123d/OCCT install here to test against -- see this module's own header docstring) -- please try this in place of the plain `union(*solids)` call in `solid_from_mesh_slices(branches=True)` and report back whether the disconnected-fragment symptom is gone, and at what station count (`n`) it starts up again, if at all.

UNVERIFIED against a real build123d/OCCT install, same caveat as `union_fuzzy()` above. Falls back to `union_fuzzy()` (chained pairwise) if the `BOPAlgo_Builder` path fails for any reason, so this is safe to try -- worst case it behaves exactly like `union_fuzzy()`.

Mesh, Triangulation & Point-Cloud Solids ⁽¹⁶⁾

mesh_boundary_loop()

```
mesh_boundary_loop(shape, direction, side='top', angle_tol=80.0, tolerance=None, weld_tol=None)
```

Exposed to scripts | backend/brep.py:10954 | Used in: —

The exact, no-offset-needed answer to a natural follow-up question after `mesh_split_triangles()`: what IS ``shape``'s own top (or bottom) mesh region's true edge, in real 3D -- not a flattened approximation of it?

`shape_outline_raster()`'s own "outer" loop is a FLATTENED silhouette: every mesh vertex projected straight down onto one reference plane and rasterized there. That's the right tool for "what does shape look like from this direction", but it is NOT the same curve as the true 3D edge of a classified mesh region -- the two only coincide exactly where the true edge happens to sit directly "above" its own flattened shadow, which an organic/ curved boundary generally does not do everywhere along its own length. Filling that flattened outline with scan lines and ray-casting them onto the real mesh (`mesh_cap_solid()/surface_cap_solid()`, or a hand-built pipeline the same way) therefore has to shrink the outline inward by some safety margin first, or scan points land just past the true edge and miss the mesh on their way up/down -- CONFIRMED, a real limitation reported this session, not a tuning problem the margin can be tuned away: the result comes back a genuine "sub-surface", smaller than `shape``'s own true silhouette by however much margin was needed.

This function sidesteps the whole issue by never flattening anything -- it reads `shape``'s own actual mesh topology directly. Classifying triangles into "top"/"bottom" (the same per-triangle facing-direction rule `mesh_split_triangles()` uses) splits the mesh's own triangle-adjacency graph into two pieces; the edges where a "top" triangle borders something that ISN'T also a "top" triangle are, together, `shape``'s own true 3D edge for that side, exactly, down to the mesh's own tessellation -- no projection, no flattening, and nothing to offset inward at all.

```
loops = mesh_boundary_loop(sol, [0, 0, 1], side="top")
show(*[line3d(loop, 0.2) for loop in loops], 'blue')
```

Usually returns just one loop, for a simple silhouette; more than one is a real, valid result too -- e.g. a washer's own top skin has both an outer rim and an inner bore loop, same "outer" + "holes" split `shape_outline_raster()` returns for the flattened case, just as genuine 3D curves here rather than 2D ones. The first loop returned is always the largest by point count.

``weld_tol``: see `_mesh_side_boundary_loops()`'s own docstring for why this matters (`to_mesh()` isn't index-welded across B-rep face seams). Defaults to a small fraction of `shape``'s own bounding-box diagonal if left None.

Same ``angle_tol`/`tolerance`` meaning as `mesh_split_triangles()`'s own.

Returns a list of closed loops, each an ordered list of [x,y,z] points (first point NOT repeated at the end).

Raises `ValueError` if ``side`` has no triangles at all, or no boundary at all (would mean ``angle_tol`/`direction`` selected the WHOLE closed mesh, with nothing excluded to have an edge against).

mesh_cap_ribbon()

```
mesh_cap_ribbon(shape, direction, side='top', plane_point=None, n_lines=40,
points_per_line=20, angle=0.0, angle_tol=80.0, tolerance=None, weld_tol=None,
min_hole_fraction=0.02, ruled=True, tol=1e-06)
```

Exposed to scripts | backend/brep.py:11210 | Used in: —

The exact-edge answer to `surface_cap_solid()`'s own "sub-surface" limitation -- same overall ribbon-loft technique (angled scan lines across the footprint, each paired with its own curved twin and closed into a ring, all lofted at once via `solid_from_closed_rings()`), but the scan region's own edge is now `shape``'s TRUE mesh boundary (`mesh_boundary_loop()`) instead of a flattened, inward-offset silhouette -- so the result reaches exactly to `shape``'s real edge, not a shrunk approximation of it. Also faster: interior points project through a self-contained fast numpy mesh-ray-test (`_mesh_ray_project()`, the same math as `ocad.py`'s own `psos_tri()`) instead of `surface_cap_solid()`'s per-point exact-OCCT `psos()` calls.

HOW THE "NO OFFSET NEEDED" PART ACTUALLY WORKS: a scan line's two ENDPOINTS are, by construction, exactly where that line crosses `shape``'s own TRUE boundary loop -- so instead of ray-casting them (the step that forced every earlier version of this idea to shrink the fill region inward first, to avoid a grazing near-miss right at the edge), this function looks up which boundary-loop EDGE each crossing fell on and linearly interpolates the exact 3D point along that real mesh edge directly. No ray-cast, no miss possible, no margin needed. Only the INTERIOR points of each line (always strictly inside the true footprint, by construction, since they sit between two true-boundary crossings) go through `_mesh_ray_project()` -- and being safely interior, they're not at any real risk of missing the mesh either.

```
cap = mesh_cap_ribbon(sol, [0, 0, 1], side="top", plane_point=[0, 0, 15])
show(cap)
```

``plane_point``: same convention as `shape_outline_raster()`'s own -- where the FLAT rail of every ring sits; must be a height/position that actually passes through ``shape`` (see `mesh_cap_solid()`'s own docstring for the failure this causes otherwise -- guarded against here too, see the `ValueError` below). Defaults to ``shape``'s own bounding-box center.

``n_lines`/`points_per_line`/`angle``: same meaning as `a_lines_sec()`'s own ``n`/`s`/`theta`` (ocad.py) -- scan-line count, points per line, and hatch angle in degrees.

``angle_tol`/`tolerance`/`weld_tol``: same meaning as `mesh_boundary_loop()`'s own (mesh classification, tessellation tolerance, and vertex-welding tolerance respectively). ``angle_tol`` defaults to 80 here (matching `mesh_split_surfaces()`'s own default, NOT `mesh_split_triangles()`'s plain-hemisphere 90) for the same reason: on a genuinely organic/curved ``shape``, a 90 split leaves a band of near-vertical triangles right at the true edge whose classification flips on ordinary floating-point noise, each becoming its own tiny spurious boundary loop (CONFIRMED, a real case this session: 60 noise fragments of 2-14 points alongside the one real 87-point outline) -- ``angle_tol=80`` excludes that band at the source, before it can ever fragment into noise loops at all.

``min_hole_fraction``: a second, independent safety net for whatever noise makes it through anyway (a sufficiently complex shape can still produce a few stray loops even at `angle_tol=80`). `mesh_boundary_loop()` can return many loops; only the single LARGEST (by point count) is ever treated as ``shape``'s real outer edge, but every OTHER loop still needs a decision -- treat it as a genuine hole (route the fill around it), or ignore it as noise. A real hole (a washer's own bore) is a substantial fraction of the outer loop's own 2D area; a stray sliver triangle is not. Any non-outer loop whose area is under ``min_hole_fraction`` (2% by default) of the outer loop's own area is dropped outright rather than treated as a hole -- lower this if ``shape`` has a real hole that's smaller than 2% of its own outline (rare), raise it if noise loops are still large enough to sneak past the default.

``ruled`/`tol``: passed straight through to `solid_from_closed_rings()`.

BRANCH-AWARE, like `solid_from_mesh_slices(branches=True)` elsewhere in this app: a tapering/pinching footprint (a narrow waist, or two separate islands that merge) is tracked as separate CHAINS of rings via standard scanline connected-component tracking (a span only continues the chain from the row before it if their x-ranges actually overlap), each chain lofted on its own, then all the resulting pieces fused with `union_multi()`. CONFIRMED real failure mode this fixes: feeding every row's rings into ONE flat sequence instead lofts straight across wherever the span count changes row to row, producing a localized chaotic zigzag right at that transition even though the rest of the surface lofts cleanly.

Returns one Solid (a single chain's own result directly if only one chain came out at all -- the common case -- otherwise `union_multi()`'s own fused result across every chain).

Raises `ValueError` if ``plane_point`` doesn't actually pass through ``shape`` (every scan point misses it -- see the same diagnostic `surface_cap_solid()` gives), or if no chain ended up with at least 2 usable rings.

mesh_cap_solid()

```
mesh_cap_solid(shape, direction, side='top', plane_point=None, angle_tol=80.0,
               tolerance=None, fuzzy_value=0.001)
```

Exposed to scripts | [backend/brep.py:11887](#) | Used in: —

Builds one genuine closed SOLID for the volume between a flat reference plane and ``shape``'s own top (or bottom) mesh skin -- the alternative to `mesh_split_surfaces()+split()` for when `split()` itself is the fragile step: cutting a solid with a Shell made of thousands of ragged mesh facets can come back ``Null TopoDS_Shape`` even when that Shell renders perfectly cleanly on its own (CONFIRMED real failure mode -- a real user report, this session).

Sidesteps `split()` entirely, and sidesteps any edge/boundary-loop STITCHING too -- exactly the class of tolerance problem that broke the earlier, since-removed `shape_silhouette()` -- by building one tiny watertight solid PER TRIANGLE instead: each classified triangle is paired with its own straight-line projection onto the reference plane along ``direction`` (each of its 3 vertices moved along ``direction`` until it lands on the plane), and the original triangle + its projected twin + the 3 planar quads between their corresponding edges close into one small, trivially-valid wedge solid (`solid_from_faces()` on exactly 5 faces) -- no chaining, no matching endpoints against a neighbor, nothing to mismatch. Every triangle produces an independent, already-closed solid entirely on its own.

Those (typically hundreds to thousands of) tiny solids are then fused into ONE solid via `union_multi()` -- this app's own N-ary BOPAlgo_Builder union, already hardened specifically for fusing many solids that share an exactly-coincident boundary with their neighbors (precisely what adjacent triangles' own wedges do, edge-for-edge along their shared triangle edge) -- rather than `union()`'s plain pairwise chain, which is the documented weak spot for exactly this shape of problem (see `union()`'s own docstring: fusing several already-unioned pieces against more pieces across a large flush/coincident contact area is a confirmed real failure mode there).

```
top_solid = mesh_cap_solid(sol, [0, 0, 1], side="top", plane_point=[0, 0, 40])
trimmed = intersection(sol1, top_solid)
show(trimmed, 'red')
```

``side``: "top" (default -- triangles facing ``direction``, within ``angle_tol`` of it) or "bottom" (facing away from it) -- same per-triangle classification `mesh_split_surfaces()` uses, including the same equatorial-band exclusion ``angle_tol`` controls (see that function's own docstring for why plain hemisphere sign alone -- `angle_tol=90` -- isn't used: it throws every near-vertical triangle in with the classified side on floating-point noise alone).

``plane_point``: where the reference plane sits along ``direction`` -- defaults to ``shape``'s own bounding-box center, same convention as `shape_outline_raster()`'s own ``plane_point``. Every triangle vertex is projected onto THIS plane (`point - dot(point - plane_point, direction) * direction`) to build its own wedge's flat side.

``tolerance``: to `_mesh()`'s own tessellation tolerance -- the single biggest lever on how long this takes, since it's directly the number of small solids being built and fused. Auto-derived from ``shape``'s own bounding-box diagonal if left `None`, same formula `shape_outline_raster()` uses; loosen it (fewer, larger triangles) if this is slow.

``fuzzy_value``: passed straight through to `union_multi()`.

Returns one Solid: the swept volume between the reference plane and ``shape``'s own top (or bottom) skin, following the true curved surface up to the mesh's own tessellation.

Raises `ValueError` if no triangles are classified for ``side`` at all (``direction`/`side`` combination is probably wrong, or ``angle_tol`` is excluding too much), or whatever `union_multi()/solid_from_faces()` itself raises if a specific wedge is too degenerate to close, or if the final fuse itself fails outright.

CAVEAT: this can build and fuse a genuinely large number of tiny solids -- slower than `mesh_split_surfaces()` for the same ``tolerance``, but trades `split()`'s own fragility for a different (and, on this app's own past experience, more robust) OCCT code path: BOPAlgo_Builder fusing many small solids, rather than BRepAlgoAPI cutting one big solid by a single huge faceted shell. If `union_multi()` itself is slow or comes back visibly fragmented, loosen ``tolerance`` first before trying anything else.

mesh_split_surfaces()

```
mesh_split_surfaces(shape, direction, angle_tol=80.0, tolerance=None, sew_tol=0.0001)
```

Exposed to scripts | [backend/brep.py:10560](#) | Used in: —

Separates ``shape``'s own tessellated mesh into a "top" set of triangles (facing along ``direction``, within ``angle_tol`` degrees of it) and a "bottom" set (facing away from it, within ``angle_tol`` of ``-direction``), then sews EACH set into its own real B-rep Shell -- ready to hand straight to `split()` as ``bisect_by`` (a curved Face/ Shell, exactly what its own docstring's wavy-surface example takes).

This is the fix for `faces_facing()`'s own "can't combine to 1 face with `shell_from_faces`, there are always some holes in it" problem: `faces_facing()` classifies whole B-rep Faces by their overall normal, so any face that's genuinely near-vertical (or just large and gently curved across the equator) can't be cleanly called "top" or "bottom" at all, and gets dropped -- leaving a gap right where the two halves should meet. A triangle from `to_mesh()` is small and (for a smooth-enough tessellation) very nearly flat, so classifying triangle-by-triangle instead follows the TRUE top/bottom split almost exactly along the shape's own silhouette line.

``angle_tol`` (degrees, default 80) is why this doesn't just repeat `faces_facing()`'s own failure mode one level down: a plain "dot product > 0" hemisphere split (`angle_tol=90`) throws every near- VERTICAL triangle -- the whole equatorial side-wall band on a shape like a mouse body -- into "bottom" on sign alone, since their true dot product is only ever a hair positive or

negative either way from ordinary tessellation noise. That doesn't leave a hole (every triangle still lands SOMEWHERE), but it does silently hand you a "bottom" shell that isn't the shape's actual underside at all -- it's the underside PLUS a chunk of its side wall, wrapped part way up, which is a much harder cutting tool for `split()` to reason about and a real, confirmed cause of `split()` raising "Null TopoDS_Shape object" instead of a clean cut. Requiring within ``angle_tol`` of straight up/down instead EXCLUDES that whole ambiguous equatorial band from both sets -- "top" is only the genuine top skin, "bottom" only the genuine bottom skin, and the thin excluded ring in between is simply not part of either returned Shell. That's fine for `split()`: the cutting surface only needs to fully cross whatever you're cutting, not literally touch every last triangle of ``shape`` itself.

```
halves = mesh_split_surfaces(sol, [0, 0, 1])
trimmed = split(sol1, halves["top"], keep="bottom")
trimmed = split(trimmed, halves["bottom"], keep="top")
show(trimmed, 'red')
```

``angle_tol``: lower this (e.g. 60) to exclude a wider equatorial band -- worth trying if a shape's "sides" aren't near-vertical but more gently sloped, so 80 still lets some of them leak into one set. Raise it towards 90 only if you've confirmed the shape has no real near-vertical wall at all (rare for an organic body).

``tolerance``: to `_mesh()`'s own tessellation tolerance -- smaller = more, smaller triangles = a split that hugs ``shape``'s real curved surface more closely, at the cost of a heavier cutting Shell (see the caveat below). Auto-derived from ``shape``'s own bounding-box diagonal if left None, same formula `shape_outline_raster()` uses.

``sew_tol``: the tolerance `_sew_faces_to_shell()` sews adjacent triangle edges at. Looser than that function's own 1e-6 default on purpose -- to `_mesh()`'s per-face independent tessellation (see `smooth_mesh()`'s own docstring) means triangles from adjacent original B-rep faces can be coincident-but-not-exactly-equal at the seam; 1e-6 is tight enough to leave some of those unsewn. Loosen further (e.g. 1e-3) if `split()` ends up seeing a Shell with unexpected extra free edges.

Returns {"top": Shell, "bottom": Shell, "top_triangles": int, "bottom_triangles": int}.

Raises `ValueError` if either side comes back with zero triangles (``direction`` is probably wrong -- try flipping its sign, picking something closer to perpendicular to ``shape``'s own flattest side, or lowering ``angle_tol`` if it's excluding too much).

CAVEAT -- same fragility class already documented on `union()`'s own docstring: each returned Shell is a genuine FACETED surface, one small planar Face per triangle (typically several hundred to a few thousand for an organic shape at a reasonable tolerance). OCCT's boolean/split machinery handles this fine in the ordinary case, but cutting against thousands of tiny facets is real, non-trivial work for it -- if `split()` is slow or still raises here, try a looser ``tolerance`` first (fewer, larger triangles) before concluding the whole approach doesn't work on a given shape.

mesh_split_triangles()

```
mesh_split_triangles(shape, direction, angle_tol=90.0, tolerance=None)
```

Exposed to scripts | [backend/brep.py:10713](#) | Used in: —

The lightweight, pure-mesh sibling of `mesh_split_surfaces()`: the exact same per-triangle top/bottom classification (facing along ``direction``, within ``angle_tol`` degrees of it -- see that function's own docstring for what ``angle_tol`` does and why it matters), but returns each side as a flat triangle list `[[p0,p1,p2], ...]` -- to `_triangles()`'s own output convention, and exactly what `ocad.py`'s `psos_tri()`/`plos_tri()` and similar low-level mesh-intersection tools expect -- instead of building a real sewn B-rep Shell.

Reach for THIS one, not `mesh_split_surfaces()`, whenever the classified triangles are headed straight into one of those mesh-native functions rather than into `split()`/a boolean. `mesh_split_surfaces()` pays real, nontrivial OCCT cost building one genuine Face per triangle (Edge.make_line x3 + a Wire + a Face) and sewing all of them into a Shell (BRepBuilderAPI_Sewing) -- CONFIRMED, real numbers, this session: for a "classify triangles, hand them straight to `psos_tri()`" pipeline specifically (to `_triangles(mesh_split_surfaces(sol, d, 90))["top"]`), that whole Face-building/sewing pass was pure wasted overhead, since `_triangles()` immediately RE-TESELLATES the just-built Shell back into essentially the same triangle data `mesh_split_surfaces()` started from a moment earlier -- a full round trip through real B-rep construction and OCCT sewing for data that was already sitting in plain mesh form the whole time, and the single biggest cost in that pipeline. This function stops at the mesh: classify, then hand the matching triangles straight back as plain point

data, no Face/Wire/Shell/sewing involved at all.

```
top_tris = mesh_split_triangles(sol, [0, 0, 1])["top"]
curved = o.psos_tri(top_tris, la, [0, 0, 1], 1e7, 0)
```

A second, independent lever worth knowing about for the SAME pipeline: ``tolerance`` here only controls how finely ``shape`` is tessellated into the TARGET triangles `psos_tri()/psos()` ray-test against -- it has nothing to do with how smooth the final result looks, since that's entirely controlled by however many scan lines/points-per-line you sample (`a_lines_sec()`'s own ``n`/s``) and fed into `solid_from_closed_rings()` afterward. `psos_tri()` is a per-QUERY-POINT loop that matrix-inverts across every target triangle each time (see its own source) -- its cost scales directly with triangle count, so a deliberately LOOSER ``tolerance`` here (fewer, bigger triangles) is close to a free speedup: it barely changes which point on ``shape``'s surface each ray actually hits, but can cut the triangle count, and therefore `psos_tri()`'s own runtime, substantially.

Same ``angle_tol`` meaning as `mesh_split_surfaces()`'s own; defaults to 90 here (a plain hemisphere split, no excluded equatorial band) since the usual reason to exclude one -- handing a faceted Shell straight to `split()` -- doesn't apply to a flat triangle list at all. Pass a lower value if you specifically want that exclusion for some other reason.

Returns {"top": [[p0,p1,p2],...], "bottom": [[p0,p1,p2],...], "top_triangles": int, "bottom_triangles": int} -- same keys as `mesh_split_surfaces()`, just "top"/"bottom" are plain triangle lists instead of Shells.

Raises `ValueError` under the same conditions `mesh_split_surfaces()` does (see its own docstring).

o 3dc tri()

```
o_3dc_tri(l1, s1, r=1, o=0.02, option=0)
```

Exposed to scripts | backend/brep.py:2403 | Used in: hook.py

`o_3d_tri(l1, s1, r)`, with the same self-intersection cleanup pass `ocad.py`'s own `o_3dc()` applies on top of `o_3d()`'s raw output -- `ocad.py` grew that "clean" pass for the triangulation_type-based `o_3d()/o_3dc()` pair, but never grew a matching one for their `_tri` siblings (`o_3d_tri()`), which takes an already-triangulated ``s1`` -- e.g. `to_triangles()`'s own output -- instead of a `triangulation_type` to triangulate one itself). Offsetting a line by ``r`` can leave small self-crossing loops right where it crosses itself at a tight/concave stretch of ``s1`` (wherever ``r`` exceeds the local radius of curvature) -- this flattens each such loop down to a single point, the same fix `o_3dc()` makes for the non- `_tri` case. See `o_3dc()`'s own docstring for what ``o`` (the self- intersection tolerance, not to be confused with this file's own module-level ``o`` -- `ocad.py`'s opt-in bridge alias -- this is a plain local parameter name collision, harmless since Python resolves the parameter first inside this function) and ``option`` mean; ``triangulation_type`/`outside`` aren't here at all since `o_3d_tri()` itself has neither (a pre-triangulated mesh needs no triangulation strategy to pick).

No `build123d` dependency (same as `o_3d_tri()` itself) -- this is pure point-list/mesh math, so it reuses `ocad.py`'s own already- correct helpers directly (`seg()/s_int1_3d_with_list()/vcost2c()`, etc) rather than reimplementing them.

REAL BUG FIX (real user report -- an offset curve traced across a fillet where the mesh has two nearby surfaces close together, e.g. right where a rounded corner meets a thin remaining wall, or along a thin curled feature like a hook blade): plain `o_3d_tri()` finds each line point's surface normal via `i_p_n_tri()->ipt_tri()`, and projects the offset point back onto the surface via `plos_v_2_tri()` -- BOTH of these match/ray-cast against the WHOLE triangle soup ``s1`` at once, independently per point, with no awareness of which "sheet" of the surface is actually the correct local one. Right where two surfaces of the mesh pass close together (a thin wall, a tight fillet, a curled-back blade), the wrong-sheet triangle can be genuinely the closest match/ray-hit for some points even though it belongs to the opposite side of the surface -- flipping their normal and offset direction relative to the rest of the curve, and in the later `plos_v_2_tri()` projection step can also make the offset point snap onto the wrong sheet outright. Both are the same underlying issue: no "same side" check.

Fixed here (not in plain `o_3d_tri()/plos_v_2_tri()` -- bypassed below in favor of this file's own reimplementation) in three passes:

Pass 1: compute each point's normal the old (global, un- restricted) way, exactly as `o_3d_tri()` does, then run neighbor-to-neighbor sign correction along the curve (a genuine surface normal only rotates gradually point-to-point along a reasonably sampled curve -- see `sample_curve()`'s own docstring for what "reasonably sampled" means -- so an outright >90-degree flip vs. the already-corrected previous point is always the wrong- triangle-match artifact, never real curvature).

This gives a rough but globally-consistent reference DIRECTION per point, even at points where the underlying triangle match was wrong.

Pass 2: re-match each point's normal AGAIN, this time only against a small LOCAL neighborhood of `s1`'s triangles (the nearest few dozen by centroid distance to that point) that are ALSO still roughly aligned with that point's Pass-1 reference

```
direction (dot product > 0). Orientation alone turned out not to
```

be enough on its own (a real follow-up report: offsetting to the *other* side, i.e. negative `r`, could still jump to a distant, unrelated patch of the mesh that happens to face the same rough direction as the correct local one -- same-side is necessary but not sufficient, it also has to be nearby) -- combining "roughly the same facing direction" with "one of the nearest few dozen triangles to this point" pins the match down to the correct local patch even on a mesh where the same orientation recurs in several disconnected places. The same local+same-side restriction is then reused when projecting the offset points back onto the surface (`plos_v_2_tri()`), so that ray-cast can't tunnel through to the opposite sheet or jump to a distant patch either -- anchored on each point's own original (pre-offset) position, since that position is already known to sit on the correct patch.

Pass 3 safety net: even a local+same-side-restricted candidate set can still legitimately contain a real but too-distant triangle in a coarsely-tessellated region (a real case: on one real hook-shaped mesh, the nearest 80 triangles by centroid to a point near a tight bend spanned up to 4.5mm away -- wider than a 1.25mm offset -- because only ~24 triangles actually existed within 3mm there). So after projecting, each point's final distance back to its own original `l1[i]` is checked against the requested offset distance `r` -- a correct projection should land close to `r` away; if it's landed much farther (>1.8x), the projection is rejected and the un-projected offset point is kept instead, the same safe fallback `plos_v_2_tri()` itself already uses when it finds no hit at all.

```
l1: line to offset -- a plain 3D point list, e.g.
    sample_curve()'s own output
s1: a pre-triangulated mesh, e.g. to_triangles()'s own output
r: offset distance
```

planarize ring()

```
planarize_ring(ring)
```

Exposed to scripts | backend/brep.py:4690 | Used in: —

Projects every point of a closed point loop onto the single best-fit plane through it (`centroid + _ring_best_fit_normal()`'s normal), returning a NEW ring that is exactly planar by construction -- each point is nudged only along the normal direction, so its position WITHIN the plane (and hence the ring's overall shape) is otherwise preserved.

Exists so `mixed_polygon()` (needs an exactly planar input) can succeed on every station of a wedge grid, including ones a raw B-rep/mesh pipeline left slightly non-planar (e.g. per-station points independently computed via offset-along-surface, which can drift a fraction of a unit out of plane station to station) -- replacing `solid_from_wedge_grid()`'s old per-row fallback to `cap_wire()` (a filling-surface approximation that can bulge slightly and doesn't align as predictably with its neighbors) with an exact flat wedge on every row instead. A row that's already planar is unaffected (projection distance ~0), so this is safe to apply unconditionally to a whole grid, not just the rows that would otherwise fail.

shape_outline_raster()

```
shape_outline_raster(shape, direction, grid_res=None, tolerance=None, plane_point=None)
```

Exposed to scripts | backend/brep.py:10415 | Used in: —

Flattens `shape`'s own silhouette into a real, closed 2D outline -- the actual outer-boundary points (and any hole loops), plus an already-built, already-positioned Face -- as seen looking straight along `direction`. Same underlying technique as `slider_from_faces()`'s own default rasterization path (see its docstring for the full rationale), just applied to a whole shape's outer boundary instead of a picked-faces footprint: `to_mesh()` the shape, flatten every vertex into the projection plane's own (u, v) coordinates, rasterize every triangle onto a fine boolean occupancy grid (pure numpy), and trace the grid's OWN boundary back into a real 2D polygon.

This deliberately avoids the OTHER natural way to do this -- stitching together individual HLR (Shape.project_to_viewport()) edge segments by matching up their endpoints within some tolerance -- because that approach has a real, confirmed failure mode on organic/complex geometry: too tight a tolerance and real gaps never close; too loose and unrelated nearby points get welded together, producing a loop that closes but CROSSES ITSELF (CONFIRMED directly, this app: a wedge-grid/loft/boolean-difference organic body had no single tolerance that both closed AND stayed a simple polygon). A rasterized grid's own boundary trace has NO such failure mode AT ALL: it's built entirely from exact integer grid- corner topology (_trace_grid_boundary_loops()), and a boolean raster's own boundary can never self-intersect by construction -- there's no endpoint-matching tolerance to get wrong in either direction. The tradeoff is precision (bounded by `grid\_res`, a real right-angle staircase on anything not grid-aligned) rather than exact analytic geometry -- Douglas-Peucker simplification (same _dp_simplify_closed_loop() slider_from_faces() already uses) cleans that staircase back down to a clean outline within about one grid cell of the true boundary before this returns anything.

```
top = shape_outline_raster(mouse, [0, 0, 1])
show(top["face"], c="red") # already a real, positioned Face
show(mouse, alpha=0.2)
```

Returns a dict:

```
{ "outer": [[u, v], ...], # outer loop, FLAT 2D points in
                               # the projection plane's own
                               # (u, v) frame (same frame
                               # `face` below is built on)
  "holes": [[[u, v], ...], ...], # any hole loop(s), same frame
  "face": <a real build123d Face> } # "outer" already capped
                                   # into a Face AND already
                                   # positioned/oriented at the
                                   # real plane in 3D (via
                                   # build123d's own native
                                   # Plane multiplication, the
                                   # exact same idiom
                                   # slider_from_faces() already
                                   # relies on -- no separate
                                   # placement step needed)
```

`plane\_point`: where the projection plane sits along `direction` -- defaults (None) to `shape`'s own bounding-box center (so the plane passes straight through the middle of the shape, same as slider_from_faces()'s own default); pass an explicit [x, y, z] to place it elsewhere (e.g. `plane\_point=[0, 0, 40]` to sit 40 units up along Z).

`grid\_res`/`tolerance`: same meaning as slider_from_faces()'s own `grid\_res` (rasterization cell size, auto-sized to the shape's own bounding-box diagonal if left None) and its internal mesh tessellation tolerance (`to\_mesh()`'s own `tolerance`, auto-derived from the shape's diagonal the same way if left None) -- see that function's own docstring for the full tradeoffs of each.

Raises ValueError if the rasterized footprint comes back empty or degenerate (near-zero area) -- try a different `direction`, or a smaller `grid\_res` if the shape is very thin along the view axis.

slice mesh to rings()

```
slice_mesh_to_rings(V, F, axis=(0, 0, 1), n=20, heights=None, tol=1e-07)
```

Exposed to scripts | backend/brep.py:1350 | Used in: —

Slices an arbitrary triangle mesh [V, F] (the SAME [vertex list, face-index-triple list] convention to_mesh()/to_triangles() use) with a stack of planes perpendicular to `axis`, and returns one closed cross-section point-loop ("ring") per plane -- plain numpy point math, no B-rep involved yet. This is the first half of "contour stacking": turn an arbitrary mesh into a stack of ring outlines, then hand those rings to solid_from_mesh_slices() (or build your own pipeline with polygon()/loft()/surface_from_closed_rings()) to fit a genuine smooth B-rep solid through them.

`axis`: the slicing direction, any vector (not necessarily unit length, doesn't need to be X/Y/Z -- an arbitrary tilted axis works too, see the third example below).

``n``: number of evenly-spaced planes spanning the mesh's own extent along ``axis`` (ignored if ``heights`` is given explicitly).
``heights``: an explicit list of signed distances along ``axis`` (measured from the origin) to slice at instead.

``tol``: vertices within ``tol`` of a given plane are treated as lying exactly ON it. This matters more than it looks -- a mesh with any real symmetry (e.g. slicing exactly through a model's own mirror-plane, or through a flat face that happens to lie in one of your slicing planes) can put a whole ring of vertices exactly on a plane at once; without this, every triangle touching that ring would report "no crossing" on both its edges and the slice would come back empty even though the mesh clearly has geometry right there.

Returns a list of (height, ring, extra_loops) tuples, one per plane, in the same order as ``heights``/the generated ``n`` planes:
height: the signed distance along ``axis`` this plane sits at. ring: a list of 3D points with `ring[0] == ring[-1]` (closed), or

None if this plane missed the mesh entirely, or the raw crossing segments didn't stitch into any closed loop at all (e.g. the mesh has a hole/isn't watertight right at this height).

extra_loops: 0 normally. If the plane crossed the mesh at more

than one separate place (e.g. slicing through two separate fingers of a forked shape), only the LARGEST loop (by point count) is returned as ``ring``, and extra_loops reports how many smaller ones were dropped -- `loft()/surface_from_closed_rings()` each need exactly one profile per station; a genuinely branching cross-section needs its branches sliced and lofted separately, using their own height lists. If the dropped loop is substantial (at least 15% as many points as the one kept -- real geometry, not just a stray mesh sliver), this ALSO prints a loud console warning naming the height and both loop sizes -- confirmed necessary directly: a real union-of-spheres mesh had a genuine second lobe silently vanish from the reconstructed solid with no indication anything was wrong until the result was visually inspected and a chunk of it was missing.

```
v, f = to_mesh(some_shape)          # or your own raw mesh
for h, ring, extra in slice_mesh_to_rings(v, f, n=30):
    if ring is None:
        continue                    # this height missed the mesh
    if extra:
        print(f"height {h}: dropped {extra} smaller loop(s)")
    # ring is ready for polygon()/translate()/loft(), or stack
    # every ring together for surface_from_closed_rings()
```

UNVERIFIED against a live install in the sense that `solid_from_mesh_slices()` below (which calls into real OCCT) hasn't been exercised on an actual build123d install (see module docstring) -- but THIS function is pure numpy triangle/plane math with no B-rep dependency at all, and has been directly unit-tested standalone (axis-aligned box, a subdivided icosphere checked against its own analytic $r = \sqrt{R^2 - h^2}$ at every height including the exact equatorial degenerate case above, two disjoint solids, and a tilted non-axis-aligned slicing direction).

smooth_mesh()

```
smooth_mesh(V, F, iterations=10, lam=0.5, mu=-0.53, boundary_fixed=True, weld_tol=1e-06)
```

Exposed to scripts | [backend/brep.py:12430](#) | Used in: —

Laplacian/Taubin smoothing on an arbitrary triangle mesh [V, F] -- the missing "clean up the mesh" step that sits between `to_mesh()` and `solid_from_mesh_slices()`'s own mesh->B-rep round trip:

```
v, f = to_mesh(rough_shape, tolerance=0.05)
v2 = smooth_mesh(v, f, iterations=15)
smooth = solid_from_mesh_slices(v2, f, axis=[0, 0, 1], n=40)
show(smooth)
```

Each iteration moves every vertex toward the average of its direct edge-neighbors (a uniform-weight discrete Laplacian, built once from F's own edge list): $V += \text{lam} * (\text{mean}(\text{neighbors}) - V)$.

Plain Laplacian smoothing shrinks the mesh a little every pass (a noisy sphere slowly implodes toward its centroid as it smooths). ``mu`` (default -0.53, paired with the default `lam=0.5`) alternates that shrink pass with an oppositely-signed "inflate" pass every iteration, cancelling the volume loss while still removing high-frequency noise -- this is the standard Taubin (1995) `lambda/mu` algorithm, and is almost always what you want; pass `mu=None` to fall back to plain (shrinking) Laplacian smoothing instead.

``boundary_fixed=True`` (default) leaves TRUE open-mesh boundary vertices (an edge used by only 1 triangle, after welding -- see ``weld_tol`` below) untouched, so smoothing an open patch doesn't drag its own rim inward. Has no effect on a genuinely closed/ watertight mesh, which has no such edges. Set False to smooth boundary vertices too.

``weld_tol`` (default 1e-6): vertices within this distance of each other are treated as ONE topological node when building the adjacency graph and detecting boundaries, THEN smoothed/moved together so they stay exactly coincident afterwards (positions are only ever averaged among genuinely-distinct welded nodes, never across the weld itself) -- pass None to disable and use raw vertex INDICES instead (only correct if V/F are already known to share one index per unique position).

This matters because `to_mesh()` (this app's own B-rep tessellation bridge) does NOT return an index-welded mesh: each Face is tessellated independently, so two triangles from ADJACENT faces that share a real 3D edge get their own separate, merely COINCIDENT copies of that edge's vertices, not the same indices. CONFIRMED (this app, a real build123d install) to matter in practice, not just in theory: without welding first, every one of those coincident-but-distinct-index seam vertices reads as "used by only 1 triangle" and gets FROZEN by `boundary_fixed=True`, while the genuinely-interior vertices around it keep moving -- tearing the mesh open right along every face seam instead of smoothing it, which was enough to make `solid_from_mesh_slices()` fail outright ("need at least 2 valid rings") on a mesh that sliced fine before smoothing. Welding first collapses each such duplicate group back into one shared node before any adjacency/boundary decision is made, so a closed shape reads as closed (0 boundary vertices, not "every seam vertex") and no seam can crack open, by construction -- reproduced directly (a synthetic mesh with real coincident-but-separately-indexed seam duplicates): the un-welded version misread 100% of duplicated vertices as boundary and left the mesh completely unsmoothed (or torn, on a partially-duplicated mesh), while the welded version correctly read 0 as boundary, smoothed normally (RMS noise cut ~64%, matching the plain-index numbers below), and left seams at a floating-point-exact 0 gap afterwards.

Returns a NEW vertex list V2 in the same `[[x, y, z], ...]` convention and LENGTH as V -- F/topology is unchanged, only positions move (welded duplicates move together, staying coincident, rather than being merged away/removed).

Pure numpy, no OCCT call in this function at all, so it works equally on a `to_mesh()`-tessellated B-rep shape or on any other [V, F] mesh in this app's own convention (`repair_mesh()`/ `offset_mesh()`/ `mcm()` in `ocad.py`, `polyhedron()`'s own input).

Verified directly (standalone, outside this app) on a synthetic 642-vertex noisy icosphere (noise added, then smoothed): after 15 Taubin iterations (`lam=0.5`, `mu=-0.53`), RMS radius deviation from the ideal sphere dropped ~64% while mean radius stayed within 0.3% of the original (-0.24% net) -- 15 iterations of plain Laplacian (`mu=None`) over the same noise removed ~81% of the RMS deviation but shrank the mean radius by 8.4%, the expected plain-Laplacian volume-loss trade-off. A vectorized (`np.add.at` scatter-sum) implementation was cross-checked element-for-element against a naive per-vertex Python loop (identical to 1e-9) and measured ~15x faster on that same mesh; also verified an open hemisphere patch left its 44 free-boundary vertices at exactly zero displacement with `boundary_fixed=True` (default) while interior vertices still moved/smoothed normally, and (see ``weld_tol`` above) that a mesh with real coincident-but-separately-indexed seam vertices is welded, smoothed, and kept watertight correctly with the default `weld_tol=1e-6`.

solid_from_closed_rings()

```
solid_from_closed_rings(rows, ruled=False, tol=1e-06)
```

Exposed to scripts | backend/brep.py:5217 | Used in: —

A real closed B-rep SOLID, built directly from an ordered list of closed cross-section rings -- the one-line version of the standard 3-step combination:

```
b_1 = surface_from_closed_rings(b, ruled=True)
c = cap_wire(b[0])
```

```
d = cap_wire(b[-1])
b_2 = solid_from_faces(b_1, c, d)

b_2 = solid_from_closed_rings(b, ruled=True) # same result
```

`surface_from_closed_rings()` (see its own docstring) builds the curved wall, open at both ends -- this caps it into a solid the same way that function's own docstring shows by hand: `cap_wire()` on the FIRST/LAST ROW'S OWN RAW POINT DATA, not on `wall.edges()/wall.wires()` pulled back off the already-built wall Shell. That distinction matters: a ``ruled=True`` wall built from many stations can produce far more than the 2 boundary wires you'd expect (internal per-band seams aren't always recognized as shared/interior edges once sewn), so picking the end boundary back off the wall itself is unreliable at real station counts. Capping straight from the ORIGINAL row data -- which is exactly the real boundary curve either way -- sidesteps that failure mode entirely.

``ruled`/`tol`` are passed straight through to `surface_from_closed_rings()` unchanged -- see its own docstring: ``ruled=False`` (default, matching that function's own default) is the smoothed single-surface mode; ``ruled=True`` facets straight between stations instead, useful when a sculpted/high-curvature ring stack makes the smoothed fit struggle, fail outright, or (a real case hit in practice) fail to TESSELLATE for on-screen preview even though the underlying B-rep may technically still be valid -- ``ruled=True`` is the safer default to reach for on a genuinely organic/non-primitive ring stack; try ``ruled=False`` afterward if you want fewer surface seams in an exported STEP and the shape isn't too extreme.

Raises whatever `surface_from_closed_rings()/cap_wire()/solid_from_faces()` themselves raise -- same failure modes as calling them by hand.

solid_from_faces()

```
solid_from_faces(*faces, tol=1e-06)
```

Exposed to scripts | backend/brep.py:12747 | Used in: —

Sews several B-rep Faces that TOGETHER bound one closed, watertight volume into a real Solid -- build123d's own native `Shell(faces)` (sews the faces together along their shared edges) + `Solid(shell)` (`ShapeFix_Solid().SolidFromShell()` under the hood, which both finishes closing the shell into a solid and repairs a shell that's almost-but-not-quite perfectly watertight).

This is the real B-rep answer to "I have several curved surface patches, how do I turn the volume they enclose into one solid" -- the case `loft()` specifically CANNOT handle: `loft()` needs every cross-section to already be a flat/planar Face (it calls `section.faces()` then uses each face's own `outer_wire()` -- a Face can only be built from a wire that's planar in the first place), so a part that genuinely bends/twists in 3D (most of its intermediate cross-sections non-planar) can't be lofted through directly no matter how many sections you feed it. `surface_from_points_grid()` has no such restriction (a true 3D NURBS fit through a whole point grid, curvature and all) -- fit the curved wall(s) with that, then sew them to their flat end caps here:

```
wall = surface_from_points_grid(surf) # the bent tube's own curved wall
cap_a = mixed_polygon(a) # flat profile at one end
cap_b = mixed_polygon(b) # flat profile at the other end
solid = solid_from_faces(wall, cap_a, cap_b)
show(solid)
```

``tol`` (default 1e-6): the sewing tolerance -- see `_sew_faces_to_shell()` for when to loosen this (a large face selection, e.g. `faces_facing()`'s output, built up through several prior `loft/boolean` steps).

Raises whatever build123d itself raises if the given faces don't actually close up into a single watertight volume (a gap, an overlap, mismatched edges) -- try a looser ``tol`` first if that seems like a real possibility given how ``faces`` was built.

Sews via OCCT's own `BRepBuilderAPI_Sewing` directly (through OCP, build123d's own underlying binding) rather than build123d's own `Shell(list_of_faces)` Python constructor -- on at least one build123d version that constructor raised "Unable to create Shell, invalid input type" on a perfectly valid list of Faces (a curved wall Face plus two `cap_wire()`-built end caps), even though each individual Face rendered/behaved fine on its own; sewing at the OCCT level first and only wrapping the ALREADY-CORRECTLY-TYPED single result (a Shell, or occasionally a lone Face/Compound of shells if sewing didn't fully close things up) sidesteps whatever that mismatch was.

solid from mesh slices()

```
solid_from_mesh_slices(V, F, axis=(0, 0, 1), n=20, heights=None, ring_points=80,
cap=True, tol=1e-07, ruled=False, branches=False, significant_ratio=0.15,
overlap_stations=2, match_factor=0.6)
```

Exposed to scripts | backend/brep.py:1658 | Used in: —

Reconstructs a genuinely smooth B-rep solid from an arbitrary triangle mesh [V, F] by contour-stacking: slices the mesh with a stack of parallel planes (`slice_mesh_to_rings()`), resamples every resulting ring to the same point count, then fits ONE smooth NURBS surface through the whole stack (`surface_from_closed_rings()`) and caps both ends (`cap_wire()` + `solid_from_faces()`).

This is the practical way to turn an arbitrary/unstructured mesh -- something with no construction history available at all -- back into real B-rep, by reducing it to the already-solved "structured ring stack -> smooth surface" problem instead of trying to reconstruct faces/edges directly from raw triangle soup. In THIS app specifically, the mesh doesn't have to come from an external file (there's no raw STL/OBJ importer here, only `import_step()`/`import_brep()` for real B-rep formats) -- the practical use is reconstructing/smoothing a mesh you already have from within a script, e.g. `to_triangles()`-tessellating a shape built via `two_solids_intersection()`'s mesh fallback, or an `import_step()` model with coarse faceting, and fitting a smoother native surface back through it:

```
v, f = to_mesh(rough_shape, tolerance=0.05)
smooth = solid_from_mesh_slices(v, f, axis=[0,0,1], n=40)
show(smooth)
```

``axis`/`n`/`heights`/`tol``: same as `slice_mesh_to_rings()`.

``ring_points``: every kept ring is resampled to exactly this many evenly-spaced points before fitting (`surface_from_closed_rings()` requires all rows to match). Higher values track sharper in-ring detail more closely at the cost of a heavier surface fit.

Every ring is also automatically re-wound and re-indexed to line up with its neighbors (see `_align_ring_stack()`'s own comment) -- `slice_mesh_to_rings()` computes each height's ring completely independently, so consecutive rings can come back winding opposite directions and starting at unrelated angular positions; feeding that straight into a lofted surface fit twists it into a self-intersecting mess (confirmed directly, on a plain sphere, the hard way).

``cap``: True (default) closes both ends into a watertight Solid via `cap_wire()`+`solid_from_faces()`; False returns the open wall Shell only (`surface_from_closed_rings()`'s own raw return), e.g. if you plan to cap it some other way yourself, or the two ends should stay open (a tube/pipe shape).

Heights where `slice_mesh_to_rings()` found no ring at all (a gap, or a plane past the mesh's own extent) are skipped with the height noted in the raised/printed warning rather than silently producing a broken stack; at least 2 valid rings are required.

``ruled``: passed straight through to `surface_from_closed_rings()`. False (default) gives the smooth spline surface -- looks better when it works, but CONFIRMED (see below) to be capable of silently self-intersecting, no exception raised at all, when adjacent rings change shape a lot over relatively few stations. True instead connects each ring to the next with straight (faceted, visible seam at every station) patches -- more robust, at the cost of losing the smooth blend between stations. If the smooth default comes out visibly wrong (spiky, tangled, self-intersecting) for your mesh, that -- not a bug report -- is the first thing to try: call again with ``ruled=True``, or increase ``n`` so consecutive stations differ less.

CONFIRMED against a live build123d install, on two real cases: a single sphere (works cleanly either way, `ruled=True/False`), and a real `union()` of 3 overlapping spheres sliced at only `n=10` stations -- there, `ruled=False` (the default) built a technically successful Shell with no error raised at all, that was nonetheless visibly self-intersecting and took a viewer's tessellation over 6 minutes to (fail to) resolve; `ruled=True` on the exact same rings produced a correct, if faceted, watertight solid in well under a second. The rings/alignment themselves were confirmed fine in both cases (see `_align_ring_stack()`'s own comment) -- this is specifically `surface_from_closed_rings()`'s smoothed ThruSections mode overshooting on sections with real shape change between stations, same class of fragility its own docstring already documents.

The two end caps are built directly from the first/last RESAMPLED ring's own points (guaranteed exactly planar by construction, see `_planar_face_from_ring()`'s own comment) rather than from wall's own fitted boundary curve -- an earlier version of this function capped from `wall.edges()[0][-1]` instead (the pattern `surface_from_closed_rings()`'s own docstring shows) and hit a real, confirmed failure: build123d/OCCT rejecting that fitted boundary curve as non-planar even though the ring that produced it was exactly flat, since a multi-station smoothed surface fit doesn't preserve its own end sections' planarity the way a single-ring interpolation does. If ``cap=True`` still fails for some mesh (the cap not sewing cleanly against wall's boundary, for instance), call with ``cap=False`` and cap it yourself after inspecting ``wall``'s own edges via this app's Pick Edges panel.

``branches``: False (default) is everything described above -- one ring per station, fast, but only correct for a mesh whose cross-section never splits into more than one loop at a time. Real meshes routinely DO branch (fingers on a hand, forked mechanical parts, tree-like structures) -- with `branches=False`, `slice_mesh_to_rings()` silently keeps only the largest loop at any height with more than one, which can delete real geometry (see its own docstring; it does at least print a loud warning when this happens rather than staying silent about it). ``branches=True`` instead: keeps every loop at every height, groups the height range into runs of constant (significant) loop count, matches loops across each multi-loop run by nearest-centroid assignment, and builds each resulting thread -- one trunk/branch/finger, whatever its own shape does along the axis -- as its own small independent solid, extended slightly into its neighboring segments so they share real volume, then union()s all of them into one final result. ``significant_ratio``/``overlap_stations``/``match_factor`` tune that process -- see `_build_thread_segments()`'s own comment for exactly what each one guards against; the defaults were chosen against a real branching test mesh (see that comment) and are a reasonable starting point, not required tuning for every mesh. ``cap`` is ignored when `branches=True` (every thread is always fully capped -- union() needs real closed solids, not open shells, to combine).

CONFIRMED (pure point-math, no B-rep needed to test this part) on a real union-of-3-spheres mesh with two genuine branch regions of very different width: correctly tells apart a lobe that TERMINATES at its own tip from one that CONTINUES past the branch, and drops a too-narrow transient branch with a warning rather than trying to loft a degenerate 1-station "solid." The final union() of the resulting per-thread solids is, like the rest of this file's real OCCT calls, UNVERIFIED against a live build123d install.

solid from wedge grid()

```
solid_from_wedge_grid(rows, ruled=False, planarize=True, closed_loop=None,
max_stations=None)
```

Exposed to scripts | [backend/brep.py:4725](#) | Used in: —

Builds a real B-rep SOLID by lofting through an ALREADY-BUILT grid of closed per-station wedge/profile rows -- the same last two lines `loft_fillet()` does internally (`mixed_polygon()` each row into a closed profile, then `loft()` through the whole stack), for when you already have the grid itself rather than a fresh `line_a/line_b/` spine triple for `loft_fillet()` to build it from.

The direct match for `ocad.py`'s own mesh-based `convert_3lines2fillet()+swp_c()` combo's OUTPUT specifically: each row already closed into a wedge (the spine point repeated at both ends of every station -- `swp_c()`'s own auto-close-last-to-first convention, see `loft_fillet()`'s own docstring), and -- if `closed_loop=1` was used there -- the first row already repeated again at the very end for a fillet that wraps all the way around a closed spine. Both of those are exactly what this function expects as-is; it does no reclosing or repeating of its own, unlike `loft_fillet()` (which builds that structure itself from raw `line_a/line_b/spine` curves). Skips straight to the loft, so there's no separate "cap the two ends" step needed the way there is after `surface_from_closed_rings()` -- each station being a CLOSED wedge (not an open ring) is what lets `loft()` close the result into a solid directly.

CLOSED-LOOP SEAM (`rows[0] == rows[-1]`, e.g. a fillet that wraps all the way around a closed spine): a single `loft()` call through the WHOLE stack does NOT reliably close this seamlessly, even when the repeated first/last row is bit-for-bit identical. `BRepOffsetAPI_ThruSections` (`loft()`'s own underlying OCCT engine) treats its section list as a plain start-to-end sequence, not a periodic ring -- it has no notion that the last section is meant to BE the first one again, so it still builds a distinct (if razor-thin) closing segment between them, which shows up as a visible sliver/kink right at that join once the result is subtracted from something. This is the exact same failure `ocad.py`'s own `swp()` has at the MESH level -- `swp_c()` exists specifically to dodge it there, and this function dodges it the same way here: when `closed_loop` is True

(auto-detected from `rows[0]==rows[-1]` if left as `None`), the duplicate final row is dropped and `ONE` `loft()` runs through only the distinct stations (still a single efficient `ThruSections` call, fast regardless of station count), leaving exactly one gap unclosed between the last station and the first. A second, small `loft()` bridges that gap -- but deliberately **OVERLAPS** the main loft by one extra station (`station[-1] -> station[0] -> station[1]`, not just `station[-1] -> station[0]`) rather than merely touching it at a single coincident end face. Two solids that only share a bare face with zero volumetric overlap are exactly the fragile, tolerance-sensitive case for `BRepAlgoAPI_Fuse` that tends to leave slivers; a real shared **VOLUME** (even a thin sliver of one) gives the fuse solid material to work with and is numerically robust. `union_fuzzy()` then merges the two into one seamless closed solid. No single `loft()` call ever sees the same station twice, and only **ONE** boolean op happens regardless of how many stations the grid has, unlike an earlier version of this function that lofted every consecutive pair separately and fused the whole ring with `union_multi()`, which was correct but far too slow on a 100+ station grid (an N-way `BOPAlgo_Builder` fuse over that many small solids, plus N-1 separate `loft()` calls, versus 2 `loft()` calls and 1 fuse here).

```
f3 = o.convert_3lines2fillet(l3, l2, l1, closed_loop=1) # still
# fine to reuse for
# the point math --
# this is a plain
# point grid either
# way, nothing here
# cares which
# pipeline built it
fillet_solid = solid_from_wedge_grid(f3) # rows[0]==rows[-1] ->
# closed_loop auto-
# detected, pairwise-
# loft-and-fuse path
# used automatically

sol = difference(sol, fillet_solid)
```

``ruled`` means the same as `loft()`'s own -- piecewise-straight between stations (`True`) vs a smooth blended loft (`False`, default), same tradeoff/diagnostic use as everywhere else in this file (try `ruled=True` first if a smoothed loft struggles, to check whether the rows themselves are the problem or just the smoothing).

EACH ROW'S FACE: with `planarize=True` (the default), every row is first passed through `planarize_ring()` -- projected onto its own best-fit plane (see that function's docstring) -- BEFORE `mixed_polygon(row)` is tried, so a row that started out a fraction of a unit out of plane (independently-computed offset points drifting station to station, a sculpted/non-spherical seam, etc) still gets an exact flat wedge face instead of failing planarity. This keeps the genuinely **SHARP** pinch corner at the repeated spine point and a consistent flat-faceted look across the **WHOLE** grid, not just the rows that happened to already be planar. `mixed_polygon(row)` still has a final fallback to `cap_wire(row)` (a non-planar-capable interpolating-spline boundary) for the rare row where even the planarized points fail to form a valid simple polygon (e.g. a genuinely self-intersecting projection). Pass `planarize=False` to restore the old behavior (only fall back to `cap_wire()` on rows that fail `mixed_polygon()` as-is, un-projected) if you want to compare.

Raises whatever `loft()` itself raises if the grid is too irregular for a single loft to follow even after that per-row fallback -- same real failure mode as `loft_fillet()` (see its own docstring): a spine/seam that twists tightly relative to the fillet's own width is the usual cause. Thin a very fine-stationed grid down with `pick_key_stations()` first if that happens.

`max_stations`: the direct lever for **SPEED**, not just correctness. A fine-stationed grid (100+ rows, each row built from a fairly resolved wedge -- e.g. `loft_fillet()`'s own `bezier(..., s)` arcs) genuinely costs real time here: `ThruSections` has to weave together every one of those sections, and the resulting solid -- easily thousands of faces for a 100-station x 20-edge grid -- then still has to be fused against the closer piece. That cost is proportional to grid density, not wasted work (this function is already down to 2 `loft()` calls and 1 fuse total, regardless of station count -- see the `closed_loop` branch below). If `max_stations` is set and there are more rows than that, `pick_key_stations()` (see its own docstring) thins the grid down first -- clustering the kept stations where the wedge's own shape is actually changing fastest, rather than blindly every Nth row, so a long near-straight stretch of the seam gets thinned hard while a sharp turn keeps most of its resolution. Always keeps the first and last row, so a `closed_loop` grid's own `rows[0]==rows[-1]` convention survives thinning intact. `None` (default) leaves the grid exactly as given -- opt in explicitly if a render is taking too long.

to_mesh()

```
to_mesh(shape, tolerance=0.1, angular_tolerance=0.3)
```

Exposed to scripts | backend/brep.py:12380 | Used in: —

Tessellates a B-rep shape into a triangle mesh in the SAME [V, F] convention used throughout ocad.py (repair_mesh()/offset_mesh()/mcm() all return this, polyhedron() expects it) -- the bridge for viewing a B-rep result in SanPyCAD's existing mesh viewer, or feeding it into any of ocad.py's own mesh functions, without needing a real CAD program:

```
v, f = to_mesh(some_brep_shape)
fo(f'''polyhedron({v},{f});''')
```

tolerance/angular_tolerance control facet density for this tessellation ONLY -- the underlying shape stays exact B-rep; export_step()/export_brep() ignore these entirely and write the real analytic/NURBS geometry, not this mesh. Smaller tolerance = more triangles = closer to the true curved surface, same idea as ocad.py's own grid/samples_per_edge tradeoffs, just for display rather than for a computed result.

UNVERIFIED against a real build123d install (see module docstring) -- shape.tessellate(tolerance, angular_tolerance) is expected to return (vertices, triangle_index_tuples); if the real API differs slightly (e.g. vertices as build123d Vector objects needing an explicit .to_tuple()/(.X,.Y,.Z) conversion rather than unpacking directly), that conversion is what would need adjusting here.

This is the SAME OCCT/OCF NbNodes fragility class documented on import_step()/_reraise_nbnodes() -- tessellate() (OCCT's BRepMesh_IncrementalMesh under the hood) is actually the MORE common real-world trigger of it than the STEP reader itself: a shape whose import itself succeeded can still hit this the moment it's first displayed, if triangulating one of its faces trips the same null internal handle. Caught and re-raised with the same clearer explanation as import_step()'s for exactly that reason. Called directly, this always fails outright on that bug -- see _to_mesh_lentient() below for the best-effort fallback the Drawing tab's own show()/auto-show path actually uses, which can still display everything EXCEPT whatever specific solid/face triggers it.

to_triangles()

```
to_triangles(shape, tolerance=0.1, angular_tolerance=0.3)
```

Exposed to scripts | backend/brep.py:12681 | Used in: hook.py

Tessellates a B-rep shape straight into a FLAT list of resolved triangles [[p0, p1, p2], ...] (each p a plain [x, y, z]) -- the format ocad.py's own low-level mesh-intersection building blocks expect (two_tri_intersection(a1, b1), triangle_triangle_intersection(), triangulate_solid_openx()'s own output), which is NOT the same as to_mesh()'s [V, F] indexed format (a vertex list plus face-index triples). Passing to_mesh()'s [V, F] tuple directly into one of those functions is a common mistake once `import ocad` is in the picture -- it fails with something like "ValueError: setting an array element with a sequence... inhomogeneous shape", because V and F are two differently-shaped arrays being stuffed into one, not a triangle list at all. Use to_mesh() when you want the compact indexed form (e.g. to feed polyhedron()); use to_triangles() when a function needs each triangle spelled out in full, like this:

```
import ocad as o4
segs = o4.two_tri_intersection(to_triangles(a), to_triangles(b))
loops = o4.contiguous_chains(segs)
```

Projection & Intersection Queries (10)

nlos()

```
nlos(points, surface, tolerance=0.1)
```

Exposed to scripts | backend/brep.py:11136 | Used in: —

B-rep equivalent of ocad.py's own nlos() ("nearest line projection on surface") -- for each point in `points`, finds its TRUE closest point on `surface`. NOT a directional ray-cast -- that's psos() (the same math as ocad's own psos_tri()/psos()) -- this instead projects the query point onto every triangle's own plane, classifies whether that projection lands inside the

triangle or past one of its 3 edges (clamping to the nearest edge segment or vertex when it does), then picks the globally closest candidate across every triangle. Exact port of ocad's own algorithm (same per-triangle case analysis), just vectorized with numpy instead of looped in plain Python.

```
s1 = surface_line_vector(l3, wall_vec) # a simple open grid
l4 = nlos(l1, s1)
```

``points``: the points to project, a plain list of [x,y,z].

``surface``: either a real build123d Shape (tessellated via `to_triangles()`, using ``tolerance``), an already-triangulated flat (M,3,3) list, or a plain point grid -- see `_surface_to_triangles()` for the exact rules (same forms ocad's own `nlos()`/`psos()` accept, and the same order as THIS file's own `psos()` -- points first, surface/target second).

Returns a plain list of [x,y,z] points, one per input point, each the closest point actually found on ``surface``.

normal_vector()

```
normal_vector(sec)
```

Exposed to scripts | backend/brep.py:3600 | Used in: —

The unit normal vector of a planar point list ``sec`` (3+ points) -- exactly 3 points uses a plain cross product; more uses Newell's method (robust for any planar polygon, convex or not, as long as the points are genuinely coplanar).

plos()

```
plos(points, target, direction, dist=None, unidirection=True)
```

Not yet exposed to scripts | backend/brep.py:11117 | Used in: —

Thin convenience alias for `psos()` when you specifically have a flat point list (not a grid) and want that spelled out -- `psos()` already auto-detects a flat list vs. a grid on its own (ocad.py's own `plos()` is, in fact, exactly its own `psos()` called on a 1-row grid -- see its source), so this does nothing `psos()` itself wouldn't already do correctly given a flat ``points`` list; it's only here for the familiar name.

Same arguments, same order, same meaning as THIS file's own `psos()` -- (points, target, direction) -- which is NOT the argument order ocad.py's own `plos()`/`psos()` use ((surface, points, direction), target/surface FIRST there). Keep that in mind porting a script over directly; swap the first two arguments.

plos_v()

```
plos_v(points, target, focal, dist=None, unidirection=True)
```

Not yet exposed to scripts | backend/brep.py:9908 | Used in: —

Flat-list convenience alias for `psos_v()` -- see its own docstring (`psos_v()` already handles a flat ``points`` list on its own; this is only here for the familiar ocad.py name).

plos_v_1()

```
plos_v_1(points, target, focal_line, dist=None, unidirection=True)
```

Not yet exposed to scripts | backend/brep.py:9915 | Used in: —

Flat-list convenience alias for `psos_v_1()` -- see its own docstring (`psos_v_1()` already handles a flat ``points`` list on its own; this is only here for the familiar ocad.py name).

psos()

```
psos(points, target, direction, dist=None, unidirection=True)
```

Exposed to scripts | backend/brep.py:9637 | Used in: —

Projects a raw point list or point grid onto a real target Face or Solid along ``direction`` -- ocad.py's own `psos()` family (`psos`/`psos_v`/`psos_n`/`psos_v_1`/`psos_v_2`/`psos_n_b`, ~10 near-duplicate variants total), collapsed into ONE real B-rep

function instead of ported literally. The originals all triangulate `target` into a mesh first (several different triangulation helpers depending on the shape's topology -- open/closed/donut-like/etc), then do manual ray-triangle intersection math (numpy) to find where each point's projection ray crosses that triangle soup -- real, working code, but the exact kind of mesh-native machinery this app has consistently avoided (see `concave_hull()`'s own drop for the same reasoning). None of that triangulation is needed here: `build123d`'s own `Face.find_intersection_points(axis)` answers "where does this ray hit the real surface" EXACTLY, against the actual analytic/ NURBS geometry, no faceting step at all. For projecting a real curve/Edge instead of raw points, see `project_curve_on_face()` above; for projecting a whole real Face/Shape (not points sampled off one), see `project_face_on_shape()` above -- exact there too, with no point grid or NURBS re-fit involved at all.

``points``: a flat `[[x,y,z], ...]` list, OR a 2D grid (`[[[x,y,z],`

`...], ...]`, a "surface" the same shape `ocad.py`'s own `psos()` ``s3`` argument took) -- either way every point is projected independently and the result comes back in the exact same shape/nesting ``points`` was given in.

``target``: the real Face (or Solid -- all its faces are tried) to

project onto.

``direction``: the projection direction -- either one `[x,y,z]` used

for every point, or a flat list of one `[x,y,z]` PER point (matching ``points`` flattened row-major, same order `psos_v()/psos_v_1()/psos_n()` below build internally) -- covers `ocad.py`'s own `psos_v_2()` (an arbitrary externally-supplied per-point direction list) without a separate function; those three functions are really just this same per-point-direction case with the direction list computed FOR you a particular way (toward/away from a point, from a line, from ``points``'s own surface normal).

``dist``: the maximum projection distance -- a point whose nearest

hit is farther than this, or that never hits ``target`` at all, is left unmoved at its original position (same "don't lose the original points" fallback `ocad.py`'s `psos()` used). None (default) means no distance limit.

``unidirection``: True (default) only projects forward along

``direction``; False also considers hits found going backward (`ocad.py`'s `unidirection=0`). Either way, the single CLOSEST valid hit is used per point.

```
pts = [[x, y, 0] for x in range(-10, 11, 2) for y in range(-10, 11, 2)]
target = sphere(20)
projected = psos(pts, target, [0, 0, -1])
show(target, alpha=0.3)
show(color(points(projected, .3), "cyan"))
```

``target`` doesn't have to be a real Shape -- pass a raw point grid (e.g. `surface_line_vector()`'s own output) and this falls back to a mesh-ray-cast projection against it instead (`_surface_to_triangles()` auto-triangulates it, then the same worker `nlos()` uses for its own triangle math -- `ocad`'s own `psos_tri()`), same "never lose a point" fallback either way. The exact analytic-geometry path above is still used whenever ``target`` IS a real Shape -- only reached for the cases that path can't handle at all.

psos_n()

```
psos_n(points, target, direction=1, dist=None, unidirection=False)
```

Not yet exposed to scripts | `backend/brep.py:9880` | Used in: —

B-rep equivalent of `ocad.py`'s own `psos_n()/psos_n_b()` (collapsed into one function here -- ``unidirection`` picks between them: True matches `psos_n`'s own forward-only search, False -- the default -- matches `psos_n_b`'s bidirectional one) -- projects ``points`` onto ``target`` along EACH point's own local surface normal (`surface_normals()`) rather than one shared

direction.

```
s2 = surface_line_vector(l3, wall_vec)
s3 = psos_n(some_grid, s2)
```

`direction`: 1 (default) or -1, flips which way the normals (and so the projection) point -- same as `surface_normals()`'s own.

`points` MUST be a genuine 2D grid (not a flat list) -- see `surface_normals()`'s own docstring for why.

`target`/`dist`/`unidirection`: same meaning as `psos()`'s own.

psos_v()

```
psos_v(points, target, focal, dist=None, unidirection=True)
```

Not yet exposed to scripts | backend/brep.py:9785 | Used in: —

B-rep equivalent of `ocad.py`'s own `psos_v()` -- like `psos()` above, but instead of one shared direction, each point is projected RADIALLY away from (or toward, depending which side `target` is on) a single `focal` point -- a point light source, rather than parallel rays. Just computes each point's own (point - focal) direction and hands the whole list to `psos()`'s own per-point-direction support.

```
s2 = surface_line_vector(l3, wall_vec)
s3 = psos_v(some_grid, s2, [0, 0, 100])
```

`focal`: the single [x,y,z] every ray radiates from.

`points`/`target`/`dist`/`unidirection`: same meaning as `psos()`'s own.

psos_v_1()

```
psos_v_1(points, target, focal_line, dist=None, unidirection=True)
```

Not yet exposed to scripts | backend/brep.py:9811 | Used in: —

B-rep equivalent of `ocad.py`'s own `psos_v_1()` -- like `psos_v()` above, but the radial source is a LINE instead of a single point (a linear/tube light source): each point's own direction is (point - `closest_point_on(focal_line, point)`).

```
s2 = surface_line_vector(l3, wall_vec)
s3 = psos_v_1(some_grid, s2, [[0, 0, 100], [50, 0, 100]])
```

`focal\_line`: a plain [[x,y,z], ...] polyline (not assumed closed).

`points`/`target`/`dist`/`unidirection`: same meaning as `psos()`'s own.

s_int1()

```
s_int1(points, closed=True)
```

Exposed to scripts | backend/brep.py:9543 | Used in: —

Every point where the segments of a 2D point-list loop cross themselves -- `ocad.py`'s own `s_int1()`, ported: e.g. run this on an inward offset() of a sharp-cornered profile to find where the offset has crossed itself (gone invalid). Checks every pair of non-adjacent segments for a PROPER crossing (both segments' own interior, not just their infinite extensions -- see `i_p2d()` for that unclipped version) and returns the list of actual crossing points. `closed=True` (default) also checks the closing segment from the last point back to the first, same as `polygon()`'s own convention; `closed=False` treats `points` as an open polyline.

Edge / Face / Vertex Utilities (11)

close_faces_to_plane()

```
close_faces_to_plane(faces, plane_point, direction=(0, 0, 1), tol=None)
```

Exposed to scripts | backend/brep.py:7170 | Used in: —

Closes an open patch of Faces into a real solid by extending its own free boundary along `direction` until it hits a flat plane through `plane\_point`, and capping it there -- "project this surface up onto a plane to make a solid" as a real function, the standard way to turn a picked cavity/core surface (e.g. `faces_facing()`'s own output) into a real mold slider/core block:

```
f1 = faces_facing(sol.faces(), [0, 0, 1])
f1_clean = face_islands(f1)[0] # drop any disconnected slivers
slider = close_faces_to_plane(f1_clean, [0, 0, 60]) # cap at z=60
show(slider)
```

`direction` is NOT limited to straight up -- a real slider can pull in any direction, so the CAP PLANE ITSELF is always built perpendicular to `direction` (its normal IS `direction`), passing through `plane\_point`, whatever that direction happens to be:

```
# a slider pulling sideways instead of vertically
slider = close_faces_to_plane(f1_clean, [80, 0, 0], direction=[1, 0, 0])
```

`plane\_point` is any single point that lies on the flat plane you want the cap to sit on -- it does NOT need to be near `faces` itself; the reference plane built internally is automatically sized and re-centered on `faces`' own boundary before use, only `plane\_point`'s position ALONG `direction` (which plane, out of the infinite family of parallel planes through that normal) actually matters.

How it works: finds `faces`' own free boundary edges (every edge owned by exactly ONE of `faces` -- same shared-edge adjacency `face_islands()/faces_within_boundary()` use, so any interior seam between two of `faces` is correctly skipped and only the true outer rim is used), chains them into one closed Wire (`wire_from_edges()` -- pass `tol` through to it if that boundary doesn't chain cleanly on the first try), projects that boundary wire onto the flat target plane along `direction` (`project_curve_on_face()`, exact B-rep projection, no sampling), builds the "skirt" wall as a ruled Face between the original and projected boundary (`Face.make_surface_from_curves()`, the same engine `surface_line_vector()` uses), and caps the far end with a flat Face bounded by the projected (and therefore genuinely planar) wire -- then sews `faces` + skirt + cap into one solid with `solid_from_faces()`.

Raises whatever `wire_from_edges()/project_curve_on_face()/solid_from_faces()` themselves raise -- most commonly from `wire_from_edges()` if `faces`' free edges don't chain into exactly one closed loop (a real gap in the selection, or `faces` covering more than one disconnected patch -- run `face_islands()` first and feed it just one group, as in the example above).

edges_in_box()

```
edges_in_box(edges, box, mode='inside', tolerance=1e-06)
```

Exposed to scripts | [backend/brep.py:6651](#) | Used in: —

Picks edges by drawing a probe box around/through them, instead of reading indices off `label_edges()`'s numbers -- build, position, and size any shape (almost always just `box([dx, dy, dz])`, `translate()`'d/ `rotate()`'d into place) around the region you want, pass it in as `box`, and get back exactly the edges that fall inside or cross it. Only `box`'s own `bounding_box()` is used -- its actual geometry never matters, so a plain axis-aligned `box()` is normally enough even to isolate edges on a tilted/curved shape.

`mode`: "inside" -- (default) only edges FULLY CONTAINED in the probe

box's extent -- draw a box that ENCLOSES just the edges you want and nothing else.

"crossing" -- edges whose own bounding box overlaps the probe box

AT ALL, including ones only partly inside it -- draw a (usually thin) box that CUTS THROUGH the edges you want, even if it doesn't fully contain them.

```
a = cube([20, 20, 20])
show(a, alpha=0.2)
probe = translate([-10, -10, 8], box([20, 20, 4]))
show(probe, alpha=0.15, color='red') # eyeball/adjust probe first
picked = edges_in_box(a.edges(), probe)
show(sol := fillet(a, picked, 1))
```

Pass the SAME list you'd index into for `fillet()/chamfer()` -- almost always `shape.edges()` itself, unfiltered -- and the returned list is already the plain list of Edge objects `fillet()/chamfer()` expect, no further indexing needed. Combine with the other `filter_by()/sort_by()/label_edges()` techniques as needed -- e.g. ``edges_in_box(a.edges(), probe, mode="crossing")`` still returns a real ShapeList-friendly list, so ``[e for e in edges_in_box(...) if e.length > 5]`` works fine on top.

label_edges()

```
label_edges(edges, text_size=1, h=0.1)
```

Exposed to scripts | backend/brep.py:6555 | Used in: —

Labels each edge in ``edges`` with its own index (0, 1, 2, ...) as small flat text at that edge's midpoint -- the edge-selection counterpart of `track_points()` (same idea, indices instead of point numbers), for exactly the workflow `fillet()/s/max_fillet_radius()`'s own docstrings point at: figuring out which index(es) into `shape.edges()` are the ones you actually want, before calling `fillet(shape, [shape.edges()[i] for i in [...]], radius)` -- no more guessing indices blind or filleting the whole shape to see what happens.

Pass the SAME list you'd index into for `fillet()/chamfer()` -- almost always `shape.edges()` itself, unfiltered, so the numbers shown line up with `shape.edges()[i]`:

```
a = cube([20, 20, 20])
show(a, alpha=0.2)
show(label_edges(a.edges()), 'red')
# read the indices you want off the viewer, then:
show(fillet(a, [a.edges()[i] for i in [3, 7, 11]], 2))
```

If you pass an already-filtered/sorted/sliced selection instead (e.g. `a.edges().filter_by(Axis.Z)`), the numbers are just that selection's own 0-based position in the order given -- NOT the original `shape.edges()` index -- so that's only useful once you've already narrowed a group down and want to double check which ones ended up in it, not for picking indices back out of the full `shape.edges()` list.

``text_size`/`h`` control the label size/thickness, same meaning as `track_points()`'s own parameters -- there's no automatic scaling to your model's size, adjust by hand. Combine with `show_edges(edges)` to see the actual edge curves highlighted alongside the numbers:

```
show(show_edges(a.edges()), 'blue')
show(label_edges(a.edges()), 'red')
```

label_faces()

```
label_faces(faces, text_size=1, h=0.1)
```

Exposed to scripts | backend/brep.py:6605 | Used in: —

Labels each face in ``faces`` with its own index (0, 1, 2, ...) as small flat text at that face's own center -- `label_edges()`'s counterpart for face selectors, for the exact same workflow: figure out which index(es) into `shape.faces()` are the ones you want before handing them to something that takes a face selection (e.g. an `offset()`'s ``openings=`` argument, or picking a face to build a workplane/sketch on) instead of guessing blind.

Pass the SAME list you'd index into afterward -- almost always `shape.faces()` itself, unfiltered, so the numbers shown line up with `shape.faces()[i]`:

```
a = cube([20, 20, 20])
show(a, alpha=0.2)
show(label_faces(a.faces()), 'red')
# read the index you want off the viewer, then e.g.:
hollow = offset(a, -2, openings=[a.faces()[i] for i in [4]])
```

If you pass an already-filtered/sorted/sliced selection instead (e.g. `a.faces().sort_by(Axis.Z)[-1:]`), the numbers are just that selection's own 0-based position in the order given -- NOT the original `shape.faces()` index -- same caveat as `label_edges()`.

``text_size`/`h`` control the label size/thickness, same meaning as `label_edges()`'s/`track_points()`'s own parameters -- no automatic scaling to your model's size, adjust by hand.

seam_edges()

```
seam_edges(solid, a, b, tol=0.1, samples=3, mesh_tol=None)
```

Exposed to scripts | backend/brep.py:8533 | Used in: —

Finds the edges of ``solid`` that lie on BOTH ``a``'s and ``b``'s own surfaces within ``tol`` -- the genuine seam where two solids' surfaces crossed to produce ``solid`` (typically `solid = intersection(a, b)`, but works for `union()`/`difference()` results too). Generalizes `24_mouse_body_loft_sphere_intersection.py`'s own `_on_sphere()` trick -- which only worked because its second solid was a perfect analytic sphere() and could use a closed-form distance-from-center formula -- to ANY pair of solids, sculpted/ non-primitive ones included, by testing real surface distance instead of a formula.

Every edge of ``solid`` is, by construction, either (1) leftover boundary from ``a`` alone (trimmed by ``b``), (2) leftover boundary from ``b`` alone (trimmed by ``a``), or (3) a genuinely NEW edge that only exists because ``a``'s surface crossed ``b``'s. Category (3) is the real seam, and is exactly the edges that sit close to BOTH surfaces at once -- edges from (1)/(2) only ever sit close to one.

PERFORMANCE, and why this ISN'T built on `Shape.distance_to()`: ``solid.edges()`` can easily run into the thousands for anything built from many-station ruled lofts, and each call to `.distance_to()` against a real, complex solid re-pays that solid's OWN internal setup cost from scratch (build123d/OCCT don't cache anything between calls) -- confirmed directly in this app: even with a cheap single-point pre-filter added first, checking a few thousand edges against two complex multi-hundred-face solids still took over 10 minutes. Tessellating ``a`` and ``b`` into a triangle mesh ONCE instead (`to_mesh()`, the same tessellation this app's own viewer already relies on) and doing every distance check as a vectorized numpy POINT-TO-TRIANGLE distance against that fixed mesh is a completely different cost shape: one bounded tessellation pass per solid, then every subsequent query is microseconds instead of the 100ms+ a single real OCCT `distance_to()` call was costing here. On a solid with many edges, even that per-point numpy check adds up if it's still done one Python call per point (confirmed directly in this app to make `seam_edges()` slow again once edge counts got large) -- `_points_to_mesh_min_dist_batch()` above batches EVERY candidate point (all edge midpoints in one call, then every sample point from the surviving candidates in another) into just a handful of vectorized calls total, not one per point.

This measures distance to the nearest point on the tessellated surface (true point-to-triangle distance, not just nearest mesh VERTEX -- an earlier version of this function used nearest-vertex, which is a much worse approximation than it sounds: `to_mesh()` only adds vertices where curvature actually demands them, so a genuinely FLAT stretch of a shape (a swept prism's own flat side panel, for instance) can tessellate into just one or two large triangles with vertices far apart -- a point sitting exactly ON that flat face, dead center of a triangle, could easily be much farther than ``tol`` from the NEAREST VERTEX even though its true distance to the surface is zero. Confirmed directly in this app: nearest-vertex caused `seam_edges()` to come back completely empty on exactly this kind of shape. Real point-to-triangle distance doesn't have that failure mode -- it's accurate to the tessellation's own chordal tolerance everywhere, flat or curved, matching what ``mesh_tol`` below actually promises.

Samples ``samples`` points along each edge; an edge only qualifies if EVERY sampled point is within ``tol`` of both ``a``'s and ``b``'s mesh (each edge's own midpoint is checked FIRST, alone, as a cheap reject before the full ``samples``-point pass). ``tol`` needs to be loose enough to absorb whatever construction tolerance ``a``, ``b``, or ``solid`` itself carries if any of them were built from discretized data (ring lofts, offset surfaces, etc) rather than pure analytic primitives -- start around 0.1-1.0 and widen if this comes back empty; narrow it if it's pulling in edges that clearly aren't the seam.

``mesh_tol``: the tessellation tolerance `to_mesh()` uses for ``a`/`b`` -- defaults to ``tol / 2`` (fine enough that the nearest-mesh-vertex approximation stays well inside ``tol`` of the true surface almost everywhere) if left as `None`. Pass a coarser number explicitly for extra speed on a very large solid if ``tol`` is already generous.

Two direct uses:

```
seam = seam_edges(sol, s1, b_2)
mouse = fillet(sol, seam, 8)                # real fillet(),
                                           # if the seam
```

```

# chains into
# clean loops

spine = sample_curve(seam[0], 100)
f3 = convert_3lines2fillet(line_a, line_b, spine, closed_loop=1)
# or feed ONE
# offset-curve
# ribbon
# technique,
# when the
# seam is too
# fragmented
# for fillet()
# to walk as
# one loop

```

UNVERIFIED against a real build123d/OCPC install -- to_mesh()/ shape.tessellate() is documented, standard build123d Shape API, but this exact function hasn't been run here. The point-to-triangle distance math itself (_point_to_mesh_min_dist(), above) IS verified directly -- tested standalone against known cases (point over a triangle's face/edge/vertex regions, across a multi-triangle mesh) with exact expected distances back in every case.

seq()

```
seq(sec)
```

Exposed to scripts | backend/brep.py:1078 | Used in: —

Consecutive [p_i, p_{i+1}] pairs from `sec`, wrapping the last point back to the first -- e.g. seq([0,0],[10,0],[10,10]) -> [[0,0],[10,0],[10,10],[0,0]]. Works on plain values too: seq([1,2,3]) -> [[1,2],[2,3],[3,1]]. A small building block several of the other functions below (and your own scripts) are built on.

shell_from_faces()

```
shell_from_faces(*faces, tol=1e-06)
```

Exposed to scripts | backend/brep.py:12897 | Used in: —

Sews several B-rep Faces that share edges into ONE connected Shell -- the same OCCT sewing solid_from_faces() uses, but stops there instead of also closing the result into a watertight Solid.

Use this (not solid_from_faces()) whenever the faces you have DON'T necessarily bound a closed volume on their own -- e.g. a bunch of adjacent faces picked straight off an existing shape (say, a cavity's flat wall plus the fillet blends around it) that you just want stitched into one real multi-face surface, typically to hand straight to thicken() to build a matching tool/slider body:

```

f1 = [c.faces()[i] for i in [11, 12, 9, 8, 10, 13, 14, 2, 3, 5, 7, 6, 1, 4]]
wall = shell_from_faces(f1)
slider = thicken(wall, 10)
show(slider)

```

Don't reach for union() here instead -- union() boolean-FUSES shapes (meant for solids, not bare Faces) and, on a large set of Faces that are exactly edge-adjacent like these (never a genuine volumetric overlap), it's exactly the fragile case union()'s own docstring already warns about: it can come back as a Compound rather than a clean single Face/Shell, and thicken()/other surface-only tools then fail on it (e.g. "'Compound' object is not subscriptable") since a Compound is a looser, less specific grouping than the Shell they actually need.

`tol` (default 1e-6): the sewing tolerance itself -- see _sew_faces_to_shell()'s own docstring for the full explanation, but in short: a LARGE face selection (hundreds+, e.g. faces_facing()'s own output) that's already been through a loft/boolean/face-filter pipeline can carry real floating-point drift bigger than 1e-6 between two faces' "same" shared edge -- sewing that tight doesn't raise for it, it just leaves that seam unmerged, which then shows up downstream as far too many "free" boundary edges (e.g. from close_faces_to_plane()/faces_within_boundary()) even though the Shell itself still renders looking perfectly fine. If a function built on top of this Shell's own free edges is finding way more of them than the true outer boundary should have, re-sew with a looser `tol` (start small -- 1e-4 to 1e-2 depending on your model's scale --

and increase only if needed) BEFORE running that function again, rather than trying to bridge the resulting mess downstream.

Raises whatever build123d/OCCT itself raises if the given faces don't share edges cleanly enough to sew into one connected shell at all, even at the given `tol` (a real gap, an overlap, or genuinely mismatched edges).

show_edges()

```
show_edges(edges, radius=0.3, samples=12)
```

Exposed to scripts | backend/brep.py:6498 | Used in: —

Turns an edge selection into a visible "beaded string" of small spheres following each edge's own curve (straight or curved -- e.g. a cylinder's round rim comes out as a curved string of beads, not a straight line between its endpoints) -- a debug/visualization aid for confirming EXACTLY which edges a selector expression picked out (see fillet()'s docstring for selector examples) before actually handing them to fillet()/chamfer(). The viewer has no click-to-identify-a-B-rep-edge tool yet (its picking tool works on the tessellated display mesh, not the exact underlying topology), so this is the practical way to check a selection visually in the meantime: show it alongside your real model in a different color.

```
edges = a.edges().filter_by(Axis.Z)
show(a, alpha=0.3)                # the model, faded
show(show_edges(edges), c="red")   # just the selected edges, highlighted
# once the highlighted edges look right:
r = fillet(a, edges, 3)
```

`edges` is exactly what you'd pass to fillet()/chamfer() -- e.g. a.edges(), or a filtered/sorted/sliced selection of it.

`radius` controls how thick the beads are -- there's no automatic scaling to your model's size, so adjust it by hand for a very large or very small model (the default (0.3) is a reasonable starting point for a model with dimensions in the tens, not hundreds or fractions of a unit). `samples` is how many points are sampled along each edge's own curve (more = a smoother-looking curve for a rounded edge, at the cost of more spheres -- doesn't affect a straight edge's appearance either way).

REAL PERFORMANCE NOTE: beads are grouped with a lightweight Compound() rather than union()'d together, same fix/reasoning as points()'s own docstring -- these beads are disjoint, disposable debug markers that never need to be a single watertight solid, so chaining a real boolean union() across potentially hundreds/ thousands of them (one per sample point per edge -- a large edge selection, e.g. straight off seam_edges(), easily produces a lot of beads) was pure wasted CSG work, and confirmed directly in this app to make show_edges() itself the slow part of a script rather than whatever edge-finding it was meant to just double-check.

slider_from_faces()

```
slider_from_faces(sol, faces, direction=(0, 0, 1), inset=0.1, length=None, hull=False,
exact=False, grid_res=None)
```

Exposed to scripts | backend/brep.py:7556 | Used in: —

Builds a mold slider/core INSERT with the NEGATIVE (opposite, complementary) contour of `sol`'s own surface -- a genuine SOLID PRISM built from the flattened, slightly-shrunk FOOTPRINT of `faces`, extruded `length` units along `direction`, with `sol`'s own material then carved OUT of it (difference(tool, sol)): the prism's own material everywhere EXCEPT where `sol` occupies that same space. That's deliberately NOT the same as a slice of `sol`'s own material (intersection(sol, tool), which just looks like a chunk of `sol` itself, confirmed directly in this app to be the wrong shape for an actual tooling insert) -- a real mold slider/core is a separate piece of tool steel with a cavity/relief cut into it matching the INVERSE of the part surface it forms, not a copy of the part's own material.

Instead of stitching `faces`' exact BOUNDARY into a closed solid by hand (close_faces_to_plane()'s approach -- needs every free edge to chain into exactly one loop, and fails outright on a boundary riddled with small holes from facet-level noise, e.g. faces_facing() right at its own angle cutoff on a fine mesh), this computes the real AREA covered by `faces` once flattened: by default, it tessellates `faces` (to_mesh(), one fast call), flattens every mesh vertex into the footprint plane's own (u, v) coordinates, and RASTERIZES every triangle onto a fine boolean occupancy grid (pure numpy -- see

`_rasterize_uv_triangles()`) -- the union of many triangles' areas, computed as a raster fill instead of thousands of individual OCCT boolean calls. The grid's own boundary is then traced back into a real 2D polygon (`_trace_grid_boundary_loops()`), and that traced outline -- a real right-angle STAIRCASE wherever the true boundary isn't grid-aligned -- is smoothed back down with Douglas-Peucker simplification (`_dp_simplify_closed_loop()`) so the stairstep pattern doesn't itself become visible cut geometry in the final tool. It's a real areal union either way, so genuine concavities come through correctly (an L-shaped cavity stays L-shaped, not ballooned out to its own convex hull), same as projecting/fusing real B-rep faces would, but without paying OCCT's fixed per-call overhead over however many thousand faces ``faces`` has (confirmed directly in this app: that per-call overhead is exactly what made the real-projection route -- still available as ``exact=True``, see below -- too slow on a ~2000-face selection). Only the SINGLE LARGEST traced loop is kept as the footprint -- any smaller loops (real small holes from facet-level noise, e.g. `faces_facing()` right at its own angle cutoff) are dropped rather than carried through as holes in the tool, since a slider/core is usually meant to be solid material anyway. If you need holes preserved exactly, or fully exact (non-rasterized) geometry, use ``exact=True``.

That footprint is shrunk inward by ``inset`` (so the cutting tool never touches ``sol``'s own real surface exactly -- a small deliberate gap, not a precision boundary) and extruded into a real, CLOSED solid prism -- a plain `intersection()/difference()` against that solid tool is a much better-posed operation than `split()` against an open/unbounded surface: no ambiguity about which "side" is which, and ``sol``'s own already-valid geometry (not the exact patch boundary) is what actually determines the result either way.

```
f1 = faces_facing(sol.faces(), [0, 0, 1])
slider, rest = slider_from_faces(sol, f1, [0, 0, 1])
show(slider)
```

``hull=True`` uses the FASTEST, always-simple CONVEX HULL of ``faces``' own vertices instead -- a deliberate trade of precision (a slider with a convex outline, not hugging any real concavity) for the cheapest possible geometry (no rasterization or mesh work at all).

``exact=True`` uses the ORIGINAL real-B-rep-projection route (every one of ``faces``' own Faces projected via OCCT's own `Face.project_to_shape()`, fused with `union_multi()`) instead of the rasterized default -- exact analytic geometry and real hole preservation, at real per-face OCCT cost; reach for this only on a small/moderate face selection, or when you need the holes kept.

``grid_res``: the rasterization grid's cell size (model units), only used by the default (non-hull, non-exact) path. Defaults (None) to an automatic size scaled to ``faces``' own bounding-box diagonal, capped so the grid never exceeds ~1.5M cells regardless -- the traced outline is then simplified (Douglas-Peucker, tolerance tied to this same resolution) to remove the grid's own stairstep error, so the final outline comes out clean regardless of ``inset``. Pass a smaller number for a more precise outline on a very large/flat selection, or a larger one to speed up an extremely fine/dense mesh further.

``length``: how far the prism extends along ``direction``, split evenly on both sides of the footprint's own flat plane (so the tool is guaranteed to fully clear through ``faces``' own local relief on whichever side ``sol``'s material actually sits, without needing to know that up front). Defaults (None) to 4x ``faces``' own bounding-box extent SPECIFICALLY ALONG ``direction`` -- not ``sol``'s whole bounding box, and not ``faces``' full 3D diagonal either (a wide, shallow patch has a diagonal dominated by its own XY footprint, not its actual depth along the pull direction, and using that instead produced a needlessly long prism too) -- so the returned insert's length stays proportionate to how deep the local feature actually is, not how wide its footprint is or how big ``sol`` itself is. Give it an explicit number instead whenever you need a specific real backing thickness for the physical insert.

``inset`` (default 0.1): how far to shrink the flattened footprint inward before extruding it into the prism -- start small (0.05-0.5 depending on your model's scale) and only increase if the result still catches exactly on ``sol``'s own real boundary somewhere.

``faces`` accepts either a plain list of Faces or a single Shape (Shell/Solid/Compound) with its own ``faces()``.

UNVERIFIED against a real build123d/OCPT install (see this module's own header docstring) for the parts that call `build123d/OCCT` (`to_mesh()`, `polygon()`, `offset()`, `linear_extrude()`, `intersection()`, `difference()`, and the ``hull=True`/`exact=True`` paths). The rasterization/grid-tracing math itself (`_rasterize_uv_triangles()`, `_trace_grid_boundary_loops()`, `_loop_area_2d()`, `_simplify_collinear_loop()`) IS independently verified -- pure

numpy/Python, tested directly against known geometric cases before being wired in here.

Returns `(slider, rest)`.

vertex()

```
vertex(point)
```

Exposed to scripts | backend/brep.py:8322 | Used in: —

A single real B-rep Vertex at `point` ([x,y] or [x,y,z]) -- build123d's own native Vertex. Mainly useful as a loft() section: passing a Vertex as the FIRST and/or LAST section (instead of another Face) tells loft() to taper smoothly to a single point there instead of blending into another ring -- the standard, numerically robust way to cap a lofted stack of rings with a genuine rounded/pointed tip, instead of feeding loft() a nearly- degenerate near-zero-size ring (real curve/surface fitting -- interp_spline()/cap_wire()/Face-building in general -- can easily choke on a ring that's collapsed down to a tiny cluster of near-coincident points; "BRep_API: command not done" is a typical symptom of exactly that).

```
rings = [translate([0, 0, h], circle(r)) for h, r in
          [(2, 8), (8, 8)]]
show(loft([vertex([0, 0, 0]), *rings, vertex([0, 0, 10])]))
```

wrap_around()

```
wrap_around(sec, path)
```

Not yet exposed to scripts | backend/brep.py:3366 | Used in: —

Wraps the 2D/3D point list `sec` around the 3D `path` -- ocad.py's own wrap_around(), ported onto plain numpy. NARROW, special-purpose geometry, same restriction as the original: `path` must run roughly in the +Y direction, and `sec`'s own points must sit in the +Y quadrant (its own Y coordinate is read as "distance travelled along path", X as "offset sideways", Z as "height").

Import, Export & Drawings (8)

export_brep()

```
export_brep(shape, path)
```

Exposed to scripts | backend/brep.py:13227 | Used in: —

Writes `shape` to a native OCCT BREP file -- exact geometry like export_step(), but BREP is OCCT's own format rather than the cross-CAD-vendor ISO 10303 standard STEP is. Useful mainly for round-tripping back into build123d/OCCT itself (import_brep()) without any STEP-specific overhead; use export_step() if the file needs to open in a different CAD program.

export_drawing_svg()

```
export_drawing_svg(shape, path, part_name=None, author=None, date=None, views=None,
n_samples=48)
```

Exposed to scripts | backend/brep.py:13894 | Used in: —

Writes a complete engineering-drawing SVG sheet for `shape` (or a list of shapes) straight to `path` -- FRONT/TOP/RIGHT/ISO views (hidden edges dashed) laid out on one bordered sheet with a title block, in one call:

```
export_drawing_svg(part, "bracket_drawing.svg", part_name="Bracket")
```

Just technical_drawing_views(shape, views, n_samples) followed by drawing_layout.build_sheet_svg() -- see both of those for exactly what each half does/assumes (technical_drawing_views() in particular has real caveats worth reading, since HLR-based drawings are new in this app and only checked against a MOCKED build123d so far -- see this module's own header docstring).

``part_name`/`author`/`date``: title block text; ``part_name`` defaults to ``path``'s own filename (without extension) if not given, ``date`` defaults to today.

export_step()

```
export_step(shape, path)
```

Exposed to scripts | backend/brep.py:13205 | Used in: —

Writes ``shape`` to a STEP file -- real B-rep, exact geometry, openable/editable in any real CAD program (SolidWorks, Fusion 360, FreeCAD, etc.), unlike anything `ocad.py`'s own mesh functions can produce.

export_stl()

```
export_stl(shape, path, tolerance=0.001, angular_tolerance=0.1, ascii_format=False)
```

Exposed to scripts | backend/brep.py:13216 | Used in: —

Writes ``shape`` to an STL file -- tessellated (faceted) on export, same idea as `to_mesh()` above but written straight to disk by OCCT rather than returned as `[V, F]`. `tolerance/angular_tolerance` are the same facet-density controls as `to_mesh()`'s.

import_brep()

```
import_brep(path)
```

Exposed to scripts | backend/brep.py:13420 | Used in: —

Reads a native OCCT BREP file (`.brep`) -- the exact round-trip counterpart to this app's own `export_brep()`, no STEP-specific overhead. Use `import_step()` instead for a file made by (or meant to open in) a different CAD program; BREP is OCCT's own format, not the cross-vendor ISO 10303 standard STEP is.

import_step()

```
import_step(path)
```

Exposed to scripts | backend/brep.py:13279 | Used in: —

Reads a STEP file (`.step/.stp`) from disk and returns it as a real B-rep Shape -- `build123d`'s own `import_step()`, exposed directly so a script can pull in a part made elsewhere (another CAD program, or a file this app itself wrote with `export_step()`) and keep working on it with every other function here: `union()/difference()` it against new geometry, `split()` it for a cross-section, `fillet()/chamfer()` its edges, `measure()` with `volume()/bb()`, or just `show()` it to look at what's inside.

Returns whatever `build123d`'s own importer produces -- usually a Compound wrapping one or more Solids (assemblies/multi-body STEP files come back as multiple solids inside that one Compound; call `.solids()` on the result to pull them apart if a later step, e.g. `fillet()`, needs a single Solid rather than the whole Compound -- same convention `split()`'s own docstring uses for "both").

This reads the file's real geometry (exact B-rep), unlike `SanPyCAD`'s mesh-based sibling app, which can only ever pull in a STEP file's **tessellated** (faceted) surface as a triangle soup -- reading it here keeps it exact, so a subsequent `split()/fillet()/offset()` stays exact too, not limited by however coarse the mesh version's triangles happen to be. A common way to "section" an imported part for a good look inside -- cut with a plane through the shape's own center, sized off its own bounding box so the cutting plane is always comfortably bigger than the part itself:

```
part = import_step("assembly.step")
mid = part.bounding_box().center()
span = max(part.bounding_box().size) * 2
cut_plane = plane([0, 1, 0], size=span, intercept=(mid.X, mid.Y, mid.Z))
half = split(part, bisect_by=cut_plane, keep="bottom")
show(half)
```

CONFIRMED real failure mode (real user report): some `.stp` files take a very long time to import and then raise ``AttributeError: 'NoneType' object has no attribute 'NbNodes`` (or the lowercase `'nbnodes'` some OCP builds use for the same internal attribute). This is the SAME OCCT/OCP fragility class already documented elsewhere in this module (see

union()'s and sweep_sec2path()'s own docstrings for two other, unrelated call sites that can hit it) -- a None internal curve/mesh-adaptor handle getting queried deep inside OCCT's own C++ layer, not a bug in this thin wrapper (`bd.import_step()` is called here with nothing else happening first). For import specifically it means the STEP reader's own geometry-healing/validation pass choked on something in the file -- most often a handful of degenerate faces (zero area, duplicate/near-duplicate vertices, non-manifold edges) left behind by whatever CAD system wrote it, especially likely on a large, complex, multi-body assembly (the "very long time" beforehand is that same healing pass working through everything else in the file first). There's no reliable way to detect or repair this from Python before it happens -- OCCT doesn't expose a "skip healing" or "tolerant read" switch through `build123d's import_step()` -- so this just re-raises with a clearer explanation instead of the bare `AttributeError`. Things that have fixed this for other, similarly affected files: re-opening and re-exporting the STEP file from its original CAD program with geometry repair/healing turned on, or opening it in FreeCAD and using its Part > Check geometry / "Refine shape" + re-export STEP, which often cleans up exactly this kind of degenerate geometry before OCCT ever sees it again.

Automatically runs the freshly imported shape through OCCT's own `ShapeFix_Shape` repair pass (see `_heal_imported_shape()` below) before returning it -- the SAME class of automatic healing FreeCAD (and most other CAD software) already applies on STEP import, which is the real reason a file that crashes THIS app's tessellation can still open "in a few seconds, completely clean" in FreeCAD: it's not that FreeCAD is reading different geometry, it's that FreeCAD silently repairs small defects (gaps, degenerate edges, slightly inconsistent face orientations) before ever handing the shape to its own mesher, and this app's raw `build123d import_step()` call didn't used to do that at all. This is a genuine best-effort fix, not just a clearer error message -- if it works, whatever was tripping the `NbNodes` tessellation bug may simply not be there anymore, and `show()/to_mesh()` should just work without needing the tolerance-override or lenient-fallback workarounds at all. Entirely fail-safe either way: if healing itself raises for any reason (e.g. this OCP build's `ShapeFix` module isn't importable, or the repair pass itself errors), this silently falls back to the original, unhealed shape rather than turning a healing attempt into a NEW failure -- see `_heal_imported_shape()`'s own docstring.

technical_drawing_views()

```
technical_drawing_views(shape, views=None, n_samples=48, custom_views=None,
sections=None, time_budget_s=None, include_hidden=True)
```

Exposed to scripts | backend/brep.py:13686 | Used in: —

Computes hidden-line-removed 2D views of `shape` (or a list of shapes -- e.g. everything a script `show()`'d) from the standard FRONT/TOP/RIGHT/ISO camera angles (see `_drawing_view_directions()`), and returns them as plain JSON-safe data:

```
{"FRONT": {"visible": [[[x,y],...], ...], "hidden": [...]},
 "TOP": {...}, "RIGHT": {...}, "ISO": {...}}
```

This is the geometry half of an engineering drawing -- SHEET LAYOUT (border, title block, arranging these 4 views on one page) is deliberately NOT done here; that's `drawing_layout.py's build_sheet_svg()`, which takes exactly this dict as input and has zero `build123d/OCCT` dependency of its own (see its module docstring for why that split exists). `export_drawing_svg()` below does both steps for you in one call, if a plain SVG file is all you need:

```
show(part)
views = technical_drawing_views(part)
# -- or, for the finished sheet directly --
export_drawing_svg(part, "part_drawing.svg", part_name="Bracket")
```

`shape`: a single `Shape`, or a list of `Shapes` (e.g. `[a, b]` if a script `show()`'d more than one separate part) -- each shape's own visible/hidden edges for a given view are computed independently and simply concatenated into that view's polyline lists. This is a real limitation worth knowing, not just a simplification: if 2 shapes actually OCCLUDE each other from some camera angle (one genuinely hides part of the other), that occlusion is NOT accounted for -- each shape's own hidden-line-removal only knows about itself. `union()`ing shapes that should occlude each other into one shape first, before calling this, avoids that.

`views`: which views to compute -- defaults to all 4 ("FRONT"/"TOP"/"RIGHT"/"ISO"); pass e.g. `views=["FRONT","TOP"]` to only compute (and pay the OCCT computation cost for) a subset.

``n_samples``: how many points each Edge (straight or curved alike) is sampled into along its own length -- higher = smoother curves in the output, at the cost of larger polyline lists. A straight edge doesn't need many, but this doesn't try to detect straight- vs-curved automatically and skip sampling those (unlike `projected_area()`, which does exactly that via `Edge.geom_type` -- see its own docstring -- because there the segment COUNT itself was the actual problem; here it's the visual SMOOTHNESS of curves that matters, so keeping every edge uniformly sampled, straight or not, is simpler and one less thing to get subtly wrong for a cosmetic win). 48 was chosen over a lower default (this used to default to 16) after real-world use showed circles/fillets coming out visibly faceted in the drawing tab at that resolution -- especially noticeable on larger circles, where each of the 16 facets spans more actual distance. Pass a lower `n_samples` yourself if a very edge-dense model renders too slowly at 48 and you'd rather trade smoothness for speed on a case-by-case basis.

``custom_views``: extra views from an ARBITRARY angle, beyond the 4 fixed presets -- a list of `{"name": str, "direction": [x,y,z], "up": [x,y,z]}` dicts (see `_custom_view_direction()`). This is what the Drawing tab's "Add View" button (captured live from the 3D viewer's own camera orientation) feeds in. Each name becomes its own extra key in the returned dict, laid out alongside FRONT/TOP/ RIGHT/ISO by `drawing_layout.py`.

``sections``: user-defined cross-section views -- a list of `{"name": str, "axis": "x"/"y"/"z", "offset": float, "keep": "positive"/"negative"}` dicts (see `section_view()`). Each one cuts every shape at that plane and looks straight at the freshly exposed cut face, rather than reusing FRONT/TOP/RIGHT/ISO's fixed uncut angles -- this is how you get a section through a SPECIFIC area of the part instead of always the same 4 outside views. A cutting plane that misses a shape entirely (see `section_view()`'s own docstring -- expected on a disconnected multi-body assembly, not a bug) is reported back as that view's own "error"/"warning" key rather than failing the whole request.

``time_budget_s``: an optional total wall-clock budget (seconds), shared across every view this call would otherwise compute (default 4 + every `custom_views/sections` entry) -- once already- spent time exceeds it, any view that HASN'T started yet is skipped (marked with an "error" explaining why) rather than started. This exists for a real, reported problem: on a large/complex imported STEP file, each view is a completely independent, genuinely expensive OCCT HLR computation (see `_hmr_polylines()`'s own docstring) with wildly different costs depending on that view's own silhouette complexity -- an ISO view (3/4 angle, lots of overlapping geometry) can easily cost far more than a FRONT/TOP view of the same part. Without a budget, one slow view blocks every view queued after it, even ones that would have finished in seconds -- a user waiting on the Drawing tab sees nothing at all until literally everything is done. ``None`` (the default) means no budget, same behavior as before this parameter existed -- `export_drawing_svg()` callers who are fine waiting as long as it takes should leave this unset. Does NOT (cannot) interrupt a view that's ALREADY running -- Python has no safe way to abort a call already inside OCCT's own C++ code -- so a single view that is itself slower than the whole budget still runs to completion; this only ever prevents starting MORE work on top of that.

``include_hidden=False`` drops every view's "hidden" (dashed) edges entirely -- see `_hmr_polylines()`'s own docstring for exactly what this does and doesn't save (real, but partial: skips sampling/ returning hidden edges, not OCCT's own underlying visibility computation, which runs regardless either way).

text()

```
text(txt, size=10, h=1, font='Arial', center=False)
```

Exposed to scripts | [backend/brep.py:7897](#) | Used in: —

A real, extruded 3D text solid -- build123d's own native `Text()` (real glyph outlines rendered from an installed system font, not a flat placeholder), pre-extruded by ``h`` in one call, the same convenience real OpenSCAD's own `text()+linear_extrude()` two-step idiom gives you. `ocad.py` never had a real `text()` of its own (its own `text()/fo()` only ever emitted raw OpenSCAD source, for OpenSCAD itself to render later) -- this is a genuine new primitive here, not a line-for-line port.

``size`` is the font size (roughly the cap height, in model units). ``center=False`` (default) leaves the text's own baseline-left origin at [0,0,0] (matches real OpenSCAD's `text()` default `halign="left"/ valign="baseline"`); ``center=True`` centers the text on the origin on both axes instead -- handy for labels you're about to position by their own middle (see `track_points()` below).

Needs a font actually installed wherever this runs -- ``font`` defaults to "Arial", which may not exist on every system (especially Linux); pass a font name that IS installed there instead if you hit a missing-font error.

`h<=0` skips the extrude step entirely and returns the flat 2D text sketch instead (a real Face per glyph, not a solid) -- the real per-label cost here is rendering the font glyphs themselves (`bd.Text()`), not the extrude on top, but skipping the extrude still saves a real operation per call, which matters if you're calling `text()` many times (see `track_points()`'s own perf note). Still `union()/translate()/etc`-able like any other Shape either way.

```
show(translate([0, 0, 0], text("Hello", size=10, h=2)))
```

Application-Specific Helpers (4)

`pick_key_stations()`

```
pick_key_stations(rows, n_keep=10)
```

Exposed to scripts | [backend/brep.py:5284](#) | Used in: —

Thins an ordered list of `rows` (same-length point-list rings, one per sweep station -- exactly `surface_from_closed_rings()`'s own input shape) down to `n\_keep` stations, chosen where the cross-section is changing SHAPE fastest, not evenly spaced by station index.

WHY: `surface_from_closed_rings()` fits one global, genuinely INTERPOLATING surface through every station you hand it (see its own docstring) -- feed it many uniformly-resampled stations off an organic, freely-varying shape and that interpolation is prone to rippling between them, the same Runge's-phenomenon-style overshoot a high-degree global Bezier curve shows through many control points (see `solid_line_vector()/bezier()`'s own notes elsewhere in this project). Cutting the STATION COUNT helps, but picking evenly-spaced stations wastes several of them re-covering a stretch that barely changes (e.g. a long flat back panel) while starving the one place the shape actually turns a corner (e.g. a headrest-to-neck transition) -- exactly where the interpolation most needs a nearby constraint to stay on track.

This instead measures how much each ring differs from the one before it (summed point-to-point distance -- rows are already same-length and consistently ordered/oriented by construction, so corresponding indices already line up, no re-alignment needed), then picks stations evenly spaced along that CUMULATIVE CHANGE curve instead of evenly spaced by index -- clusters more stations where the shape moves fast, fewer where consecutive rings are nearly identical. The first and last stations are always kept (the two ends `surface_from_closed_rings()`'s own open boundary needs).

```
sol2 = [ o.equidistant_pathc(o.bspline_closed(o.homogenise(p, 30), 3, 30), 30)
        for p in sol1]                # smooth each ring only
keep = pick_key_stations(sol2, n_keep=10)  # <- pick stations here
sol2 = [sol2[i] for i in keep]           # instead of a uniform
                                         # bspline_open/
                                         # equidistant_path
                                         # resample pass

sol2 = surface_from_closed_rings(sol2, tol=.1)
```

Deliberately does NOT also smooth/resample the station (rail) direction itself the way an `o.bspline_open()+o.equidistant_path()` pass would -- `surface_from_closed_rings()`'s own smoothed mode already interpolates smoothly BETWEEN whichever real stations you hand it, so pre-smoothing the rail first just means feeding it a denser, already-interpolated input to re-interpolate, which is both slower and does nothing to reduce ripple. Feed it the real (ring-smoothed only) station data directly.

Returns a plain sorted list of indices into `rows` (not the rows themselves), so you can also use it to select from a DIFFERENT but same-length/same-order list if needed (e.g. picking the same stations out of an unsmoothed variant for comparison).

`slider_housing()`

```
slider_housing(shape, direction, side='top', plane_point=None, margin=10.0, backing=10.0,
slider=None, angle_tol=80.0, tolerance=None, weld_tol=None, wall_points=80,
**slider_kwargs)
```

Exposed to scripts | [backend/brep.py:11579](#) | Used in: —

Builds a mold-base housing, contoured to a slider's own true edge, with a pocket cut into it matching the slider's exact profile (`mesh_cap_ribbon()`'s own output) -- the housing that HOLDS a slider insert, standard mold-tooling layout: a block carries the slider, which itself carries `shape`'s real curved surface detail.

Call this twice, once per side, at the SAME `plane\_point`, to get a matching pair:

```
top_h = slider_housing(sol, [0, 0, 1], side="top", plane_point=[0, 0, 40])
bot_h = slider_housing(sol, [0, 0, 1], side="bottom", plane_point=[0, 0, 40])
show(top_h, 'red', alpha=0.5)
show(bot_h, 'blue', alpha=0.5)
```

THE TECHNIQUE (an exact-edge B-rep port of the working ocad reference script this was built from --

`l1/l2/l3/l4/nlos/offset_3d/surface_line_vector/swp_c`): the housing is built from FOUR rail curves, one point per station around the slider's own boundary, swept into a closed wedge-grid wall (`surface_from_points_grid_closed()` -- NOT a loft: that function rules a strip directly between each pair of adjacent closed rail curves and sews the strips into a shell, which is what actually lets this be periodic around the boundary with no seam, unlike ThruSections-based lofting which can't treat a station list as genuinely periodic -- see `solid_from_wedge_grid()`'s own history/docstring for the loft failures that approach hit on this exact shape): l1: the slider's own TRUE 3D boundary (`mesh_boundary_loop()` --

```
the exact-edge technique mesh_cap_ribbon() itself is built
on, read straight off `shape`'s mesh topology, not a
flattened silhouette), resampled to `wall_points` stations.
```

l2: l1 projected straight onto the parting plane (flattened,
same station indexing as l1).

l3: l2 grown outward by `margin`, in-plane (ocad.py's own

```
offset() -- a REAL polygon offset with self-intersection
trimming, not a naive per-vertex push, which does self-
intersect at ordinary margins wherever the boundary has any
tight concave bend -- confirmed on this exact shape).
```

l4: l1 pushed straight out to l3's own wall (l3 swept along

```
`direction` far enough to clear the slider's own farthest
point, then l1 projected onto that wall surface) -- same
height as l1, wall's own radius.
```

Each station's own closed wedge is [l1, l4, l3, l2, l1] (l1 repeated to close it, matching `swp_c()`'s own convention); lofting the whole ring of wedges gives a solid that hugs l1's own true edge exactly (no offset-margin gap right at the boundary the way a plain extruded block would have) while still reaching l3's flat outer rim, exactly at the parting plane -- the mating face for the OTHER housing half, so it's deliberately left flush there, not padded with extra backing (that would misalign the parting line).

WHY THAT GIVES A MATCHING PARTING LINE: l3 (and so the whole housing's own outer wall) is computed on the exact same plane basis `mesh_cap_ribbon()` itself uses, so two calls with the identical `plane\_point` compute the SAME wall both times -- the two housings' own parting faces match edge-for-edge automatically.

`slider`: an already-built slider (`mesh_cap_ribbon()`'s own output, e.g. `cap` from example 29) to cut the pocket for -- pass this when you already have one, so the pocket is EXACTLY that solid, not a freshly (and potentially slightly differently, if any parameters differ) rebuilt one. If omitted, this calls `mesh_cap_ribbon(shape, direction, side=side, plane_point=plane_point, angle_tol=angle_tol, tolerance=tolerance, weld_tol=weld_tol, **slider_kwargs)` itself.

`angle\_tol`/`tolerance`/`weld\_tol`: forwarded to `mesh_boundary_loop()` for reading the slider's own true edge (and to `mesh_cap_ribbon()` too, when building the slider internally) -- pass the SAME values you used to build `slider` if you built it yourself with non-default ones, so the wall traces the same boundary the pocket was actually cut from.

`margin`: how far the housing's own outer wall extends past the slider's own true boundary, at every point around it (not just at the widest point, the way a bounding rectangle would).

`backing`: how far l3's own wall surface is swept along `direction` before l1 is projected onto it for l4 -- needs to clear the slider's own farthest point along `direction` (computed automatically from the slider's own bounding box, not guessed) plus

this much extra, so the projection is always well-defined and the housing has solid material past the slider's own deepest point, not just barely touching it.

``**slider_kwargs``: forwarded to `mesh_cap_ribbon()` when building the slider internally (only used if ``slider`` isn't passed in) -- e.g. ``n_lines=``, ``points_per_line=``.

``wall_points``: how many evenly-spaced stations `l1` (and so every other rail) is resampled to -- the raw boundary loop follows the mesh's own triangle edges (hundreds of tiny, unevenly-spaced points), not a station count fit for a loft. Raise it if the housing's wall needs to hug a tight local detail more closely.

Returns the housing Solid (the wedge-grid wall with the slider- shaped pocket already cut out) -- NOT the slider itself; keep that separately (and pass it back in as ``slider`` if you build the housing more than once, to avoid rebuilding it each time).

thicken()

```
thicken(shape, amount, both=False)
```

Exposed to scripts | backend/brep.py:5414 | Used in: —

Thickens a Face (e.g. `surface_line_vector()`'s own output, or any Face at all -- planar or not) into a real solid, offset along its own surface normal at every point -- build123d's own native `thicken()`. Unlike `linear_extrude()`, which needs the extrusion direction spelled out yourself, this follows whatever direction the surface itself curves in point by point, which matters for a non-planar surface (`surface_line_vector()`'s ruled Face, most of the time) that has no single straight-line extrusion direction to begin with.

``amount``'s SIGN picks which side gets the material: positive thickens along the surface's own normal ("outside", from however it was built/wound), negative thickens against it ("inside") -- a Face's normal direction isn't always obvious up front, so if it comes out on the wrong side, just flip the sign and try again.

``both=True`` thickens in BOTH directions at once and fuses the result into one solid centered on the original surface -- note this means ``amount`` on EACH side (`2*amount`` total thickness across the surface), not ``amount`` split between the two.

```
a = rot('x90', polygon(cr2dt([...], s=10)))
wall = surface_line_vector(a, [0, 100, 0])
solid = thicken(wall, 2) # 2 units thick, one side only
shell = thicken(wall, 1, both=True) # 2 units thick total, centered
```

track_points()

```
track_points(line, text_size=1, h=0.1, step=1)
```

Exposed to scripts | backend/brep.py:8034 | Used in: —

Labels each point in ``line`` with its own index number (0, 1, 2, ...) as small real 3D text -- ocad.py's own `track_points()`, ported: the original generated raw OpenSCAD source `text()` calls, only usable through `fo()`; this returns one real, already-positioned Shape (every digit label grouped together) you `show()/color()` directly, no text-code step needed. Handy for figuring out which index in a point list corresponds to which point when debugging a curve/path/profile -- the numeric sibling of `points()` above (which just marks each point, without saying which is which).

```
pts = bezier([[0, 0], [5, 15], [15, 15], [20, 0]], 8)
show(color(polygon(pts), "lightgray", alpha=0.3))
show(track_points(pts, text_size=1.5), "blue")
```

``text_size`/`h`` are passed straight through to `text()` (font size, extrusion height). No `text_color` parameter here (unlike the original) -- same convention as `points()` above: pass a color to `show()/color()` yourself when you actually display the result.

REAL PERFORMANCE NOTE: rendering N real text labels is N real font-render-and-extrude operations no matter how the results get grouped afterward -- `Compound()` (used here instead of `union()`, see below) only removes an EXTRA, unnecessary cost on top of that, not this one. For a path with many points (a densely- sampled `cr2dt()` outline, say), the label count itself is usually the real bottleneck, so two direct levers: ``step`` (default 1, every point) -- e.g. `step=5` labels only

```
every 5th point, cutting the label count (and cost) by 5x.
Each label still shows its point's TRUE index in `line`, not
a renumbered 0,1,2,... count, so it stays meaningful even
```

skipped.

``h=0`` -- skips the extrude step per label (see `text()`'s own

note), rendering flat labels instead of solids -- meaningful savings if you don't need solid/printable labels, just something to look at.

Labels are grouped with a lightweight `Compound()` rather than `union()`'d together regardless -- these are disjoint, disposable debug labels that never need to be one watertight solid, so a real boolean fuse between them (the $O(N)$ chained-boolean cost documented in `p_line3d()`'s own docstring) was pure wasted CSG work on top of whatever building each label itself costs.

Alphabetical Index

Names marked with * are not yet exposed to scripts (see “Exposed vs. not-yet-exposed” on page 2).

ang	ang3points
ang_2lineccw	ang_2linecw
arc	arc_2p
arc_2p_3d	arc_3p
arc_3p_3d *	arc_edge
area	axis_rot_1
axis_rot_1_pts	bb
bb2d	bezier
box	bspline_closed
bspline_open	c23
c32	c_polygon
c_solid	cap_wire
chamfer	cir_2p
cir_3p	circle
close_faces_to_plane	closest_points_between_two_lines
cog	concave_hull *
contiguous_chains	convert_3lines2fillet
convex_hull	corner_n_radius_list
cosinewave	cp_3p
cp_arc *	cp_cir_3d *
cr2dt	cr3dt
cube	cw *
cylinder	d2r *
difference	distanceOfPointFromLine
draft_check	draft_check_markers
draft_check_points	edges_in_box
equation_of_plane	erode
erosion_series	export_brep
export_drawing_svg	export_step
export_stl	extend_arc2d *
extend_arc3d *	extrude_wave2path *
face_from_wire	face_islands
faces_facing	faces_in_box
faces_within_boundary	fill_face_holes
fillet	fillet3points
fit_pline2line	flip
group	helix
honeycomb	hull
i_p2d	import_brep

import_step	interp_spline
intersection	l_len
l_lenv	label_edges
label_faces	line2length *
line3d	line_as_unit_vector
line_as_vector	linear_extrude
loft	loft_fillet
max_fillet_radius	mesh_boundary_loop
mesh_cap_ribbon	mesh_cap_solid
mesh_split_surfaces	mesh_split_triangles
mid_line *	mid_point
min_d_points	mirror
mirror_line	mirror_point
mirror_surface	mixed_polygon
mixed_wire	nlos
normal_vector	o_3dc_tri
offset	offset_3d
offset_face	offset_shell
organic_fillet	p_line3d
perpendicularProjectionOfPointOnLine	pick_key_stations
pies1	planarize_ring
plane	plane_from_equation
plos *	plos_v *
plos_v_1 *	point_in_polygon
points	polygon
prism	project_curve_on_face
project_face_on_shape	projected_area
projected_area_mesh	psos
psos_n *	psos_v *
psos_v_1 *	r2d *
remove_extra_points	resize_solid
resize_solid_centered	rot
rot2d	rot_p
rotate	rotate_extrude
s_int1	sample_curve
scale	seam_edges
sec2vector *	section_view *
seg	shape_outline_raster
shell_from_faces	show_edges
sinewave	slice_mesh_to_rings
slider_from_faces	slider_housing
smooth_mesh	solid_from_closed_rings
solid_from_faces	solid_from_fillet

solid_from_fillet_closed	solid_from_mesh_slices
solid_from_wedge_grid	solid_line_vector
sort_points	sphere
split	square
surface_cap_solid	surface_from_2_waves
surface_from_closed_rings	surface_from_points_grid
surface_from_points_grid_closed	surface_line_vector
surface_normals *	surface_solid_closed_intersection
surface_solid_open_intersection	sweep_sec2path
swp_c *	tangent_arc
tcct	technical_drawing_views
text	thicken
to_mesh	to_points
to_triangles	track_points
translate	turtle2d
turtle3d	two_cir_tarc *
two_solids_intersection	union
union_fuzzy	union_multi
vertex	volume
wire_from_edges	wrap_around *
xrot	xrot4d
xyc	xzc
yrot	yrot4d
yzc	zrot
zrot4d	